

UNIVERSIDADE FEDERAL DO PAMPA

Nícolas Naves Rezende Faria

**G-FShield: Distributed and Observable Feature
Selection for Intrusion Detection in Cyber-Physical
Systems**

Alegrete
2026

Nícolas Naves Rezende Faria

**G-FShield: Distributed and Observable Feature
Selection for Intrusion Detection in Cyber-Physical
Systems**

Dissertation submitted to the Graduate Program in Software Engineering (PPGES) as a partial requirement for the degree of Master in Software Engineering.

Advisor: Prof. Dr. Silvio E. Quincozes

**Alegrete
2026**

Provisional catalog card

This dissertation version does not yet include the final catalog card issued by the library.
Replace this block with the validated version before the final submission.

Nícolas Naves Rezende Faria

G-FShield: Distributed and Observable Feature Selection for Intrusion Detection in Cyber-Physical Systems

Dissertation submitted to the Graduate Program in Software Engineering (PPGES) as a partial requirement for the degree of Master in Software Engineering.

Dissertation defended and approved on _____ of _____ of _____.
Examining committee:

Prof. Dr. Silvio E. Quincozes

Advisor
Unipampa

Committee member 1

Examiner
Unipampa

Committee member 2

Examiner
Unipampa

RESUMO

A seleção de características reduz a dimensionalidade de dados usados por sistemas de detecção de intrusão, mas implementações sequenciais dificultam distribuir a busca e inspecionar resultados intermediários. Esta dissertação apresenta o *G-FShield*, uma arquitetura distribuída e orientada a eventos para GRASP-FS aplicado a sistemas ciber-físicos. A proposta separa geração da lista restrita de candidatos e busca local, usa Apache Kafka para comunicação e persiste eventos, métricas, estados, identificadores e marcas temporais consultados por uma camada web.

Uma campanha reproduzível, limitada a dez dias, avaliou 24 configurações G-FShield, dois monólitos e um baseline com todas as 51 características. Cada braço teve oito sementes independentes, o mesmo particionamento estratificado de treino/validação/teste, Weka J48 3.8.6 como avaliador final e teto agregado de oito CPUs e 16 GiB. Foram obtidas 216 execuções válidas. A configuração ReliefF-VND-IWSSR, escolhida pela validação, alcançou F1 macro mediano de 0,9448 no teste com 11 características, redução dimensional de 78,4%; o baseline obteve 0,9378. Nenhuma das 72 comparações exploratórias ajustadas por Holm apresentou $p < 0,05$.

Os resultados demonstram integração arquitetural, rastreabilidade experimental e diferenças descritivas, não causalidade arquitetural, escalabilidade ou tolerância a falhas. Embora todos os braços formais usem F1 macro, os comparadores executam operadores distintos e a campanha não registrou uma linha do tempo cumulativa equivalente de candidatos nos monólitos; as classes *grayhole* e *normal* também permanecem mais difíceis. Tempo até o alvo sob algoritmo pareado, completude e custo da telemetria, recuperação, reprocessamento, escala horizontal e generalização requerem experimentos específicos.

Palavras-chave: seleção de características, detecção de intrusão, sistemas ciber-físicos, GRASP-FS, sistemas distribuídos, observabilidade.

ABSTRACT

Feature selection reduces the dimensionality of data used by intrusion detection systems, but sequential implementations make it difficult to distribute search and inspect intermediate results. This dissertation introduces *G-FShield*, a distributed, event-driven architecture for GRASP-FS applied to cyber-physical systems. It separates restricted-candidate-list generation from local search, uses Apache Kafka for communication, and persists events, metrics, states, identifiers, and timestamps queried through a web layer. A reproducible campaign, bounded to ten days, evaluated 24 G-FShield configurations, two monoliths, and an all-51-feature baseline. Each arm used eight independent seeds, the same stratified training/validation/test split, Weka J48 3.8.6 as the final evaluator, and an aggregate ceiling of eight CPUs and 16 GiB. The campaign produced 216 valid runs. The validation-selected ReliefF-VND-IWSSR configuration achieved median test macro-F1 0.9448 with 11 features, a 78.4% dimensionality reduction; the baseline achieved 0.9378. None of the 72 exploratory Holm-adjusted comparisons had $p < 0.05$.

The results demonstrate architectural integration, experimental traceability, and descriptive differences, not architectural causality, scalability, or fault tolerance. Although all formal arms use macro-F1, the comparators execute different operators and the campaign did not record an equivalent cumulative candidate timeline in the monoliths; the *grayhole* and *normal* classes also remain harder. Time to target under a matched algorithm, telemetry completeness and cost, recovery, reprocessing, horizontal scaling, and generalization require dedicated experiments.

Keywords: feature selection, intrusion detection, cyber-physical systems, GRASP-FS, distributed systems, observability.

LIST OF FIGURES

Figure 1 – Summary of the <i>G-FShield</i> architecture; the blank margin in the original file is removed only by the LaTeX inclusion.	21
Figure 2 – Generic architecture of the Distributed RCL Generation (DRG) stage.	24
Figure 3 – Concrete Distributed RCL Generation (DRG) instantiation with the four implemented metrics.	25
Figure 4 – Generic architecture of the Distributed Local Search (DLS) stage.	28
Figure 5 – Concrete Distributed Local Search (DLS) instantiation with the implemented controllers and operators.	29
Figure 6 – Deployment topology and execution boundaries of <i>G-FShield</i>	32
Figure 7 – Execution lifecycle of <i>G-FShield</i> , from the request to the states shown in the interface.	34
Figure 8 – Ingestion, persistence, and presentation pipeline for observed telemetry.	35
Figure 9 – Functional flow of the web client and its Application Programming Interface (API) queries.	36
Figure 10 – Backend services, data stores, and messaging integration.	37
Figure 11 – Test macro-F1 distributions for all 27 arms and eight seeds per arm.	46
Figure 12 – Median test macro-F1 for the 24 distributed configurations.	47
Figure 13 – Trade-off between median macro-F1 and dimensionality reduction of final solutions.	49
Figure 14 – Median per-class test F1 for the validation-selected distributed configuration and comparators.	51
Figure 15 – Observed CPU and memory by architecture under the common aggregate ceiling of eight CPUs and 16 GiB.	53
Figure 16 – Authentication screen.	62
Figure 17 – Overview dashboard.	63
Figure 18 – Performance screen.	64
Figure 19 – Algorithm comparison screen.	65
Figure 20 – Aggregated analytics activity.	66
Figure 21 – Analytics activity detail.	67
Figure 22 – Analytics topics.	68
Figure 23 – Persisted analytics feed.	69
Figure 24 – Run comparison screen.	70
Figure 25 – Comparison with selected seeds.	71
Figure 26 – Details of a specific run.	72
Figure 27 – Initial solutions.	73
Figure 28 – Consolidated outcomes.	74
Figure 29 – Execution progress.	75
Figure 30 – Best solutions.	76

Figure 31 – Execution settings.	77
Figure 32 – Dataset settings.	78
Figure 33 – Operations settings.	79
Figure 34 – Full reset dialog.	80
Figure 35 – Dataset list.	81
Figure 36 – User list.	82
Figure 37 – User editing.	83
Figure 38 – Poster for the platform demo video.	84
Figure 39 – Swagger documentation of the backend API.	85
Figure 40 – Frontend flow of the system.	86
Figure 41 – Backend architecture of the system.	87
Figure 42 – Generic DRG view with abstract data input.	88
Figure 43 – Complete architecture summary.	89

LIST OF TABLES

Table 1	– Summary of related work cited in this manuscript.	18
Table 2	– Architectural drivers, decisions, and limits.	23
Table 3	– Resources and concurrency of the local reference configuration.	38
Table 4	– Effective limits on the inspected server.	39
Table 5	– Final held-out test results and observed resources (medians of eight runs).	44

LIST OF ABBREVIATIONS AND ACRONYMS

- API – Application Programming Interface
- CPS – Cyber-Physical System
- DLS – Distributed Local Search
- DRG – Distributed RCL Generation
- GR – Gain Ratio
- GRASP-FS – Greedy Randomized Adaptive Search Procedure for Feature Selection
- IDS – Intrusion Detection System
- IEC 61850 – IEC 61850
- IG – Information Gain
- IWSS – Incremental Wrapper Subset Selection
- IWSSr – Incremental Wrapper Subset Selection with Randomization
- Kafka – Apache Kafka
- RVND – Randomized Variable Neighborhood Descent
- SU – Symmetrical Uncertainty
- VND – Variable Neighborhood Descent

CONTENTS

1	INTRODUCTION	12
2	THEORETICAL BACKGROUND	14
2.1	Cyber-Physical Systems and Intrusion Detection	14
2.2	Feature Selection	14
2.3	Greedy Randomized Adaptive Search Procedure for Feature Selection (GRASP-FS)	15
2.4	Distribution and Orchestration	15
2.5	Observability	15
2.6	Synthesis	15
3	RELATED WORK	16
3.1	Intrusion Detection in Cyber-Physical Systems	16
3.2	Feature Selection and GRASP-FS	16
3.3	Distributed Architectures and Observability	16
3.4	Identified Gap	17
3.5	Comparative Table	17
4	DEVELOPMENT	20
4.1	Overview and Input Contract	20
4.2	Architectural Drivers and Decisions	22
4.3	Distributed Restricted Candidate List Generation	23
4.4	Distributed Local Search	27
4.5	Event-Driven Orchestration	31
4.6	Persisted Telemetry and Web Layer	33
4.7	Reference Configuration and Reproducibility	38
4.7.1	Formal campaign manifest	40
4.8	Synthesis	41
5	EVALUATION	42
5.1	Experimental Protocol	42
5.2	Predictive Quality and Dimensionality Reduction	43
5.3	Time, Resources, and Statistical Interpretation	52
5.4	Observability and Traceability	54
5.5	Threats to Validity	54
6	CONCLUSION	56
	BIBLIOGRAPHY	58

	APPENDICES	61
	APPENDIX A – FRONTEND GALLERY	62
A.1	Access and Overview	62
A.2	Performance and Algorithms	64
A.3	Analytics	66
A.4	Run Comparison	70
A.5	Timeline and Results	73
A.6	Settings and Administration	77
	APPENDIX B – VIDEO AND DOCUMENTATION	84
	APPENDIX C – ARCHITECTURE DIAGRAMS	86

1 INTRODUCTION

Cyber-physical systems (CPS) connect software, networks, and physical processes in environments where performance and reliability must coexist. In critical infrastructures, a security failure can affect computational systems, physical assets, and service continuity (CÁRDENAS; AMIN; SASTRY, 2008; MITCHELL; CHEN, 2014; ALCARAZ; LOPEZ, 2010). Intrusion detection systems (IDS) must therefore handle high-dimensional data, time constraints, and operational traceability.

Feature selection reduces dimensionality and redundancy before classification (HALL, 1999; SAEYS; INZA; LARRAÑAGA, 2007; GUYON; ELISSEEFF, 2003). Original GRASP combines greedy randomized construction and local search (RESENDE; RIBEIRO, 2016). Its GRASP-FS adaptation for feature selection is addressed by Quincozes et al. (2020), Quincozes et al. (2022b), Quincozes et al. (2022c). In the monolithic implementations studied here, ranking, construction, refinement, and execution records remain in one process, restricting separation of responsibilities and stage inspection.

This dissertation proposes *G-FShield*, a distributed, event-driven architecture for GRASP-FS in Intrusion Detection Systems (IDS) applied to Cyber-Physical Systems (CPS). Distributed RCL Generation (DRG) computes four independent rankings and constructs initial solutions; Distributed Local Search (DLS) combines VND or RVND with Bit-Flip, IWSS, and IWSSR to refine them. Apache Kafka connects the stages, and a web layer queries persisted events, metrics, states, identifiers, timestamps, and intermediate results. Observability here means inspecting the recorded external state of search, not causally explaining classifier decisions.

The general objective is to design, implement, and evaluate this distributed decomposition under a traceable protocol. The specific objectives are:

- separate candidate generation, neighborhood control, local search, verification, and monitoring;
- provide a web layer for inspecting runs and persisted artifacts;
- compare quality, dimensionality reduction, time, and resources with monoliths and a baseline under common data, evaluator, and computational ceiling;
- preserve enough provenance to reconstruct runs and explicitly delimit properties not yet measured.

Four research questions guide the evaluation. **RQ1:** under the common protocol, what predictive quality and dimensionality reduction are obtained by G-FShield configurations, the monoliths, and the all-features baseline? **RQ2:** how are RCL ranking, neighborhood controller, and local-search preference descriptively related to final solutions and bounded search time? **RQ3:** what CPU and memory are observed for each architecture under the same aggregate ceiling, without conflating consumption with efficiency? **RQ4:** which events, identifiers, metrics, and artifacts allow a run to be reconstructed, and which observability and recovery properties remain unevaluated?

The contribution includes the implemented architecture and a formal campaign with 27 arms, eight seeds per arm, a stratified training/validation/test split, a common final evaluator, and a verifiable manifest. Corpus independence is architectural rather than experimental: input must be labeled tabular data compatible with the service contract. The evaluation covers only the ERENO corpus and J48 classifier.

The remainder of this dissertation is organized as follows. Chapter 2 presents the foundations; Chapter 3 discusses related work; Chapter 4 describes architecture and implementation; Chapter 5 details protocol and results; and Chapter 6 summarizes conclusions, limitations, and future work.

2 THEORETICAL BACKGROUND

This chapter presents the concepts that support *G-FShield*: CPS, IDS, feature selection, Greedy Randomized Adaptive Search Procedure for Feature Selection (GRASP-FS), distributed orchestration, and observability.

2.1 Cyber-Physical Systems and Intrusion Detection

CPS integrate computing components, sensors, actuators, and communication networks into tightly coupled systems (CÁRDENAS; AMIN; SASTRY, 2008; MITCHELL; CHEN, 2014). In digital substations and industrial automation, standards such as the IEC 61850 (IEC 61850) 61850 series support interoperable communication but expand the attack surface (INTERNATIONAL ELECTROTECHNICAL COMMISSION, 2010; MACKIEWICZ, 2006; AFTAB et al., 2020). Therefore, IDS must respond to malicious events without compromising the protected environment.

2.2 Feature Selection

Feature selection seeks attribute subsets that preserve discriminative power and reduce computational cost (HALL, 1999; SAEYS; INZA; LARRAÑAGA, 2007; GUYON; ELISSEEFF, 2003). Let X be an attribute, Y the class, and $H(Z) = -\sum_z p(z) \log_2 p(z)$ the entropy. Information Gain is

$$\text{IG}(X; Y) = H(Y) - H(Y | X), \quad (2.1)$$

the reduction in class uncertainty after observing X (QUINLAN, 1986). Gain Ratio normalizes that gain by the intrinsic information of the attribute,

$$\text{GR}(X; Y) = \frac{\text{IG}(X; Y)}{H(X)}, \quad (2.2)$$

reducing preference for attributes with many values (QUINLAN, 1993). Symmetrical Uncertainty makes the dependence symmetric and bounded:

$$\text{SU}(X, Y) = 2 \frac{\text{IG}(X; Y)}{H(X) + H(Y)}. \quad (2.3)$$

This normalized form is used in correlation-based selection (HALL, 1999). Finally, ReliefF updates the weight W_j of each attribute from differences to near neighbors of the same class (a *hit*) and different classes (a *miss*):

$$\Delta W_j = -\frac{\text{diff}(x_j, H_j)}{mk} + \sum_{c \neq y} \frac{p(c)}{1 - p(y)} \frac{\text{diff}(x_j, M_{j,c})}{mk}, \quad (2.4)$$

where m is the number of samples and k the number of neighbors (KONONENKO, 1994; ROBNIK-ŠIKONJA; KONONENKO, 2003). These four metrics produce independent rankings used to build restricted candidate lists.

2.3 GRASP-FS

GRASP alternates greedy randomized construction and local search (RESENDE; RIBEIRO, 2016). During construction, a restricted candidate list (RCL) contains well-ranked alternatives; one is selected at random to form an initial solution. Local search then applies neighborhoods to improve that solution. GRASP-FS adapts this procedure to feature selection by using relevance criteria to form the RCL and search operators to refine the subset (QUINCOZES et al., 2020; QUINCOZES et al., 2022b; QUINCOZES et al., 2022c).

2.4 Distribution and Orchestration

Distributed architectures decompose responsibilities into smaller units and can isolate failures and permit independent scale when deployment configures them accordingly (TANENBAUM; STEEN, 2007; NEWMAN, 2021). In event-driven systems, Apache Kafka (Kafka) provides a durable distributed log that decouples producers and consumers (KREPS; NARKHEDE; RAO, 2011). Combined with containerization, this model makes service dependencies and deployment explicit (ANDERSON, 2015).

2.5 Observability

Observability is the ability to infer system state from emitted signals such as events, metrics, logs, and temporal traces (BEYER et al., 2016; USMAN et al., 2022). In *G-FShield*, services record execution events, metrics, states, run identifiers, and timestamps. These signals are persisted and queried through the API; the web interface is thus a visualization of the telemetry rather than observability itself. This arrangement makes it possible to reconstruct an execution and relate intermediate results to the components that produced them.

2.6 Synthesis

The problem combines solution-quality requirements, architectural scalability, and operational observability in IDS for CPS. The next chapters position the proposal in the literature, describe its implementation decisions, and delimit the available experimental evidence.

3 RELATED WORK

This chapter positions *G-FShield* in the literature on IDS for CPS, GRASP-FS, distributed architectures, and observability.

3.1 Intrusion Detection in Cyber-Physical Systems

Cárdenas, Amin e Sastry (2008), Mitchell e Chen (2014), and Alcaraz e Lopez (2010) show that CPS security must consider the coupling of software, networks, and physical assets. In digital substations, the IEC 61850 literature discusses specific traffic and timing requirements (INTERNATIONAL ELECTROTECHNICAL COMMISSION, 2010; MACKIEWICZ, 2006; AFTAB et al., 2020). Quincozes et al. (2021) and Quincozes et al. (2022a) stress the importance of realistic corpora for IDS evaluation.

3.2 Feature Selection and GRASP-FS

Hall (1999), Saeys, Inza e Larrañaga (2007), Guyon e Elisseeff (2003) present feature selection as a technique for reducing redundancy and processing cost. Outside the specific lineage of this dissertation, Yusta (2009) compares metaheuristic strategies for feature selection, and Esseghir (2010) combines filters and wrappers in a GRASP scheme. These works address search design but not operation of an observable distributed pipeline.

GRASP is grounded by Resende e Ribeiro (2016); its use for IDS feature selection is investigated by Quincozes et al. (2020), Quincozes et al. (2022b), Quincozes et al. (2022c). Recent surveys indicate that metaheuristics remain relevant to attack detection in industrial and IIoT environments (TERMANINI et al., 2025; FRANCIS; SOURI; İNANÇ, 2026). This trajectory supports the algorithmic components reused by *G-FShield*, not a claim of a new algorithm.

3.3 Distributed Architectures and Observability

Earlier work explored parallelism in parts of GRASP-FS. Silva et al. (2023) distributes local search, whereas Faria et al. (2023) distributes restricted-candidate-list generation. In metaheuristics from another domain, Subramanian et al. (2010) exemplifies parallel search but does not address feature selection or the operational integration studied here.

In another line of work, distributed-systems research grounds component decoupling (TANENBAUM; STEEN, 2007). Service decomposition is discussed by Newman (2021), and messaging logs by Kreps, Narkhede e Rao (2011). Site-reliability work addresses operational telemetry (BEYER et al., 2016; MATEUS-COELHO; CRUZ-CUNHA; FERREIRA, 2021). Observability surveys discuss diagnosis in service architectures (USMAN et al., 2022; FASEEHA et al., 2025). Wu et al. (2013), in turn, addresses feature

selection from streaming features. That is a different problem: the evaluated G-FShield receives a pre-materialized corpus and does not perform online selection. Among the works examined, none combines DRG, DLS, event persistence, and operational queries in a GRASP-FS flow; because the search was not systematic, this conclusion is limited to the set actually analyzed.

3.4 Identified Gap

G-FShield integrates distributed generation, distributed local search, result persistence, and telemetry visualization in one architecture. Its central contribution is architectural and software-engineering-oriented: end-to-end integration, orchestration, and process traceability. DRG and DLS build on prior work and are not presented as a new search algorithm.

3.5 Comparative Table

Table 1 summarizes the discussed capabilities. “Distributed construction” means distributing initial-solution formation; “distributed refinement”, distributing local search; “E2E integration”, connecting both in one flow; and “telemetry”, persisting signals that can reconstruct an execution. The table does not attribute online selection to G-FShield.

Table 1 – Summary of related work cited in this manuscript.

Reference	CPS/ IDS	GRASP- FS	Distributed construction	Distributed refinement	E2E integration	Telemetry
Resende and Ribeiro (2016)	–	✓	–	–	–	–
Yusta (2009); Esseghir et al. (2010)	–	Partial	–	–	–	–
Quincozes et al. (2020–2022)	✓	✓	–	–	–	–
Silva et al. (2023)	✓	✓	–	✓	Partial	–
Faria et al. (2023)	✓	✓	✓	–	Partial	–
Beyer et al. (2016); Usman et al. (2022)	–	–	–	–	–	✓
Wu et al. (2013)	–	–	–	–	–	–
This work (G-FShield)	✓	✓	✓	✓	✓	✓

The table therefore supports an architectural distinction: G-FShield connects feature-selection stages and their operational signals. A claim of exhaustive literature coverage would require a systematic review or a documented bibliographic strategy.

4 DEVELOPMENT

This chapter describes the architecture and implementation of *G-FShield*. The contribution presented here is architectural and software-engineering-oriented: it integrates initial-solution construction, local-search refinement, messaging, persistence, and operational inspection. The feature-selection and search algorithms derive from previous work and are not claimed as new.

4.1 Overview and Input Contract

G-FShield was structured into three layers: a web operations layer, an orchestration and persistence layer, and a distributed execution layer. The first presents observed state and receives commands; the second centralizes the Application Programming Interface (API), database, *cache*, and messaging integration; and the third executes the distributed restricted candidate list generation (DRG) and distributed local search (DLS). Figure 1 summarizes these boundaries.

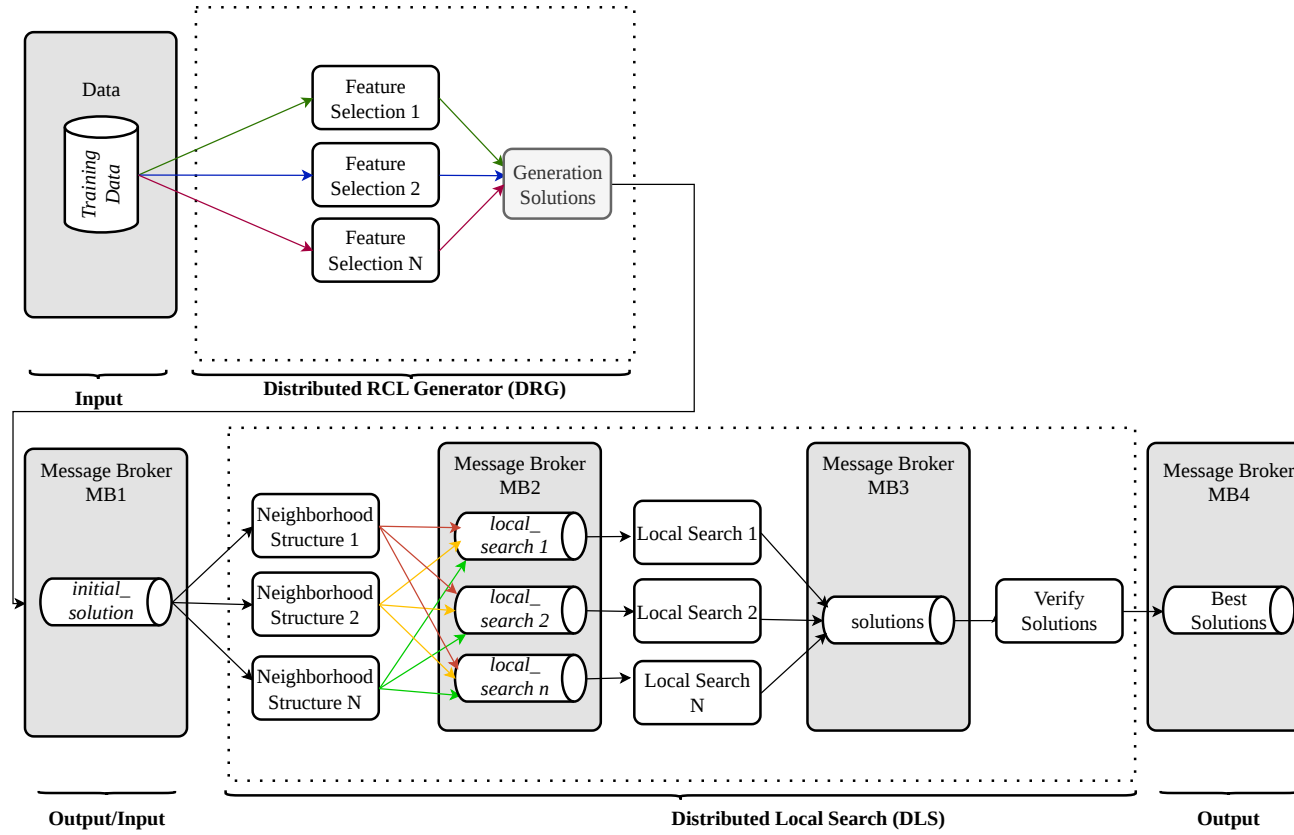


Figure 1 – Summary of the *G-FShield* architecture; the blank margin in the original file is removed only by the LaTeX inclusion.

Independence from the data source is conditional on a concrete contract. The algorithmic API receives two distinct ARFF files, one for training and one for scoring candidates, accessible at `/datasets`. They must share the same schema and keep the label as the last attribute. In the formal campaign, the second file is validation data; the held-out test is accessed only by the final evaluator. The Weka reader assigns the last attribute as the class, and predictors use one-based identifiers. The application accepts J48, Naive Bayes, and Random Forest; the campaign fixes J48.

Consequently, network traffic, protocols, or logs are not consumed in raw form: they must first be transformed into labeled matrices that satisfy this contract. Generality is therefore an architectural and input-format property, not empirical evidence of performance across heterogeneous sources. Historical CSV files reference `erenoall.arff`, whereas the current script uses the ERENO 1k split; without a manifest, one configuration cannot be attributed to the other. In the available `erenoall.arff`, *grayhole* occupies index zero and *normal* index one. Although the name `normalClass=0` remains in legacy binary routines, that path was disabled before the formal-campaign images were built; active evaluation iterates over all nine ARFF classes and computes macro-F1.

Each request supplies the training and testing files, classifier, number of generations, RCL cutoff, initial sample size, VND or RVND strategy, enabled local searches, and optional iteration limits. These fields form the configuration unit propagated between services. Each solution also receives a random `seedId`, which correlates the refinements derived from it.

4.2 Architectural Drivers and Decisions

The decomposition was guided by six drivers: throughput, decoupling, traceability, persistence, inspection of intermediate results, and the possibility of operational evolution. Table 2 makes the relationship between these drivers and the adopted decisions explicit. Service decomposition and asynchronous communication follow established principles of distributed systems and microservice architectures (TANENBAUM; STEEN, 2007; NEWMAN, 2021; KREPS; NARKHEDE; RAO, 2011).

The construction and refinement phases differ in cost and resource-use patterns. They were therefore separated into services with their own CPU, memory, and concurrency limits. This separation permits one responsibility to be observed and changed without incorporating the entire search into a single process. It also makes it possible to increase the number of instances of a service; however, the relationship between replicas, throughput, and cost requires a dedicated scaling experiment.

Persistence and event history were adopted to reduce reliance on state kept only in memory. The evaluated configuration, however, uses one Kafka broker and replication factor one. High availability or fault tolerance therefore cannot be inferred from the decomposition. Those properties would require fault injection and verification of redelivery,

Table 2 – Architectural drivers, decisions, and limits.

Driver	Design decision	Scope of evidence
Throughput	Execute the four rankings in independent services and forward seeds to decoupled search operators.	The evaluation measures the deployed configuration; it does not demonstrate efficiency under equal resources.
Decoupling	Use Kafka topics between construction, controllers, operators, verifier, and monitor.	Temporal coupling is reduced, but robustness under failure was not experimentally measured.
Traceability	Propagate <code>seedId</code> , algorithm, search, iterations, feature sets, and metrics.	The sequence observed by the monitor can be reconstructed; flow completeness is not proven.
Persistence	Store launches, runs, events, and aggregates in PostgreSQL and CSVs in the metrics volume.	Redis is a <i>cache</i> ; it does not replace persistent storage.
Intermediate inspection	Publish selected progress, refined solutions, and observed best solutions.	The interface presents telemetry; it does not make the algorithm explainable.
Operational evolution	Isolate responsibilities to allow service-level replication and replacement.	Scalability and fault tolerance are design objectives, not demonstrated results.

idempotency, recovery, and reprocessing.

4.3 Distributed Restricted Candidate List Generation

In the DRG stage, four independent services calculate Information Gain (IG), Gain Ratio (GR), Symmetrical Uncertainty (SU), and ReliefF. Each service scores only training predictors, sorts them in descending order, and forms its own RCL from the first `rclCutoff` elements; the class attribute in the last position is excluded from ranking. There is no aggregator that merges the four rankings into one list. A request containing several metrics launches independent branches that publish initial solutions to the same topic.

Construction follows the principle of an RCL combined with random choice, which is characteristic of GRASP (RESENDE; RIBEIRO, 2016). The adaptation to feature selection and the DRG stages used here derive from GRASP-FS and its distributed extensions (QUINCOZES et al., 2020; QUINCOZES et al., 2022b; QUINCOZES et al., 2022c; FARIA et al., 2023). These references support the algorithmic decision; microservice references support only the implementation decomposition.

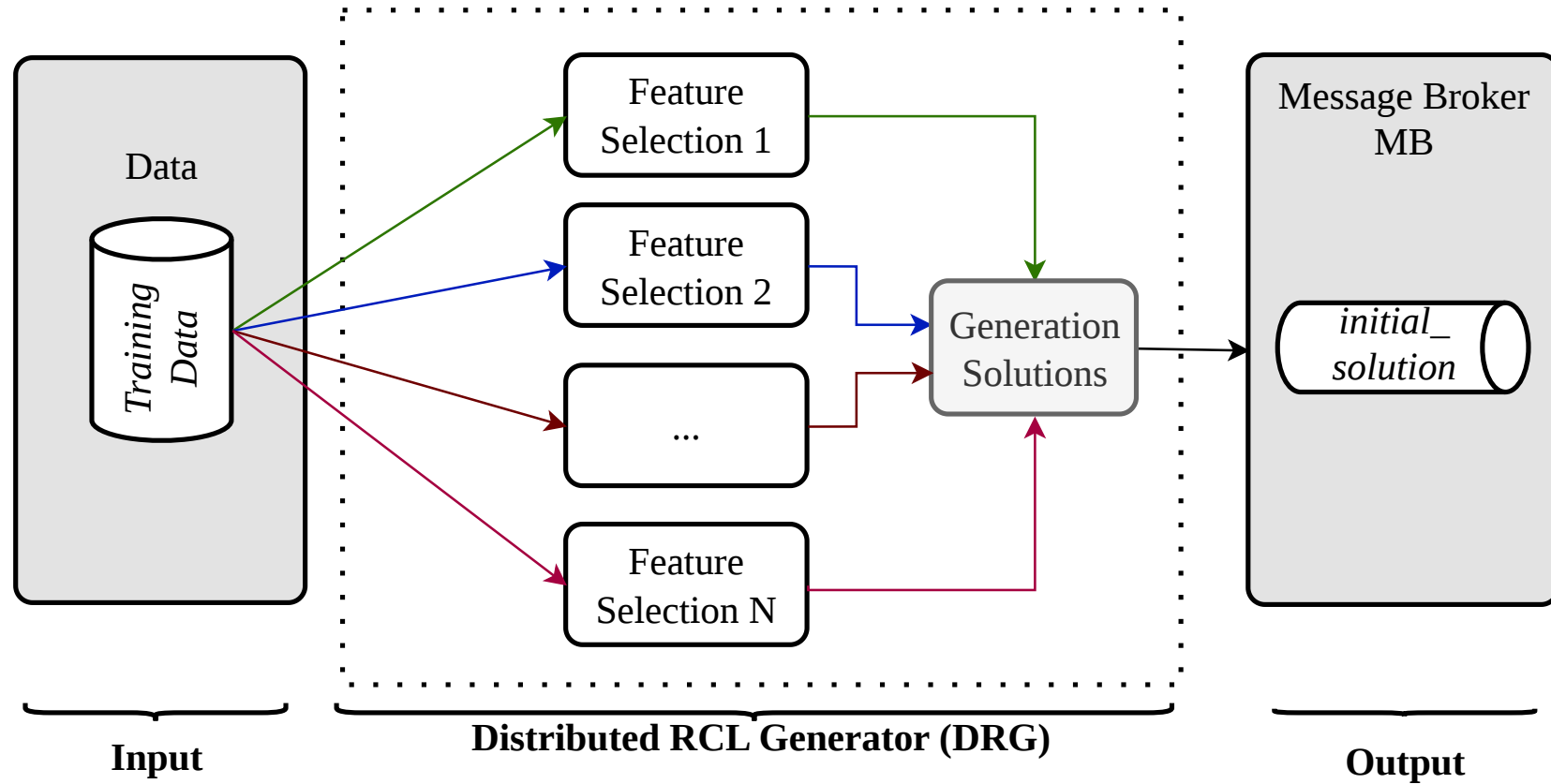


Figure 2 – Generic architecture of the DRG stage.

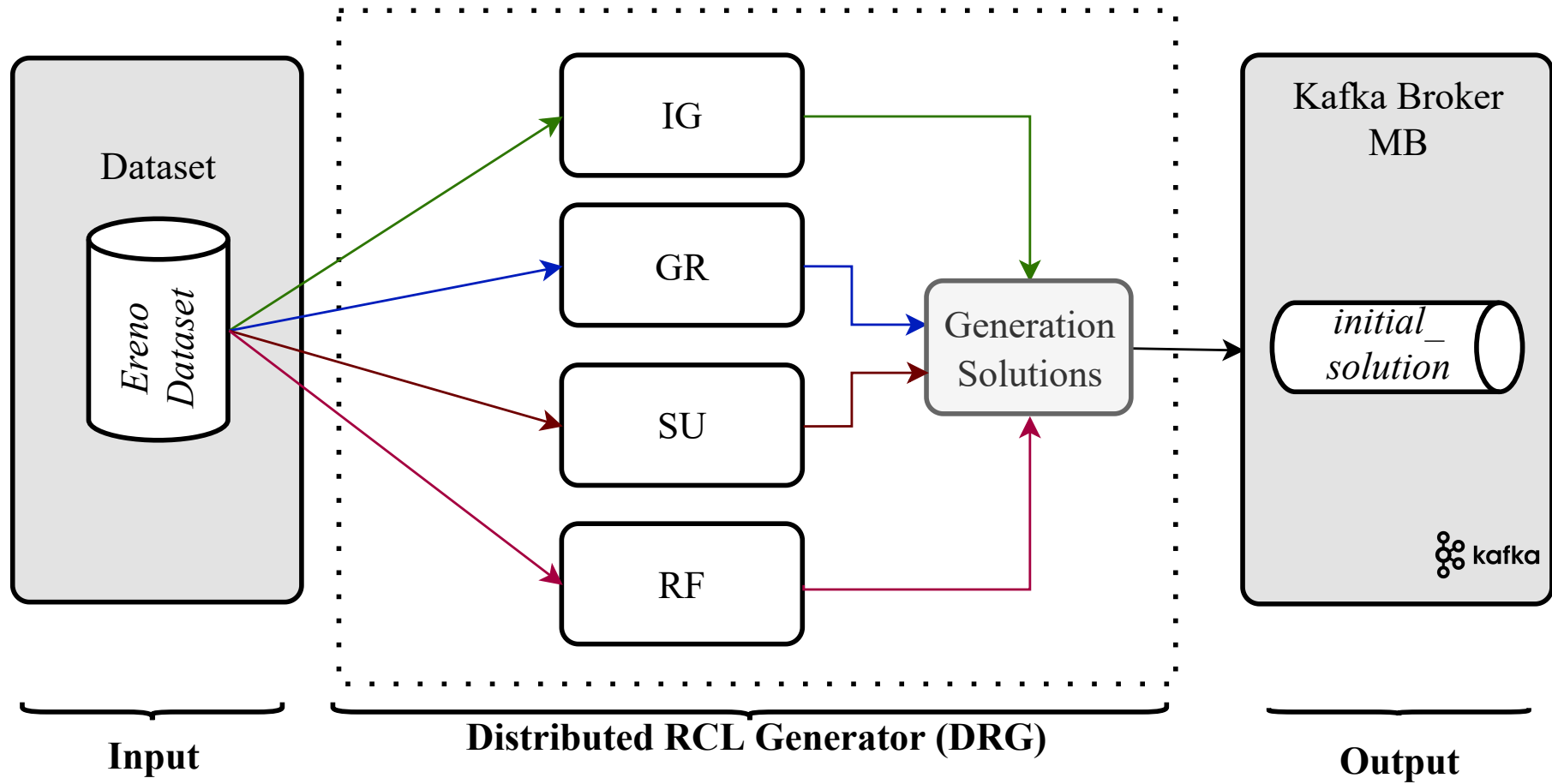


Figure 3 – Concrete DRG instantiation with the four implemented metrics.

For each generation, the service copies its RCL and samples `sampleSize` positions uniformly without replacement from that copy. In the formal campaign, the generator is initialized by `CAMPAIGN_RANDOM_SEED`; `seedId` identifies the solution, while `campaignId`, `runId`, `requestId`, and the numeric seed preserve provenance. The subset is evaluated on reduced training and validation data; the held-out test set does not participate in search. The published message preserves the subset, RCL, internal metrics, and refinement configuration.

Listing 4.1 presents this behavior as pseudocode. `metric` identifies one of the four branches. Parameters map as follows:

- `G`: `maxGenerations`;
- `k`: `rclCutoff`;
- `s`: `sampleSize`.

Listing 4.1 – Implemented RCL formation and randomized construction in the DRG.

```

procedure DRG(trainFile, validationFile, metric, G, k, s, classifier,
              neighborhood, enabledSearches, iterationLimits, runSeed)
  train <- readARFF(trainFile); train.class <- lastAttribute
  valid <- readARFF(validationFile); valid.class <- lastAttribute
  rng <- PseudoRandomGenerator(runSeed)

  scored <- empty list
  for each predictor a in train, excluding the class do
    scored.append((metric(train, a), oneBasedIndex(a)))
  end for
  ranking <- sortDescending(scored)
  rcl <- first min(k, size(ranking)) feature identifiers

  for generation <- 1 to G do
    pool <- copy(rcl)
    selected <- empty list
    repeat min(s, size(pool)) times
      p <- UniformInteger(rng, 1, size(pool))
      selected.append(removeAt(pool, p))
    end repeat

    seedId <- randomUUID()
    scores <- trainAndValidate(classifier, train, valid, selected)
    appendServiceMetricsCSV(selected, scores, classifier,
                           trainFile, validationFile)
    publish(INITIAL_SOLUTION_TOPIC,
           {seedId, selected, rcl, scores, metric,

```

```

neighborhood, enabledSearches, iterationLimits})
end for
end procedure

```

4.4 Distributed Local Search

The Variable Neighborhood Descent (VND) and Randomized Variable Neighborhood Descent (RVND) controllers receive each initial solution and forward messages to the Bit Flip, Incremental Wrapper Subset Selection (IWSS), and Incremental Wrapper Subset Selection with Randomization (IWSSr) operators. VND visits the enabled searches in configured order, compares the best cycle result with its baseline, and restarts the cycle when an improvement occurs and dispatch budget remains. RVND randomly selects an enabled search at each step and continues until the iteration limit. VND and its randomized variant are grounded in the variable-neighborhood and RVND literature (MLADENović; HANSEN, 1997; DUARTE et al., 2018; SUBRAMANIAN et al., 2010); distribution of these operators follows GDLS-FS (SILVA et al., 2023).

Bit Flip randomly exchanges one subset attribute with one attribute in the candidate list at each iteration. IWSS sequentially adds the first still-available attribute and preserves the best evaluated subset. IWSSr performs the addition move and, for every position of the resulting subset, evaluates the corresponding removal, retaining the best replacement. These neighborhood operators follow the GRASP-FS assessments (QUINCOZES et al., 2020; QUINCOZES et al., 2022b; QUINCOZES et al., 2022c). The RCL forwarded to DLS still contains initial-solution attributes, but the campaign canonicalizes the final solution and rejects empty, invalid, or duplicated subsets. Operators train on training data, score candidates on validation data, and publish results to SOLUTIONS_TOPIC. The verifier keeps the greatest internal F1 per `seedId` and publishes strict improvements to BEST_SOLUTION_TOPIC; this is the best observed result, not a global certificate of optimality.

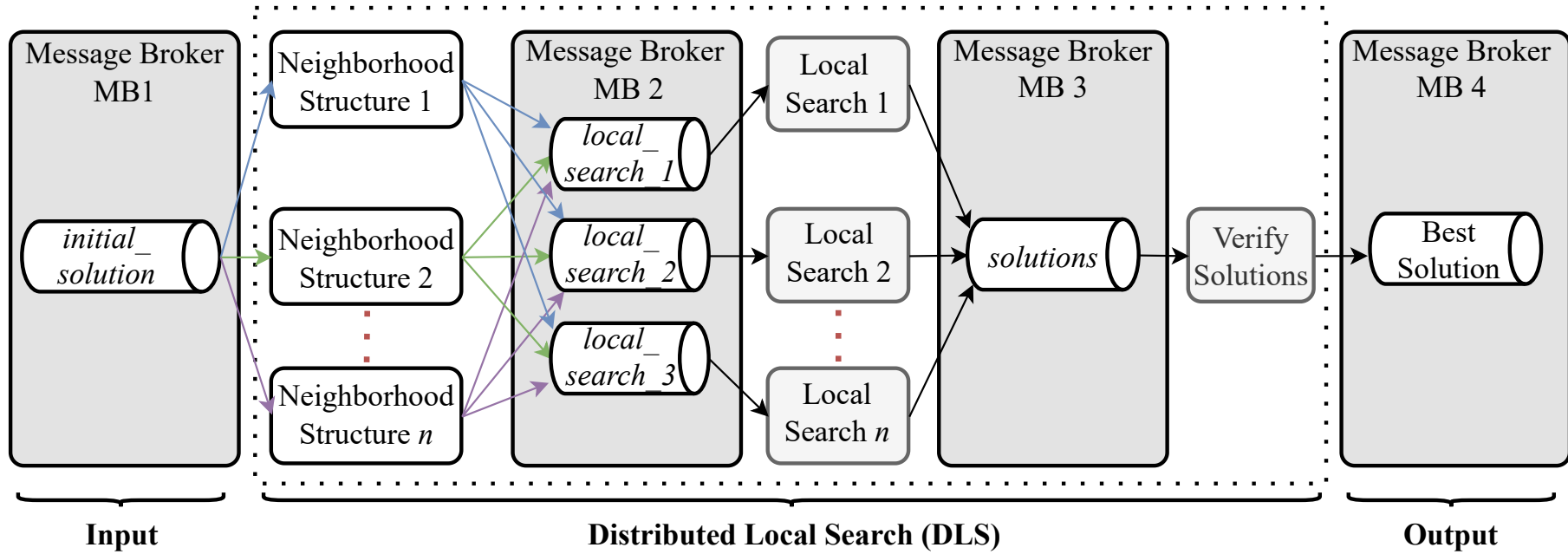


Figure 4 – Generic architecture of the DLS stage.

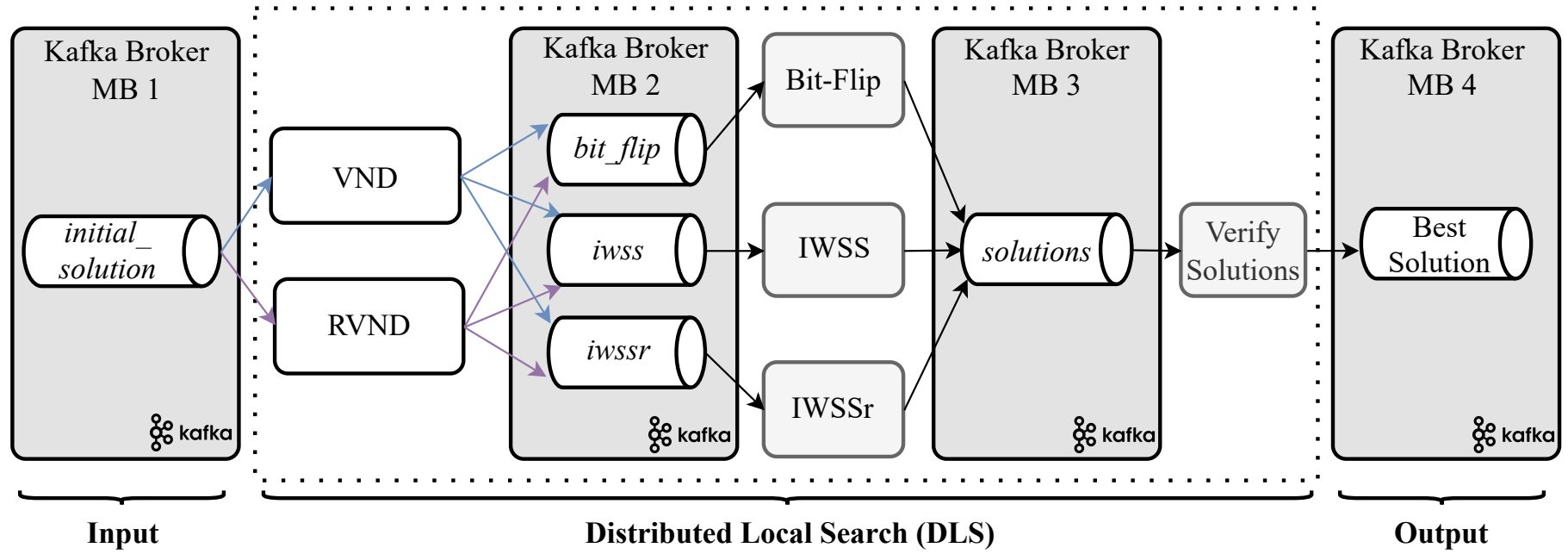


Figure 5 – Concrete DLS instantiation with the implemented controllers and operators.

Listing 4.2 summarizes the effective orchestration. The VND budget is the configured number of cycles multiplied by the number of enabled searches; RVND uses the iteration limit directly. Both controllers default to ten when the request does not provide a limit.

Listing 4.2 – Implemented distributed refinement and verification in the DLS.

on INITIAL_SOLUTION_TOPIC or NEIGHBORHOOD_RESTART_TOPIC receive seed

```

best <- seed
searches <- configured searches or [BIT_FLIP, IWSS, IWSSR]

if seed.neighborhood = VND then
  dispatchBudget <- maxCycles * size(searches)
  repeat while dispatchBudget remains
    baseline <- best.F1
    for each search in searches in configured order do
      candidate <- dispatchAndReceive(search, best)
      best <- argmaxF1(best, candidate)
      dispatches <- dispatches + 1
    end for
    if best.F1 <= baseline then stop
  end repeat
else if seed.neighborhood = RVND then
  repeat until maxIterations dispatches
    search <- UniformChoice(searches)
    candidate <- dispatchAndReceive(search, best)
    best <- argmaxF1(best, candidate)
    dispatches <- dispatches + 1
  end repeat
end if

```

```

for each candidate received by verifier do
  if bestSeen[candidate.seedId] is undefined or
    candidate.F1 > bestSeen[candidate.seedId].F1 then
    bestSeen[candidate.seedId] <- candidate
    publish(BEST_SOLUTION_TOPIC, candidate)
  end if
end for
end receive

```

The formal protocol initializes construction, Bit Flip, and RVND with the run’s numeric seed and persists it separately from UUID `seedId`. This makes the pseudorandom sequence repeatable in the versioned environment; concurrency, message ordering,

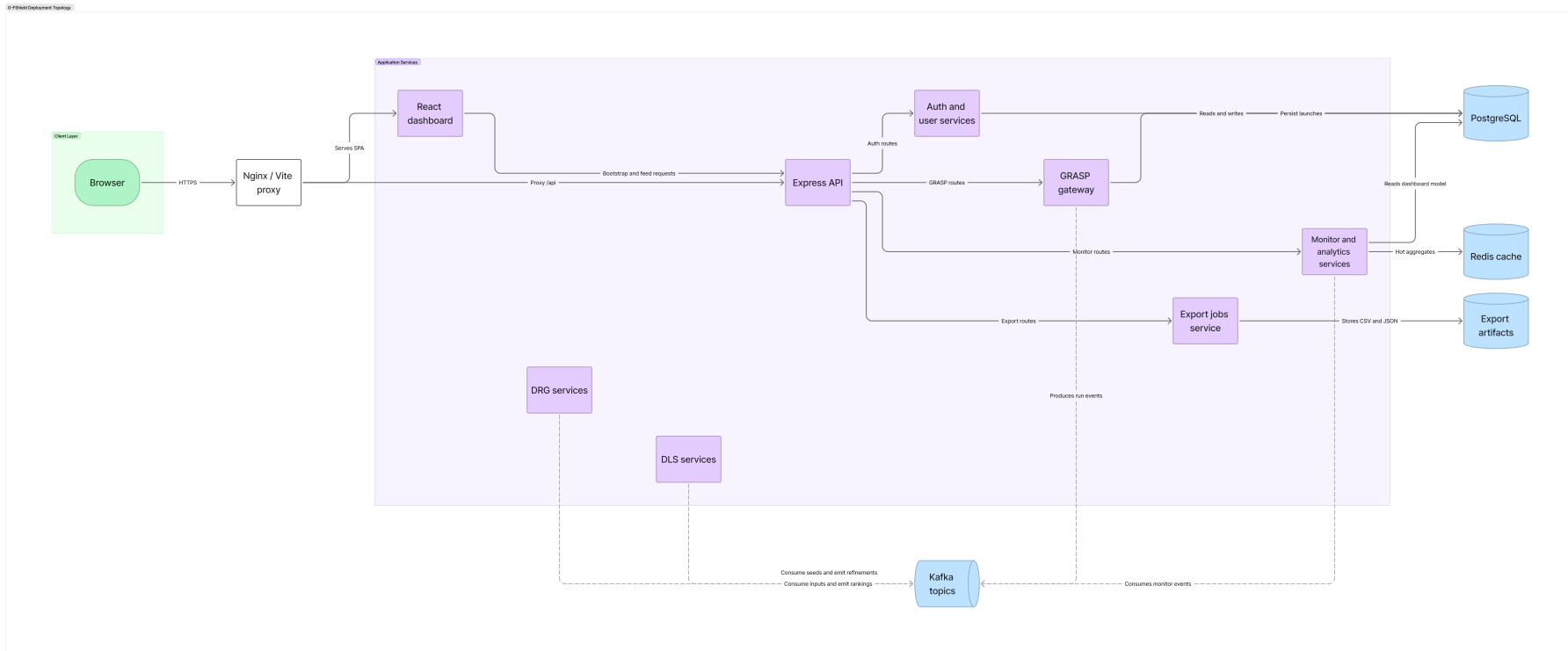
and platform may still introduce variation, so every result also preserves timestamps, command, and artifacts.

4.5 Event-Driven Orchestration

Kafka decouples producers and consumers through the following topic groups:

- construction: `<INITIAL_SOLUTION_TOPIC>`;
- operators: `<BIT_FLIP_TOPIC>`, `<IWSS_TOPIC>`, and `<IWSSR_TOPIC>`;
- coordination: `<SOLUTIONS_TOPIC>` and `<NEIGHBORHOOD_RESTART_TOPIC>`;
- monitored progress: `<LOCAL_SEARCH_PROGRESS_TOPIC>`;
- best observed result: `<BEST_SOLUTION_TOPIC>`.

The key of a solution message is the `seedId`, when available, and observed events feed the monitor (KREPS; NARKHEDE; RAO, 2011).

Figure 6 – Deployment topology and execution boundaries of *G-FShield*.

Kafka is configured as an execution event log, but potential durability is not equivalent to demonstrated recovery. The configuration enables automatic topic creation, six partitions, and replication factor one. Message loss or duplication, event completeness, instrumentation cost, post-restart behavior, consumer recovery, and reprocessing were not measured. Resilience and fault tolerance therefore remain design objectives.

4.6 Persisted Telemetry and Web Layer

Observability in this work means the ability to inspect recorded external execution state. It does not mean causally explaining classifier decisions. The API monitor consumes five topics of interest: initial solution, neighborhood restart, local-search progress, refined solution, and best solution. Each persisted event contains a type, JSON payload, and backend ingestion timestamp, as well as topic, stage, status, `seedId`, `requestId`, partition, and offset when available. A fingerprint prevents reinserting the same observed event. In the formal campaign, `campaignId`, `runId`, `requestId`, `seedId`, producer UTC timestamp, and monotonic elapsed time are propagated in experimental objects.

PostgreSQL maintains three principal levels: the requested launch, current run per `seedId`, and associated event history. Run state includes RCL algorithm, classifier, local search, neighborhood, files, iterations, F1, accuracy, precision, recall, CPU, memory, time, and feature sets. Materialized read models aggregate topics, algorithms, resources, and timeline intervals. Redis temporarily stores frequently requested responses; PostgreSQL is the persistent source.

The Java services also write per-component CSVs to the `/metrics` volume. These files contain the subset, F1, accuracy, precision, recall, time, CPU, memory, classifier, and filenames. Because the current CSVs do not record `seedId`, direct correlation with the web history must not be assumed. Java and Node logs are emitted to `stdout`; Compose configures no log store, retention policy, or collector such as Loki/ELK. Events and metrics are therefore persisted by the application, but log persistence depends on the external runtime and was not demonstrated.

The combination of identifiers, events, and timestamps reconstructs the timeline that reached the monitor and permits states of the same seed to be compared. It does not guarantee that every produced event was captured. In the formal campaign, the numeric seed, `runId`, and checksums complement this telemetry; monitor completeness and reproduction on different hardware remain limited.

The web implementation uses React and Vite in the client; Express and Swagger in the API; PostgreSQL for persistence; and Redis as an optional *cache*. The interface queries summaries, paginated events, timelines, and CSV or JSON exports. These technologies describe the implementation; by themselves, they do not prove productivity, stability, responsiveness, or performance.

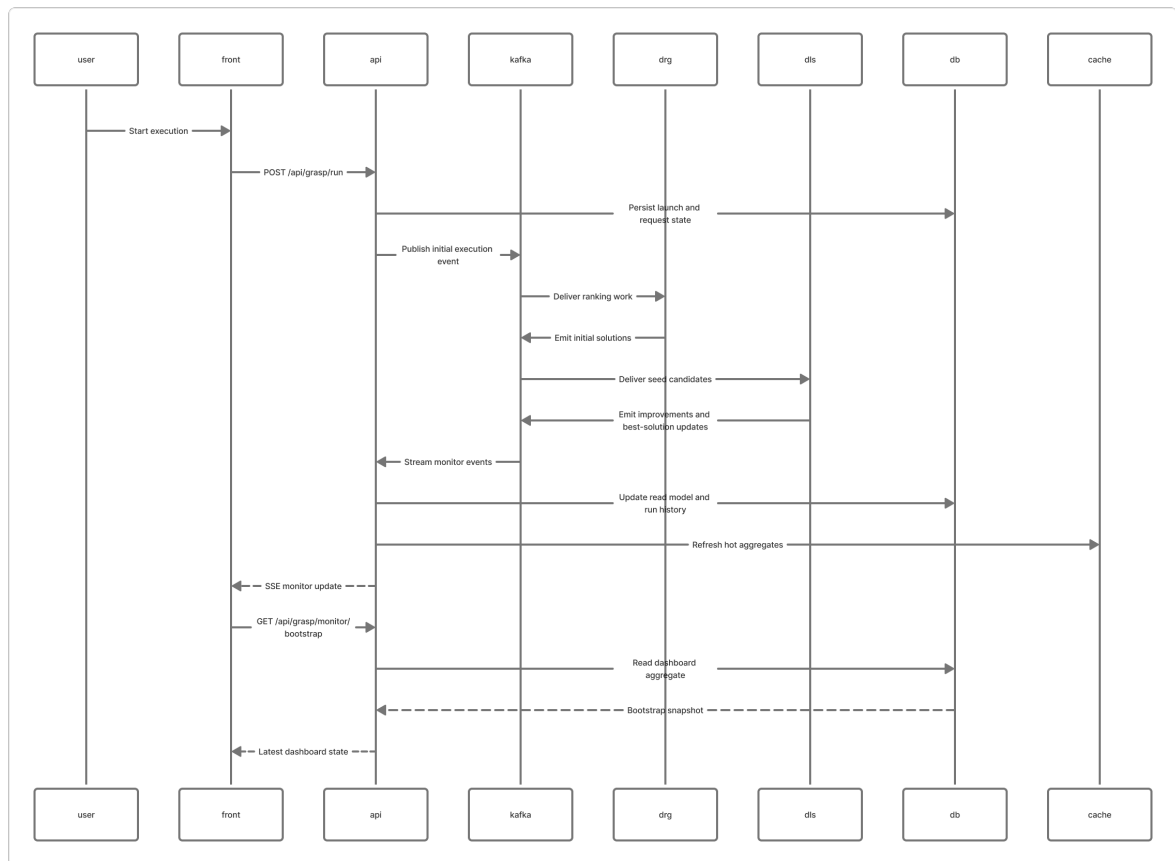


Figure 7 – Execution lifecycle of *G-FShield*, from the request to the states shown in the interface.

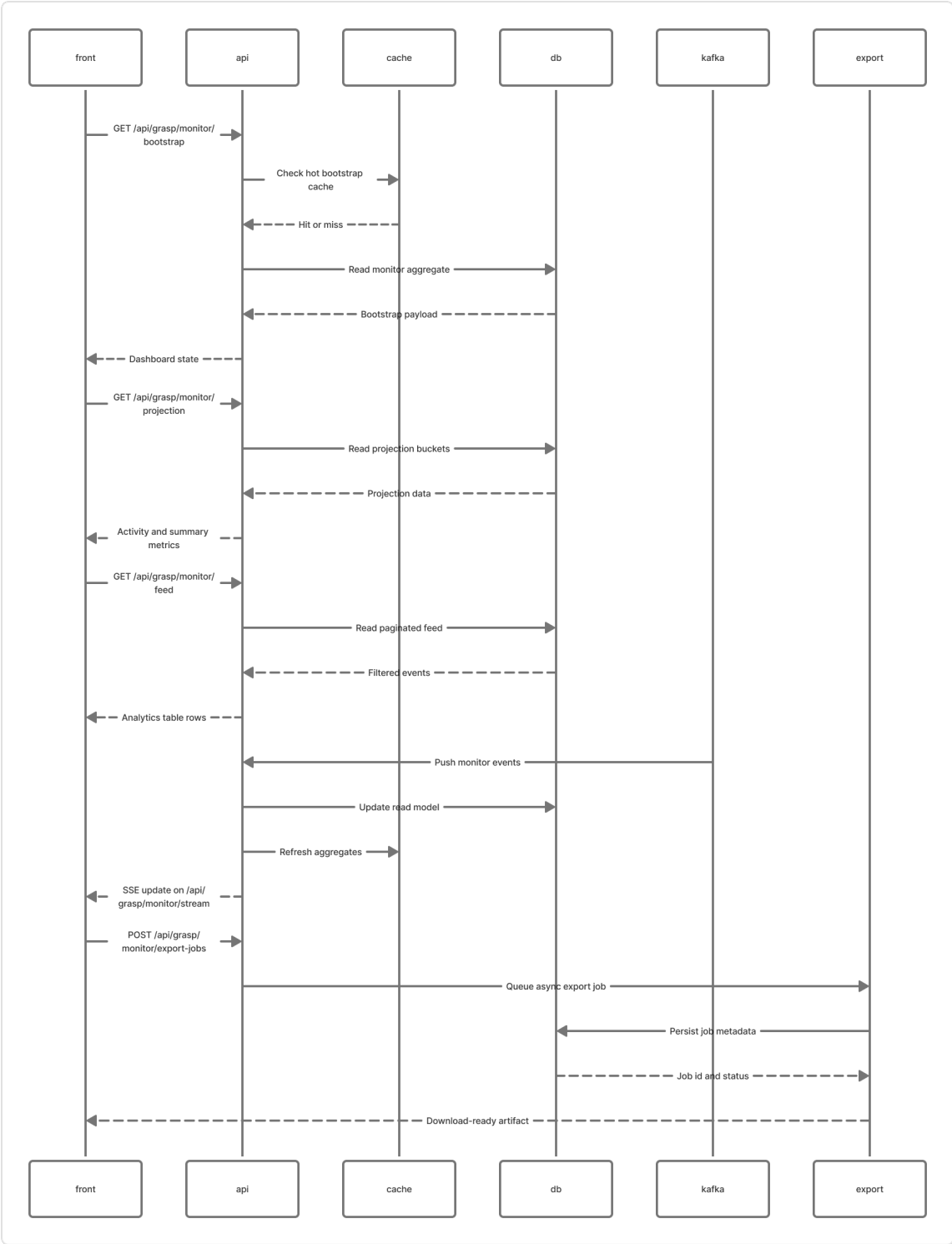


Figure 8 – Ingestion, persistence, and presentation pipeline for observed telemetry.

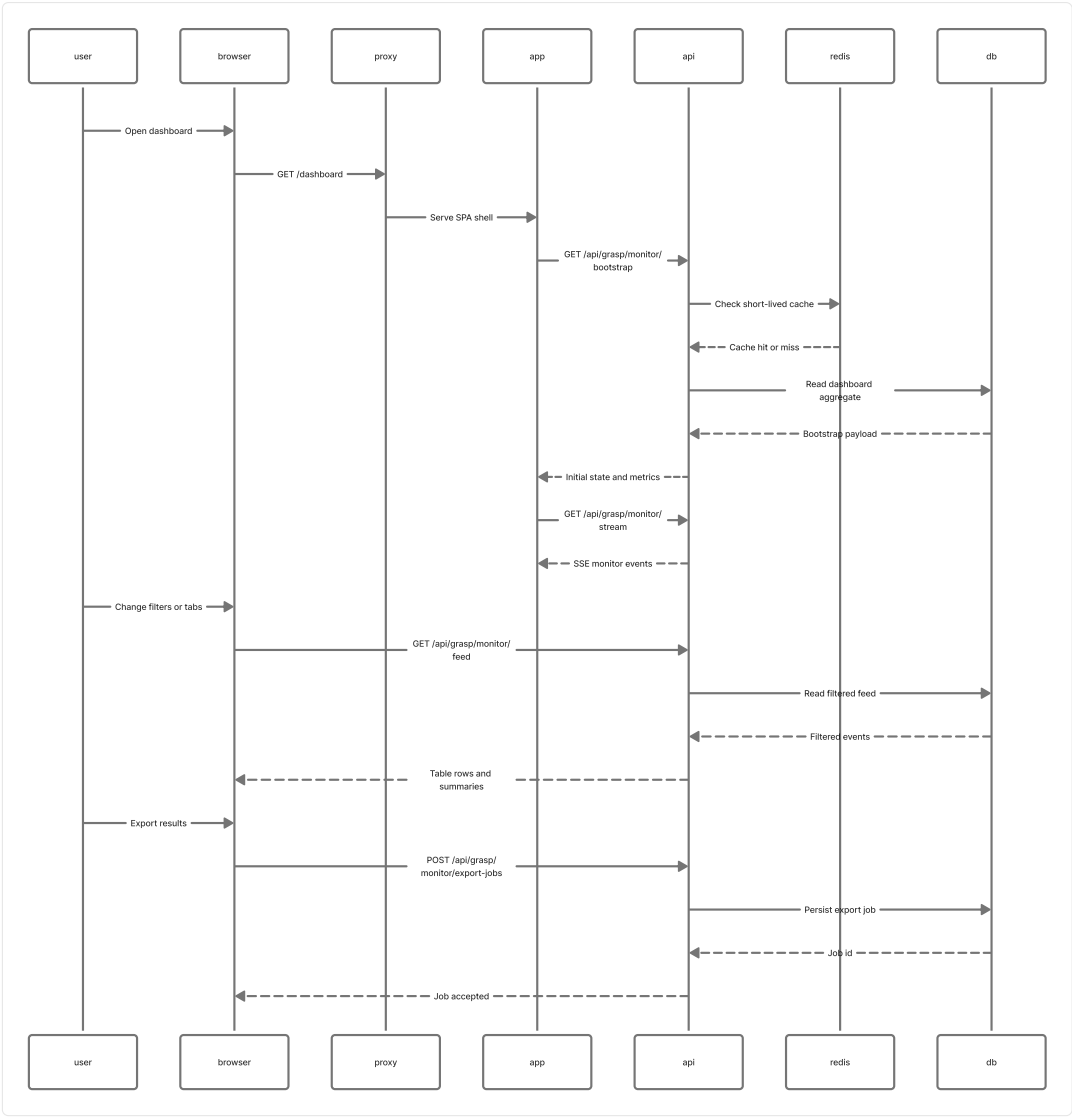


Figure 9 – Functional flow of the web client and its API queries.

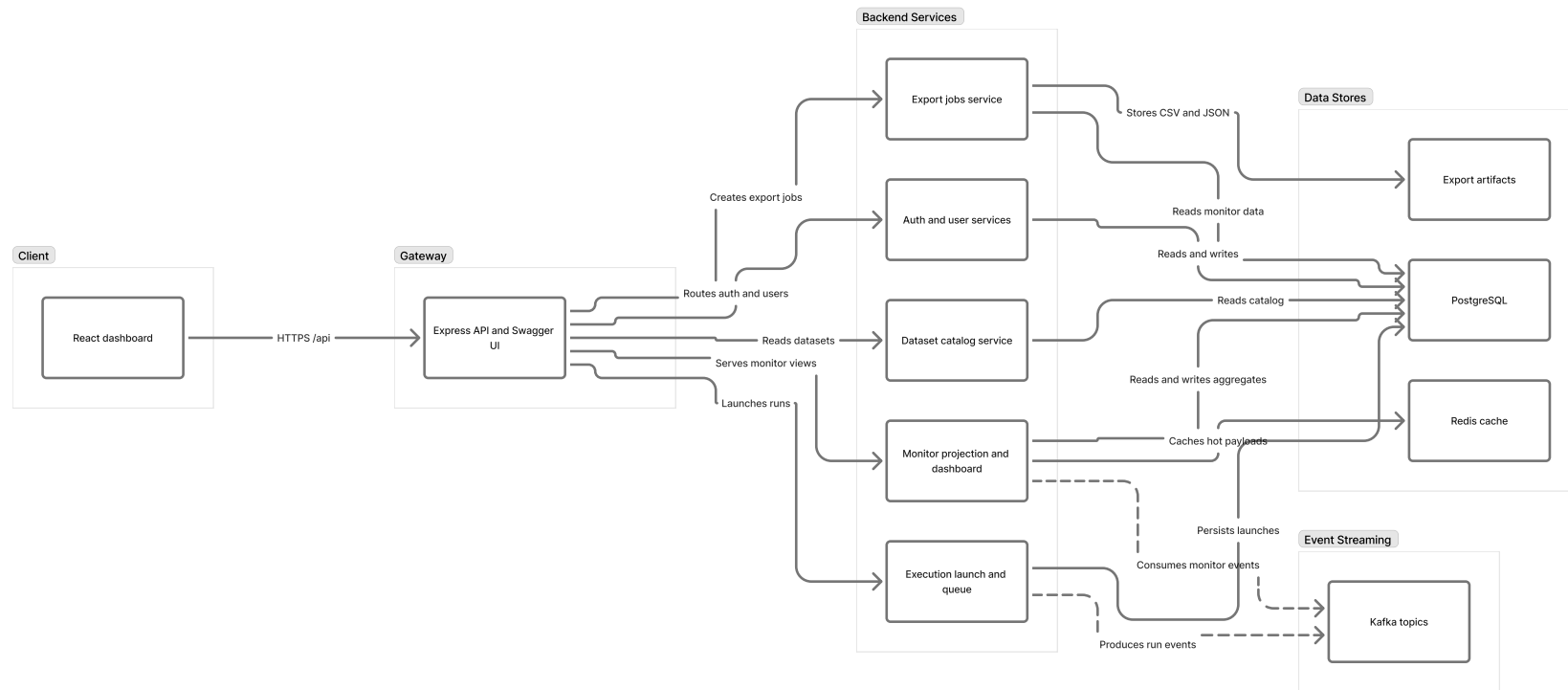


Figure 10 – Backend services, data stores, and messaging integration.

Figures 7 through 10 complement the abstract view with the implemented flows. Interface captures, API documentation, and supporting diagrams are provided in Appendices A, B, and C.

4.7 Reference Configuration and Reproducibility

Table 3 records the local configuration defined by three files:

- `<docker-compose.yml>`;
- `<docker-compose.local.yml>`;
- `<webservice/api/docker-compose.db.yml>`.

Values are per-container limits. The server preset increases some of these limits and must therefore not be mixed into an experimental comparison without an explicit record.

Table 3 – Resources and concurrency of the local reference configuration.

Service group	Count	CPU/unit	RAM/unit	Concurrency/configuration
DRG: IG, GR, SU, and ReliefF	4	1.50	1536 MB	asynchronous pool: core 2, maximum 4, queue 80
DLS: Bit Flip, IWSS, and IWSSR	3	1.00	1536 MB	one Kafka consumer per service
VND, RVND, and verifier	3	0.75	768 MB	one Kafka consumer per service
Kafka	1	1.00	1024 MB	six partitions; replication 1
ZooKeeper	1	0.50	256 MB	client port 2181
Kafbat UI	1	0.50	512 MB	internal Kafka client
PostgreSQL	1	0.50	512 MB	local database configuration
Redis	1	0.25	256 MB	AOF persistence and snapshots configured

The host identified as the execution server was inspected on August 25, 2026. The machine, identified by the operating system as `mc2-server-01`, has two Intel Xeon E5-2620 v4 processors at 2.10 GHz, with eight cores and two threads per core in each socket, totaling 16 physical cores and 32 logical processors in two NUMA nodes. Docker detected 67,447,271,424 bytes of memory (64,322.73 MiB; approximately 62.8 GiB), and the system had 8 GiB of swap. The environment runs Ubuntu 18.04.6 LTS, x86-64, and Linux kernel 4.15.0-213-generic. The inspection found Docker Engine/client 24.0.2, API 1.43, the `overlay2` storage driver, the `cgroupfs` cgroup driver, `docker compose` 2.18.1, and the `docker-compose` executable 2.30.3.

Docker labels linked the running services to the `g-fshield` project and to the overlay of `<docker-compose.yml>` with `<docker-compose.server.yml>`. Table 4 records the effective limits observed. There was one instance of each service; neither Swarm mode nor additional container replication was active. PostgreSQL and Redis belonged to a second composition formed by `<webservice/api/docker-compose.db.yml>` and `<webservice/api/docker-compose.db.server.yml>`. The API and web client did not appear in the container list, so no observed limit is attributed to them.

From the repository root, the commands corresponding to the files recorded in the labels are:

```
docker compose -f docker-compose.yml -f docker-compose.server.yml up -d
docker compose -f webservice/api/docker-compose.db.yml -f webservice/api/docker-
compose.db.server.yml up -d
```

These commands reconstruct the observed composition; they do not replace terminal history or a manifest confirming the options used in the campaign.

Table 4 – Effective limits on the inspected server.

Service group	Count	CPU/unit	RAM/unit	Concurrency/configuration
DRG: IG, GR, SU, and ReliefF	4	2.00	2048 MB	pool: core 3, maximum 6, queue 150
DLS: Bit Flip, IWSS, and IWSSR	3	1.50	2048 MB	one Kafka consumer per service
VND, RVND, and verifier	3	1.00	1024 MB	one Kafka consumer per service
Kafka	1	2.00	2048 MB	six partitions; replication 1
ZooKeeper	1	0.75	512 MB	one instance
Kafbat UI	1	1.00	2048 MB	one instance
PostgreSQL	1	1.00	1024 MB	one instance
Redis	1	0.25	256 MB	one instance

Directory `</home/idscps/nicolas/graspy2>` supplied the implementation of Monolith 1. Directory `</home/idscps/nicolas/Graspy>` supplied that of Monolith 2. For the formal campaign, both were rebuilt as versioned images and run as one container per arm under the same aggregate ceiling. Monolith 1 ranks features with Gain Ratio, constructs a five-feature subset from cutoff 30, and applies Bit-Flip for at most 100 local iterations. Monolith 2 uses Gain Ratio, constructs five features from an RCL of size 30, and applies IWSS under VND for at most 100 iterations. Both receive the same immutable training, validation, and test splits, seeds 42–49, 3,000-s limit, maximum of 500 accepted improvements, and minimum improvement 0.0001.

During formal search, both monoliths and G-FShield use Weka J48 and validation macro-F1. The best validation solution is then evaluated once on test. This adaptation removes historical classifier, objective-function, and metric-scale differences while preserving distinct comparator algorithms: GR–BF in Monolith 1, GR–IWSS in Monolith 2, and the 24 ranking, controller, and operator combinations in G-FShield. The campaign therefore compares complete solutions but does not causally isolate distribution alone.

The four DRG services expose ports 8086–8089; Bit Flip, IWSS, IWSSR, and the verifier use 8082–8085; RVND and VND use 8090 and 8091. Kafka exposes 19092, ZooKeeper 2181, JMX 9999, and Kafbat UI 8080. The API, web client, PostgreSQL, and Redis use 4000, 3000, 5432, and 6379 by default. Datasets and CSVs are bind volumes for the `datasets/` and `metrics/` directories; PostgreSQL and Redis use named volumes.

The Java images are built with Maven 3.9.4 and Eclipse Temurin 17 and run on the Temurin 17 JRE. Most evaluation modules declare Weka 3.8.0; the SU service `pom.xml` contains duplicate declarations for 3.8.6 and 3.8.0, so the resolved dependency tree must be archived. DRG modules use Spring Boot 3.0.0, and DLS modules use Spring Boot 2.7.5. The infrastructure pins these images:

- `<confluentinc/cp-kafka:7.4.0>;`

- `<confluentinc/cp-zookeeper:7.4.0>;`
- `<ghcr.io/kafbat/kafka-ui:v1.4.2>;`
- `<postgres:14>;`
- `<redis:7-alpine>.`

The optional mode that runs the web layer in containers uses `node:24`; the project declares Express 4.21.1, React 18.2.0, and Vite 8.0.3. The Compose files do not impose CPU and RAM limits on the API and web client started by the scripts; these limits must also be fixed for a performance reproduction.

4.7.1 Formal campaign manifest

The formal campaign identifier is `gfshield-10d-2026-v1`. Its tag is `experiment-10d-v1`, and the launch commit is `66565b865a7b`. Images were built from commit `b38eff3ed9c1`; the manifest preserves the SHA-256 ID of each local image and the digests of Kafka 7.4.0 and ZooKeeper 7.4.0. The manifest, resolved Compose file, and host provenance have their own hashes. Each run also archives its command, resolved Compose file, logs, metrics, telemetry, final result, and a checksum manifest.

The file `<experiments/10-day-campaign/frozen-compose.yaml>` freezes the services used by the protocol. Each distributed configuration starts one RCL generator, VND or RVND, all three BF/IWSS/IWSSR operators, the verifier, Kafka, and ZooKeeper. There is one consumer per service and one logical execution partition; Kafka uses six partitions, replication factor one, and automatic topic creation. Java services use Temurin 17; the common evaluator uses Weka 3.8.6. The monoliths are separate images derived from `<graspy2>` and `<Graspy>`. Every arm uses `-cpuset-cpus 8-15`, `-cpuset-mems 1`, and an aggregate ceiling of eight CPUs and 16 GiB. The host kernel does not expose Docker swap accounting; the memory limit was enforced, but a separate swap limit could not be verified.

Data are mounted read-only at `</data>` or `</datasets>`; results, logs, and metrics use bind volumes specific to each `run_id`. Generator ports are published only on 127.0.0.1 during execution; Kafka and the remaining services communicate over the composition's internal network. The web layer is not part of the campaign's measured path.

The durable server launch was:

```
python3 experiments/10-day-campaign/campaign_supervisor.py \
  --manifest experiments/10-day-campaign/manifest.yaml \
  --state experiments/10-day-campaign/state/campaign-state.json \
  --supervisor-state experiments/10-day-campaign/state/supervisor-state.json \
  --log experiments/10-day-campaign/logs/campaign-supervisor.log \
  --output-root experiments/10-day-campaign/results
```

The supervisor preserves the absolute deadline, checks free storage, terminates pro-

cess groups, and limits restarts. It did not restart in the reported execution. The campaign produced 216 valid results for 27 arms and eight seeds, plus 12 preserved invalid attempts that were retried. The repository contains `<experiments/10-day-campaign/analyze_results.py>`, which verifies checksums, normalizes metrics, and recreates tables and figures. Reproduction still requires access to the images or rebuilding them at the recorded commit, the corpus matching the recorded hashes, and a compatible host; dynamic CPU frequency, external containers, and physical-hardware limitations are not eliminated by containerization.

4.8 Synthesis

G-FShield integrates independent GRASP-FS construction branches, local-search controllers and operators, messaging, and verification of observed best results. A web layer queries data held in persistent storage. The input contract, pseudocode, manifest, and container configuration delimit implementation and campaign. The architecture provides decoupling, inspection, and evolution; the campaign establishes experimental traceability, while scalability, fault tolerance, and portability across hosts remain properties to evaluate.

5 EVALUATION

This chapter presents the formal experimental campaign, bounded to ten days, run on the same server for G-FShield, both monoliths, and an all-features baseline. The single source for this analysis is the result set identified by the manifest whose SHA-256 is:

29fdc973ad6b243b297e34fcd539f5826aac4ce8a0870e8a5cea5caacfc9adc4.

The historical CSV files in `tmp_metrics/`, which contained the discrepancy between 25,873 raw lines and 25,836 lines with valid F1, are not combined with the new campaign. Neither count is therefore used as a comparative result.

5.1 Experimental Protocol

The preserved ERENO corpus contains 199,998 instances, 51 predictors, and nine classes. A stratified split, frozen before the searches with seed 20260825, produced 119,998 training, 40,000 validation, and 40,000 test instances. The SHA-256 prefixes for the source, training, validation, and test data are 60ebfcdc9c20, 5876bf5570b7, c44a00ca554c, and 05d393c5881a; the full values are in the manifest. The test set remained unavailable to the search and was evaluated once after validation selected the solution.

Every final solution was reevaluated by the same image with Weka J48 3.8.6, a pruned tree, confidence factor 0.25, and a minimum of two instances per leaf. The primary metric is macro-F1 on the 0–1 scale; macro precision and recall, weighted F1, accuracy, and per-class F1 were preserved as complementary measures.

The matrix has 27 arms: 24 distributed configurations, two monoliths, and the baseline. Each arm uses independent seeds 42–49, totaling 216 valid runs. Distributed configurations cross four RCL rankings (IG, GR, SU, and ReliefF), two controllers (VND and RVND), and three initial local-search preferences (BF, IWSS, and IWSSR). All three operators remain enabled in each controller’s portfolio. ReliefF uses 1,000 reference instances selected reproducibly from training with the run seed; classifier training, validation, and testing use the full splits.

Each algorithmic run received at most 3,000 s, including a 300-s reserve for shut-down and final evaluation. Search could also stop after 500 accepted improvements, with a minimum improvement of 0.0001. G-FShield uses cutoff 30, an initial solution of five features, and 100 iterations per operator. Monolith 1 runs GR followed by Bit-Flip, with cutoff 30, five features, and 100 local iterations. Monolith 2 runs GR followed by IWSS under VND, with the same cutoff and initial size and 100 local iterations. The baseline only trains and evaluates J48 with all 51 predictors. In every case, training fits the model, validation selects the best solution, and testing provides the common final measure.

All arms were confined to logical processors 8–15 on NUMA node 1, with an aggregate ceiling of eight CPUs and 16 GiB. Containers were sampled every 30 s with `docker stats`; CPU is summed in cores and memory is summed across containers belonging to the run. Other containers present on the host were recorded separately and are excluded from these sums. The campaign started on August 25, 2026 at 08:32 UTC and finished on September 1, 2026 at 03:36 UTC, before the ten-day limit, with no supervisor restart.

Twelve distributed attempts ended without a complete solution (`<NO_COMPLETE_SOLUTION>`) and were rejected by the artifact contract. The orchestrator reran only missing seed–arm combinations until each arm had eight valid results. There were consequently 228 attempts and 216 valid runs. All 208 search runs stopped at the time limit; the eight baseline runs exited normally.

In this chapter, a *run* is one valid, independent replicate of an arm and seed. A *final solution* is the subset selected on validation and evaluated once on test. An *internal record* is a candidate row or event generated within that run; internal records are correlated and are not treated as replicates or end-to-end solutions.

5.2 Predictive Quality and Dimensionality Reduction

Table 5 summarizes the G-FShield configuration selected by the highest median validation macro-F1, both monoliths, and the baseline. Each arm uses eight runs. RF–VND–IWSSR obtained median test macro-F1 0.9448, interquartile range 0.9441–0.9449, 11 features, and 78.4% reduction. The all-features baseline obtained 0.9378. The paired median difference was 0.0069 in favor of the selected configuration; this is a descriptive result, not demonstrated statistical superiority.

Table 5 – Final held-out test results and observed resources (medians of eight runs).

Arm	Macro-F1	Precision	Recall	Accuracy	$ S $	Reduction (%)	Time (s)	CPU (cores)	Median RAM (MiB)	Peak RAM (MiB)
RF-VND-IWSSR	0.9448	0.9463	0.9432	0.9343	11	78.4	2,815.3	2.104	4,726.0	5,858.3
Monolith 1 (GR-BF)	0.9397	0.9399	0.9396	0.9293	5	90.2	2,712.5	1.069	1,531.4	1,959.4
Monolith 2 (GR-IWSS)	0.8411	0.8571	0.8454	0.8412	5	90.2	2,725.9	1.003	1,394.7	1,788.9
All 51 features	0.9378	0.9402	0.9358	0.9272	51	0.0	72.5	1.044	748.4	820.0

F1, precision, and recall are macro averages. CPU is the temporal median of the sum across run containers and then the median across seeds. Median and peak RAM use the same boundary. Time is end-to-end, including startup, search, validation selection, controlled shutdown, and final evaluation.

Monolith 1 obtained median F1 0.9397; Monolith 2 had median 0.8411 and high dispersion (mean 0.7343; standard deviation 0.2013). The baseline is deterministic under the adopted setup and repeated 0.9378 for all eight seeds. Across the 24 distributed configurations, median F1 ranged from 0.8671 to 0.9448; the median across configurations was 0.9418. Every configuration produced eight distinct final subsets. Median subset size ranged from 5 to 17 features, corresponding to reductions from 90.2% to 66.7%.

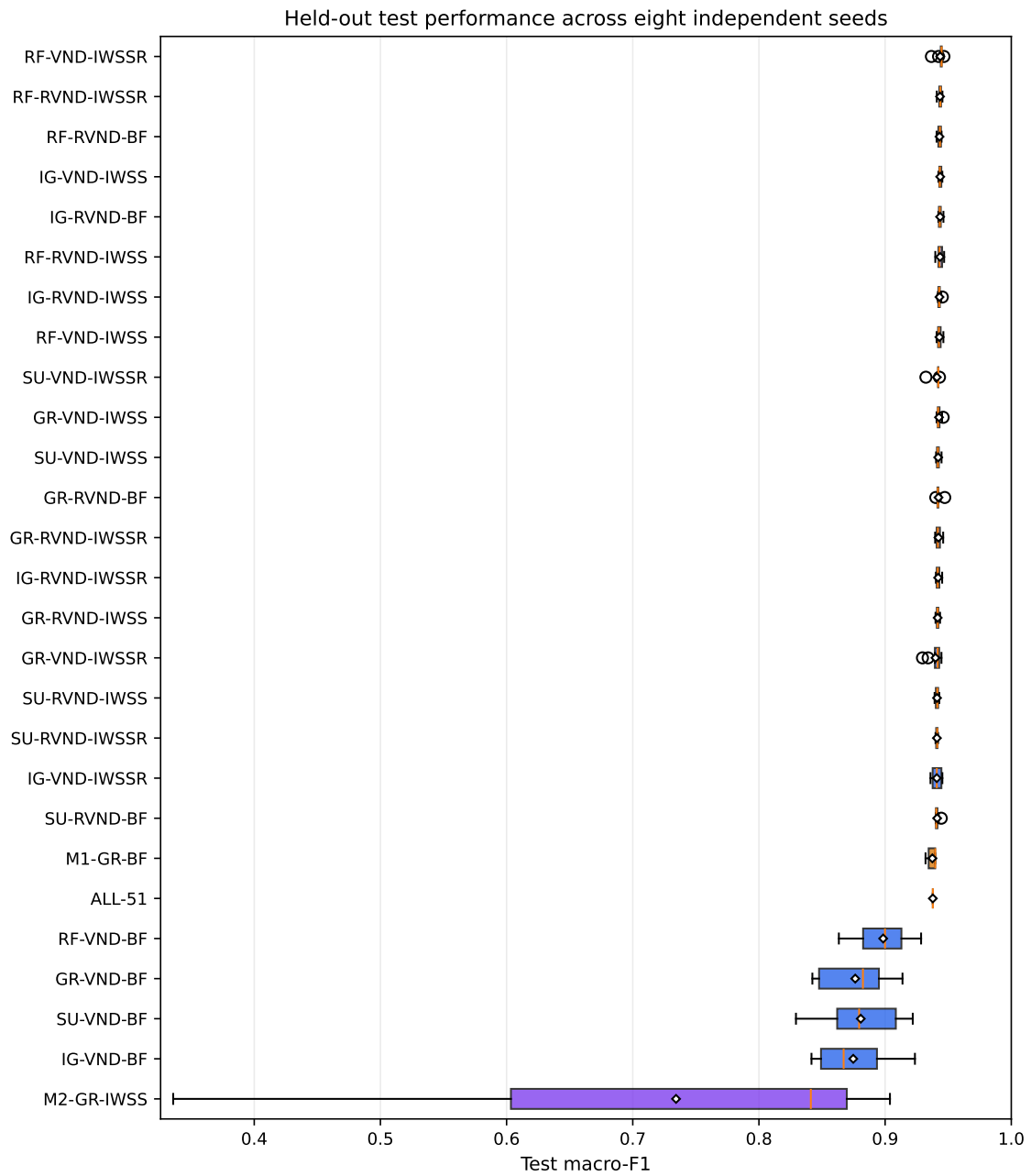


Figure 11 – Test macro-F1 distributions for all 27 arms and eight seeds per arm.

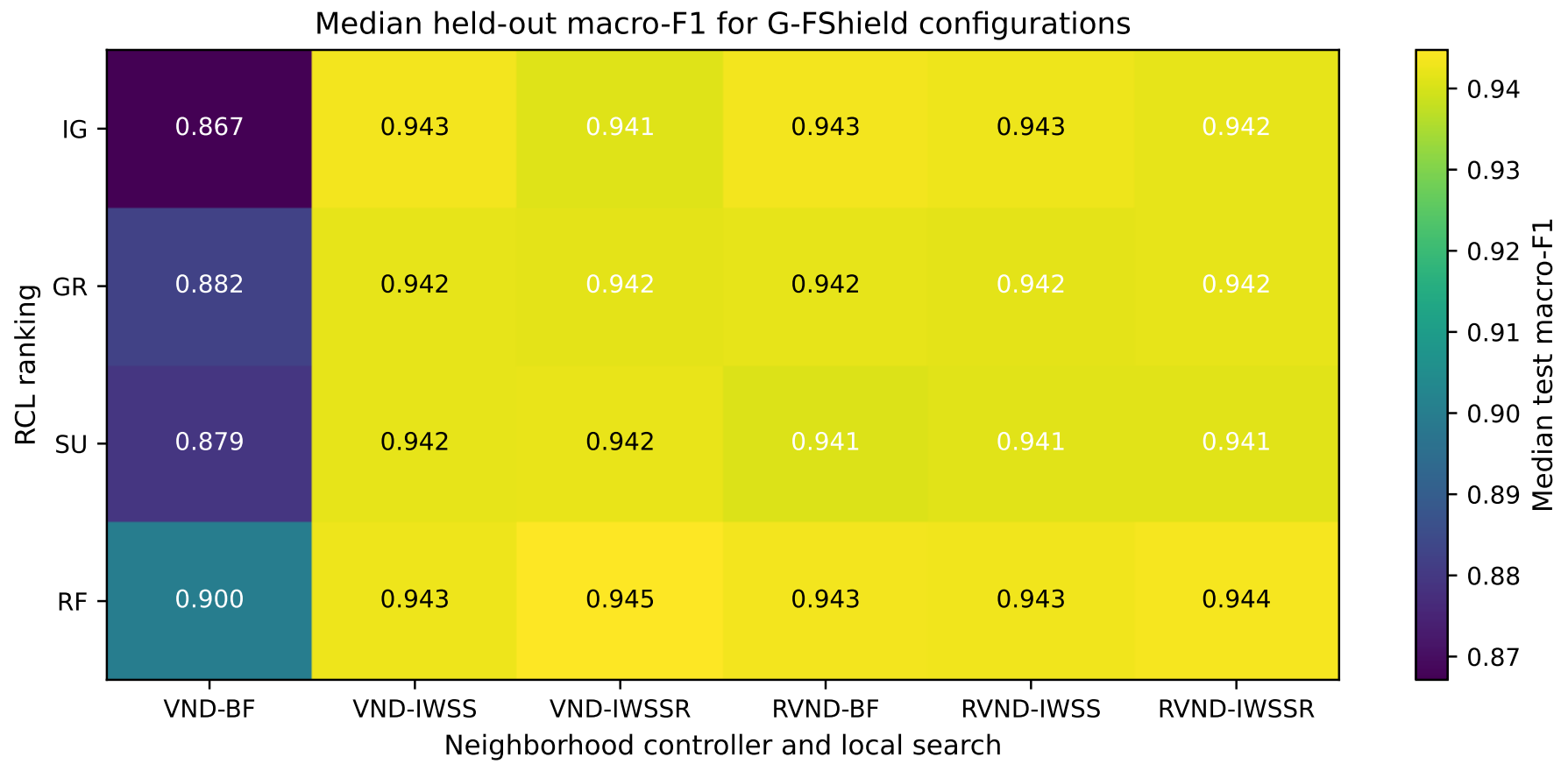


Figure 12 – Median test macro-F1 for the 24 distributed configurations.

Figure 12 shows that the four VND–BF combinations remained below configurations that effectively explored IWSS/IWSSR. This outcome depends on the protocol and the implemented search objective; it does not establish that an operator is universally inferior. Figure 13 presents the quality–reduction relationship. RF–VND–IWSSR descriptively exceeds the baseline’s F1 with 40 fewer features, while Monolith 1 uses only five features and preserves F1 close to the baseline.

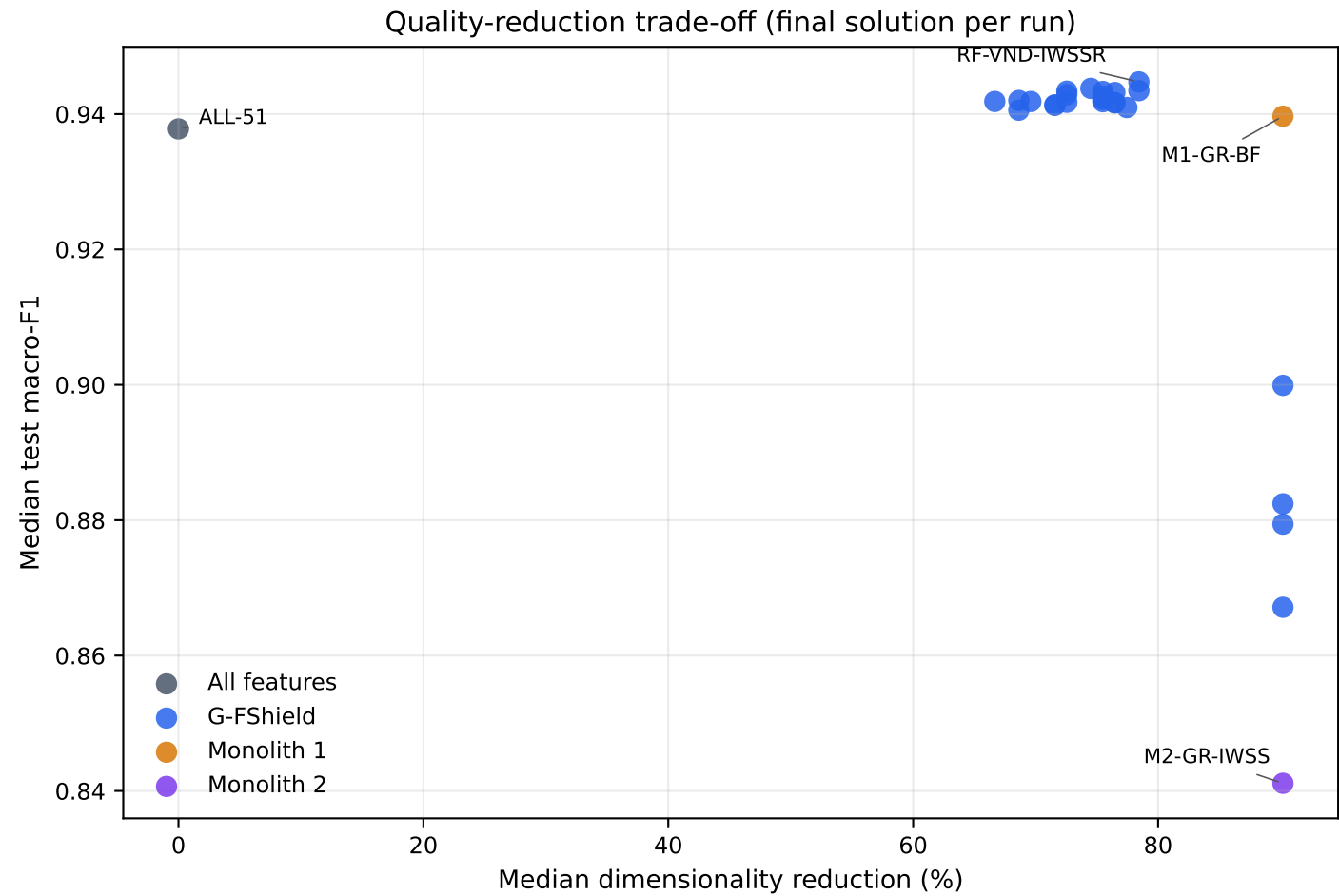


Figure 13 – Trade-off between median macro-F1 and dimensionality reduction of final solutions.

Per-class performance (Figure 14) exposes information hidden by the average: for RF-VND-IWSSR, median F1 is 0.7179 for *grayhole*, 0.8451 for *normal*, and at least 0.9565 for the other seven classes. Macro-F1 close to 0.945 therefore does not imply uniform performance across attacks.

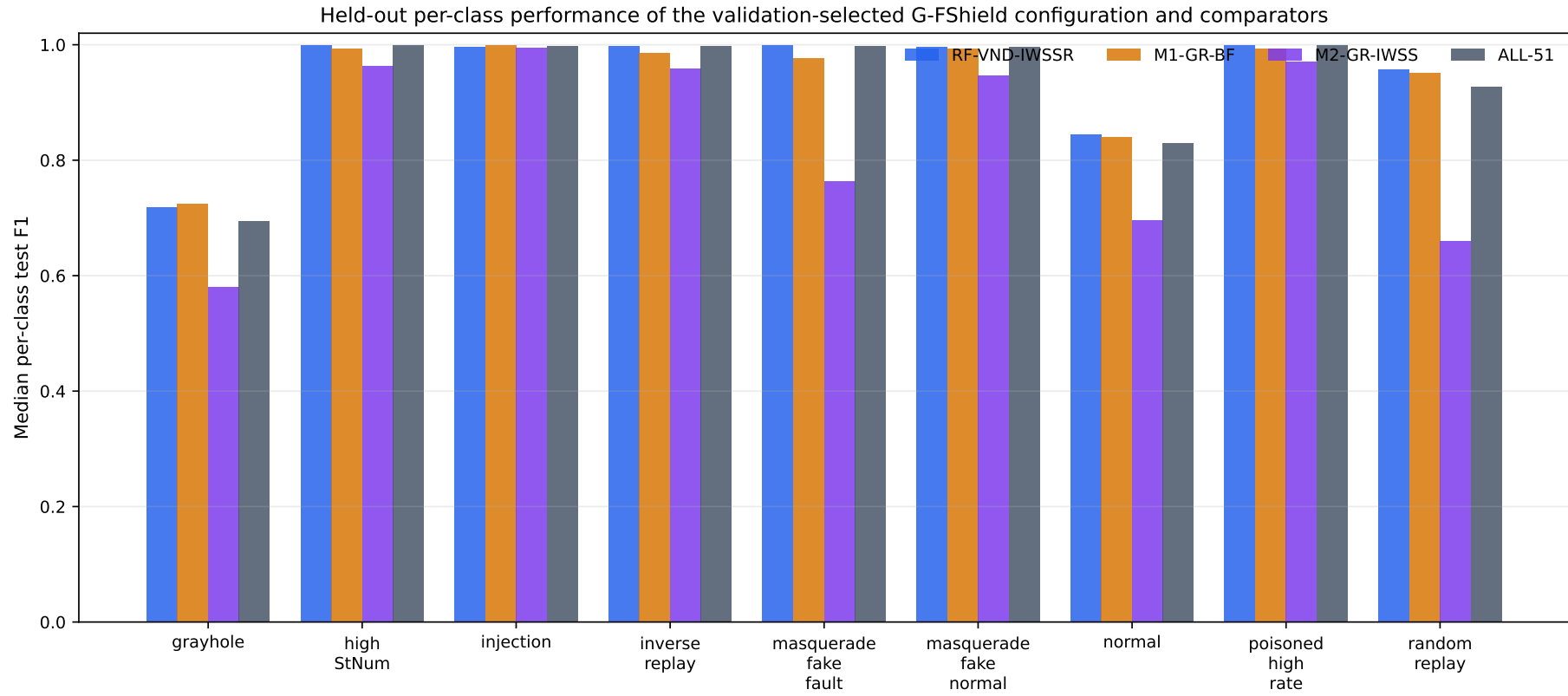


Figure 14 – Median per-class test F1 for the validation-selected distributed configuration and comparators.

5.3 Time, Resources, and Statistical Interpretation

Because all 208 searches stopped at the same limit, median end-to-end time ranged from 2,706 to 2,847 s by configuration. These values do not demonstrate faster convergence. None of the 24 distributed configurations reached internal $F1 \geq 0.95$. Audit of the image-source commit confirmed that this internal score was already multiclass macro-F1; the binary routine containing `normalClass=0` was disabled. Nevertheless, the campaign did not preserve an equivalent cumulative timeline for every candidate in both monoliths: their CSV files record individual evaluation duration rather than elapsed time since run start. A valid time-to-threshold comparison therefore cannot be computed from this campaign. That outcome requires comparable monotonic timestamps in both arms, validation-only evaluation, and censoring at the same deadline.

Under the same aggregate ceiling, distributed runs used a global median of 3.56 cores and median peak of 5.45, compared with approximately 1.07/1.83 for Monolith 1 and 1.00/1.82 for Monolith 2. Distributed memory had a global median of 5,317 MiB and median peak of 6,757 MiB, compared with median peaks of 1,959 and 1,789 MiB for the monoliths. This quantifies observed consumption but does not prove efficiency: the distributed pipeline aggregates multiple Java processes and messaging infrastructure, while the compared arms execute different rankings and local-search operators.

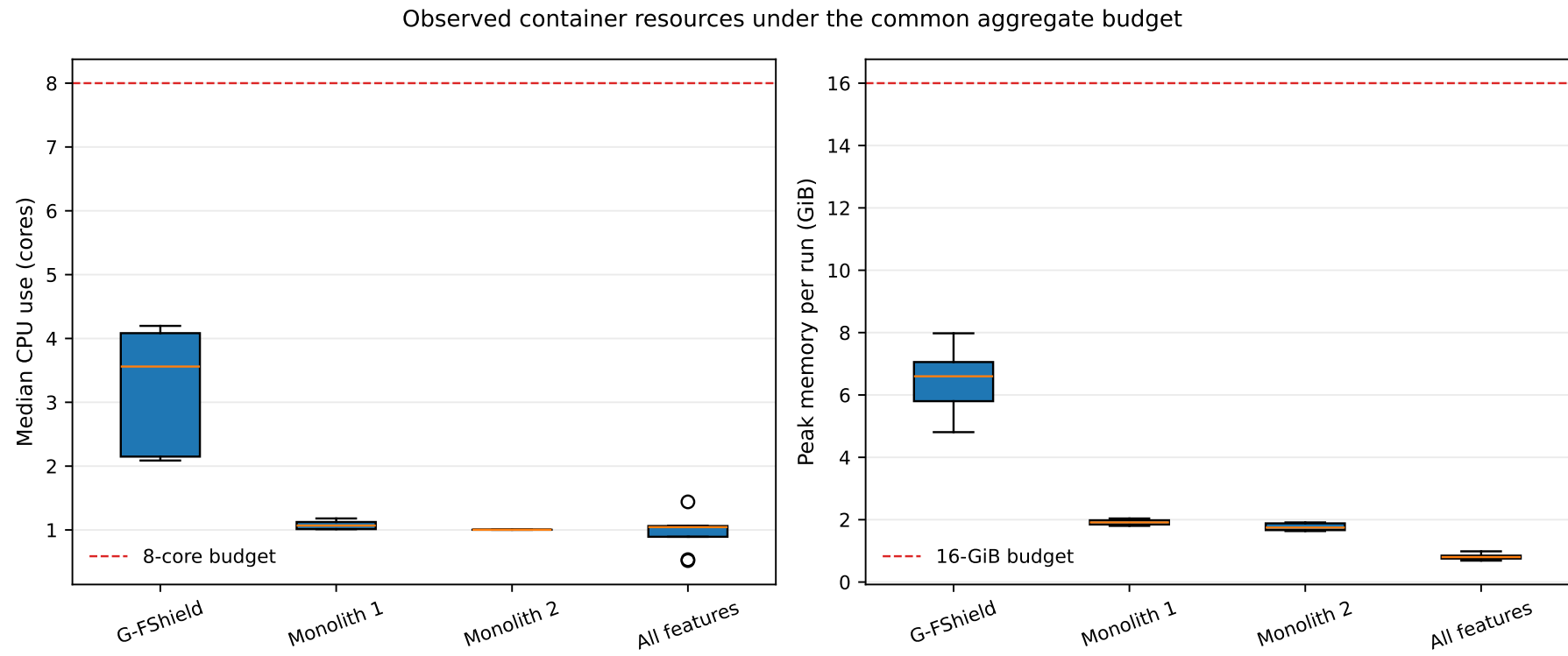


Figure 15 – Observed CPU and memory by architecture under the common aggregate ceiling of eight CPUs and 16 GiB.

As an exploratory analysis, each distributed configuration was paired by seed with each comparator using a two-sided exact sign-flip test and paired rank-biserial effect size. Holm correction was applied separately to each comparator’s family of 24 tests. RF–VND–IWSSR recorded seven wins and one loss against the baseline and eight wins against each monolith; all three adjusted p -values were 0.1875. None of the 72 adjusted comparisons was below 0.05. With eight seeds, intervals and tests have low power; the results support descriptive differences, not claims of significance or architectural causality.

5.4 Observability and Traceability

Each run preserves `campaign_id`, `run_id`, `arm`, `seed`, `parameters`, `data hashes`, `resolved command`, `states`, `stop reason`, `UTC` and `monotonic timestamps`, `selected solution`, `final metrics`, `best-solution messages`, `internal CSV files`, `logs`, and `container telemetry`. A checksum manifest for every run detects artifact changes, and campaign state reconstructs attempts, rejections, and retries.

In the application, the backend persists events for initial solution, neighborhood restart, search progress, refined solution, and best solution, plus `topic`, `partition`, `offset`, `state`, `seedId`, `timestamps`, and `JSON content` when available. The formal campaign adds `campaign_id`, `run_id`, and `request_id` to its experimental flows, removing ambiguity among concurrent runs within this protocol. Event completeness, instrumentation overhead, loss/duplication under failure, recovery time, and reprocessing success were not measured. Fault tolerance and resilience therefore remain design objectives.

5.5 Threats to Validity

Internal validity. CPU, memory, CPU set, data, seeds, time limit, J48, and macro-F1 were controlled. The comparators nevertheless execute different algorithmic sequences: selected G-FShield uses RF–VND–IWSSR, Monolith 1 uses GR–BF, and Monolith 2 uses GR–IWSS. The campaign also neither alternated execution order nor recorded an equivalent cumulative candidate trajectory for the monoliths. These differences prevent attributing outcomes solely to architecture. The 12 attempts without a complete solution were rejected and retried; this mechanism may favor runs that can initialize, although it preserves the valid seed matrix.

Construct validity. The primary measure is macro-F1 on held-out test data, and runs, final solutions, and internal records are separated. The median per arm reduces the influence of extremes but does not summarize the full distribution. CPU and memory measure containers, not energy or monetary cost. The 0.95 threshold is an analytical convention, not an operational requirement.

Conclusion validity. Pairing by seed and Holm correction avoid treating internal candidates as independent observations and limit false positives in the tested family.

Nevertheless, eight seeds provide low power, the baseline is deterministic, and selecting the best configuration by median validation performance introduces selection among 24 alternatives. Statistical significance is not claimed.

External validity. Evidence covers one server, one IEC 61850 corpus, one split, and J48. It does not demonstrate generalization to other corpora, classifiers, concurrent loads, replica counts, hardware, failures, or continuous ingestion. Future work should repeat the protocol on multiple datasets and evaluate scaling and recovery separately.

6 CONCLUSION

This dissertation presented *G-FShield*, a distributed, event-driven, observable architecture for GRASP-FS in IDS applied to CPS. The implementation separates rankings and initial construction, neighborhood control, local-search operators, verification, and monitoring; Apache Kafka integrates the stages, while the backend and interface expose persisted execution signals.

The formal campaign answered RQ1 with common end-to-end units. It evaluated 24 G-FShield arms, two monoliths, and an all-features baseline, each under eight independent seeds. RF-VND-IWSSR was selected on validation and obtained median test macro-F1 0.9448 with 11 of 51 predictors, a 78.4% reduction. The baseline obtained 0.9378, Monolith 1 0.9397, and Monolith 2 0.8411. The difference from the baseline is descriptive; none of the 72 exploratory Holm-adjusted comparisons had $p < 0.05$.

For RQ2, distributed-configuration medians ranged from 0.8671 to 0.9448. VND-BF combinations remained below those that explored IWSS/IWSSR in this protocol, but all 208 searches stopped at the time limit, so there is no evidence of convergence. Although the internal score was already macro-F1, the campaign did not record cumulative candidate time equivalently in the monoliths and therefore cannot compare time to macro-F1 0.95. For RQ3, the common ceiling was eight CPUs and 16 GiB: distributed runs consumed global medians of 3.56 cores and 5,317 MiB, above the monoliths, without this proving architectural efficiency or inefficiency.

For RQ4, the manifest, resolved commands, hashes, campaign and run identifiers, seeds, states, stop reasons, timestamps, messages, internal CSV files, logs, final metrics, and telemetry reconstruct what was captured. The application also persists events by seed and stage. Telemetry completeness and cost, loss and duplication under failure, recovery time, and reprocessing were not measured; resilience, fault tolerance, and scalability remain design objectives.

The principal remaining threat is confounding between architecture and algorithm: selected G-FShield executes RF-VND-IWSSR, whereas the monoliths execute GR-BF and GR-IWSS. All use J48 and macro-F1, but their algorithmic work and candidate trajectories differ; the common evaluation therefore makes final solutions comparable without isolating the effect of distribution. Evidence is also limited to one server, one corpus, one split, J48, and eight seeds.

Future work should:

- complete a paired campaign with the same RF-VND-IWSSR sequence, objective function, seeds, deadline, and aggregate budget in distributed and monolithic arms;
- increase the number of seeds and preregister adequately powered confirmatory comparisons;
- record intermediate candidates on a monotonic timeline to measure time to

target without consulting test data;

- run scale-out studies that vary replicas and concurrency and report throughput per cost;
- measure telemetry completeness, overhead, recovery, and reprocessing;
- repeat the protocol with other corpora, classifiers, splits, and hardware.

In summary, G-FShield demonstrates a distributed and traceable GRASP-FS integration and produces an auditable experimental dataset. Its quantitative results delimit observed quality, dimensionality reduction, and resource consumption without extrapolating to statistical significance, architectural causality, or operational readiness.

BIBLIOGRAPHY

- AFTAB, M. A. et al. Iec 61850 based substation automation system: A survey. **International Journal of Electrical Power & Energy Systems**, v. 120, 2020. Cited 2 times on pages 14 and 16.
- ALCARAZ, C.; LOPEZ, J. A security analysis for wireless sensor mesh networks in highly critical systems. **IEEE Transactions on Systems, Man, and Cybernetics, Part C: Applications and Reviews**, v. 40, n. 4, p. 419–428, 2010. Cited 2 times on pages 12 and 16.
- ANDERSON, C. Docker [software engineering]. **IEEE Software**, IEEE, v. 32, n. 3, p. 102–c3, 2015. Cited on page 15.
- BEYER, B. et al. **Site Reliability Engineering: How Google Runs Production Systems**. [S.l.]: O'Reilly Media, 2016. Cited 2 times on pages 15 and 16.
- CÁRDENAS, A. A.; AMIN, S.; SASTRY, S. Research challenges for the security of control systems. In: **3rd USENIX Workshop on Hot Topics in Security (HotSec 08)**. San Jose, CA: USENIX Association, 2008. Disponível em: <<https://www.usenix.org/conference/hotsec-08/research-challenges-security-control-systems>>. Cited 3 times on pages 12, 14, and 16.
- DUARTE, A. et al. Variable neighborhood descent. In: MARTÍ, R.; PARDALOS, P. M.; RESENDE, M. G. C. (Ed.). **Handbook of Heuristics**. Cham: Springer International Publishing, 2018. p. 341–367. Cited on page 27.
- ESSEGHIR, M. A. Effective wrapper-filter hybridization through grasp schemata. In: LIU, H. et al. (Ed.). **Proceedings of the Fourth International Workshop on Feature Selection in Data Mining**. Hyderabad, India: PMLR, 2010. (Proceedings of Machine Learning Research, v. 10), p. 45–54. Disponível em: <<https://proceedings.mlr.press/v10/esseghir10a.html>>. Cited on page 16.
- FARIA, N. N. R. et al. Microserviços para geração de rcls no grasp-fs: Uma abordagem escalável para seleção de features na detecção de intrusões em sistemas ciber-físicos. In: **Anais do XXIII Simpósio Brasileiro de Segurança da Informação e de Sistemas Computacionais (SBSeg 2023)**. [S.l.]: Sociedade Brasileira de Computação, 2023. p. 55–68. Cited 2 times on pages 16 and 23.
- FASEEHA, U. et al. Observability in microservices: An in-depth exploration of frameworks, challenges, and deployment paradigms. **IEEE Access**, v. 13, p. 72011–72039, 2025. Cited on page 16.
- FRANCIS, G. T.; SOURI, A.; İNANÇ, N. A systematic review of metaheuristic based feature selection strategies for cyber-attack detection in the iiot. **Artificial Intelligence Review**, v. 59, 2026. Cited on page 16.
- GUYON, I.; ELISSEEFF, A. An introduction to variable and feature selection. **Journal of Machine Learning Research**, v. 3, n. Mar, p. 1157–1182, 2003. Disponível em: <<https://www.jmlr.org/papers/v3/guyon03a.html>>. Cited 3 times on pages 12, 14, and 16.

HALL, M. A. **Correlation-based Feature Selection for Machine Learning**. Tese (Doutorado) — University of Waikato, 1999. Disponível em: <<https://hdl.handle.net/10289/15043>>. Cited 3 times on pages 12, 14, and 16.

INTERNATIONAL ELECTROTECHNICAL COMMISSION. **IEC 61850-7-2: Communication Networks and Systems for Power Utility Automation – Part 7-2: Basic Information and Communication Structure – Abstract Communication Service Interface (ACSI)**. 2.0. ed. [S.l.], 2010. Disponível em: <<https://webstore.iec.ch/en/publication/6015>>. Cited 2 times on pages 14 and 16.

KONONENKO, I. Estimating attributes: Analysis and extensions of relief. In: BERGADANO, F.; RAEDT, L. D. (Ed.). **Machine Learning: ECML-94**. [S.l.]: Springer Berlin Heidelberg, 1994. (Lecture Notes in Computer Science, v. 784), p. 171–182. Cited on page 14.

KREPS, J.; NARKHEDE, N.; RAO, J. Kafka: A distributed messaging system for log processing. In: **Proceedings of the NetDB Workshop**. [S.l.: s.n.], 2011. Cited 4 times on pages 15, 16, 22, and 31.

MACKIEWICZ, R. E. Overview of iec 61850 and benefits. In: **2006 IEEE Power Engineering Society General Meeting**. [S.l.]: IEEE, 2006. p. 1–8. Cited 2 times on pages 14 and 16.

MATEUS-COELHO, N.; CRUZ-CUNHA, M.; FERREIRA, L. G. Security in microservices architectures. **Procedia Computer Science**, Elsevier, v. 181, p. 1225–1236, 2021. Cited on page 16.

MITCHELL, R.; CHEN, I.-R. A survey of intrusion detection techniques for cyber-physical systems. **ACM Computing Surveys**, v. 46, n. 4, p. 1–29, 2014. Cited 3 times on pages 12, 14, and 16.

MLADENović, N.; HANSEN, P. Variable neighborhood search. **Computers & Operations Research**, v. 24, n. 11, p. 1097–1100, 1997. Cited on page 27.

NEWMAN, S. **Building Microservices**. 2. ed. [S.l.]: O'Reilly Media, 2021. Cited 3 times on pages 15, 16, and 22.

QUINCOZES, S. E. et al. A survey on intrusion detection and prevention systems in digital substations. **Computer Networks**, v. 184, 2021. Cited on page 16.

QUINCOZES, S. E. et al. Ereno: An extensible tool for generating realistic iec-61850 intrusion detection datasets. In: **Anais Estendidos do XXII Simpósio Brasileiro de Segurança da Informação e de Sistemas Computacionais (SBSeg Estendido 2022)**. [S.l.]: Sociedade Brasileira de Computação, 2022. p. 1–8. Cited on page 16.

QUINCOZES, S. E. et al. On the performance of grasp-based feature selection for cps intrusion detection. **IEEE Transactions on Network and Service Management**, v. 19, n. 1, p. 614–626, 2022. Cited 5 times on pages 12, 15, 16, 23, and 27.

QUINCOZES, S. E. et al. An extended assessment of metaheuristics-based feature selection for intrusion detection in cps perception layer. **Annals of Telecommunications**, v. 77, n. 7–8, p. 457–471, 2022. Cited 5 times on pages 12, 15, 16, 23, and 27.

- QUINCOZES, S. E. et al. Grasp-based feature selection for intrusion detection in cps perception layer. In: **2020 4th Conference on Cloud and Internet of Things (CIoT)**. [S.l.]: IEEE, 2020. p. 41–48. Cited 5 times on pages 12, 15, 16, 23, and 27.
- QUINLAN, J. R. Induction of decision trees. **Machine Learning**, v. 1, n. 1, p. 81–106, 1986. Cited on page 14.
- QUINLAN, J. R. **C4.5: Programs for Machine Learning**. San Mateo, CA: Morgan Kaufmann, 1993. ISBN 978-1-55860-238-0. Disponível em: <<https://www.sciencedirect.com/book/9780080500584/c4-5>>. Cited on page 14.
- RESENDE, M. G. C.; RIBEIRO, C. C. **Optimization by GRASP: Greedy Randomized Adaptive Search Procedures**. New York, NY: Springer, 2016. Cited 4 times on pages 12, 15, 16, and 23.
- ROBNIK-ŠIKONJA, M.; KONONENKO, I. Theoretical and empirical analysis of relieff and rrelieff. **Machine Learning**, v. 53, n. 1, p. 23–69, 2003. Cited on page 14.
- SAEYS, Y.; INZA, I.; LARRAÑAGA, P. A review of feature selection techniques in bioinformatics. **Bioinformatics**, v. 23, n. 19, p. 2507–2517, 2007. Cited 3 times on pages 12, 14, and 16.
- SILVA, E. F. C. et al. Gdls-fs: Scaling feature selection for intrusion detection with grasp-fs and distributed local search. In: **Advanced Information Networking and Applications: Proceedings of the 37th International Conference on Advanced Information Networking and Applications (AINA-2023), Volume 2**. [S.l.]: Springer, Cham, 2023. (Lecture Notes in Networks and Systems, v. 654), p. 199–210. Cited 2 times on pages 16 and 27.
- SUBRAMANIAN, A. et al. A parallel heuristic for the vehicle routing problem with simultaneous pickup and delivery. **Computers & Operations Research**, Elsevier, v. 37, n. 11, p. 1899–1911, 2010. Cited 2 times on pages 16 and 27.
- TANENBAUM, A. S.; STEEN, M. van. **Distributed Systems: Principles and Paradigms**. 2. ed. Upper Saddle River, NJ, USA: Prentice Hall, 2007. Cited 3 times on pages 15, 16, and 22.
- TERMANINI, A. et al. Using machine learning to detect network intrusions in industrial control systems: A survey. **International Journal of Information Security**, v. 24, n. 1, 2025. Cited on page 16.
- USMAN, M. et al. A survey on observability of distributed edge & container-based microservices. **IEEE Access**, v. 10, p. 86904–86919, 2022. Cited 2 times on pages 15 and 16.
- WU, X. et al. Online feature selection with streaming features. **IEEE Transactions on Pattern Analysis and Machine Intelligence**, IEEE, v. 35, n. 5, p. 1178–1192, 2013. Cited on page 16.
- YUSTA, S. C. Different metaheuristic strategies to solve the feature selection problem. **Pattern Recognition Letters**, v. 30, n. 5, p. 525–534, 2009. Cited on page 16.

Appendices

APPENDIX A – FRONTEND GALLERY

This appendix brings together high-resolution captures of the *G-FShield* operational interface. The purpose is to show navigation, analytical panels, and administrative screens without polluting the dissertation’s main flow. The images are organized by usage function so that the reader can clearly follow the progression from authentication to comparison, monitoring, and administration.

A.1 Access and Overview

The first screens present the platform entry point and the initial dashboard, and Figures 16 and 17 show that transition from access to consolidated overview.

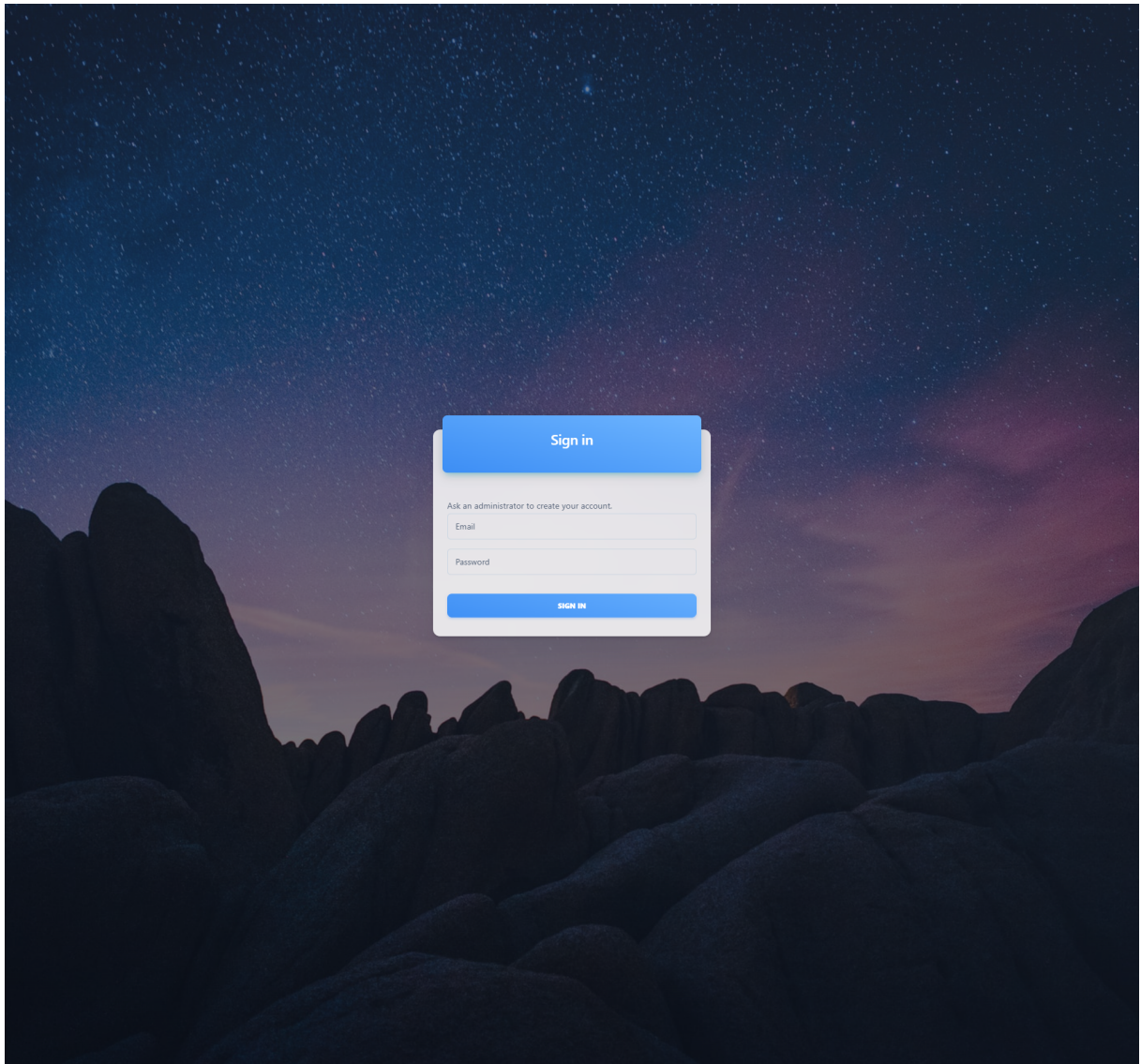


Figure 16 – Authentication screen.

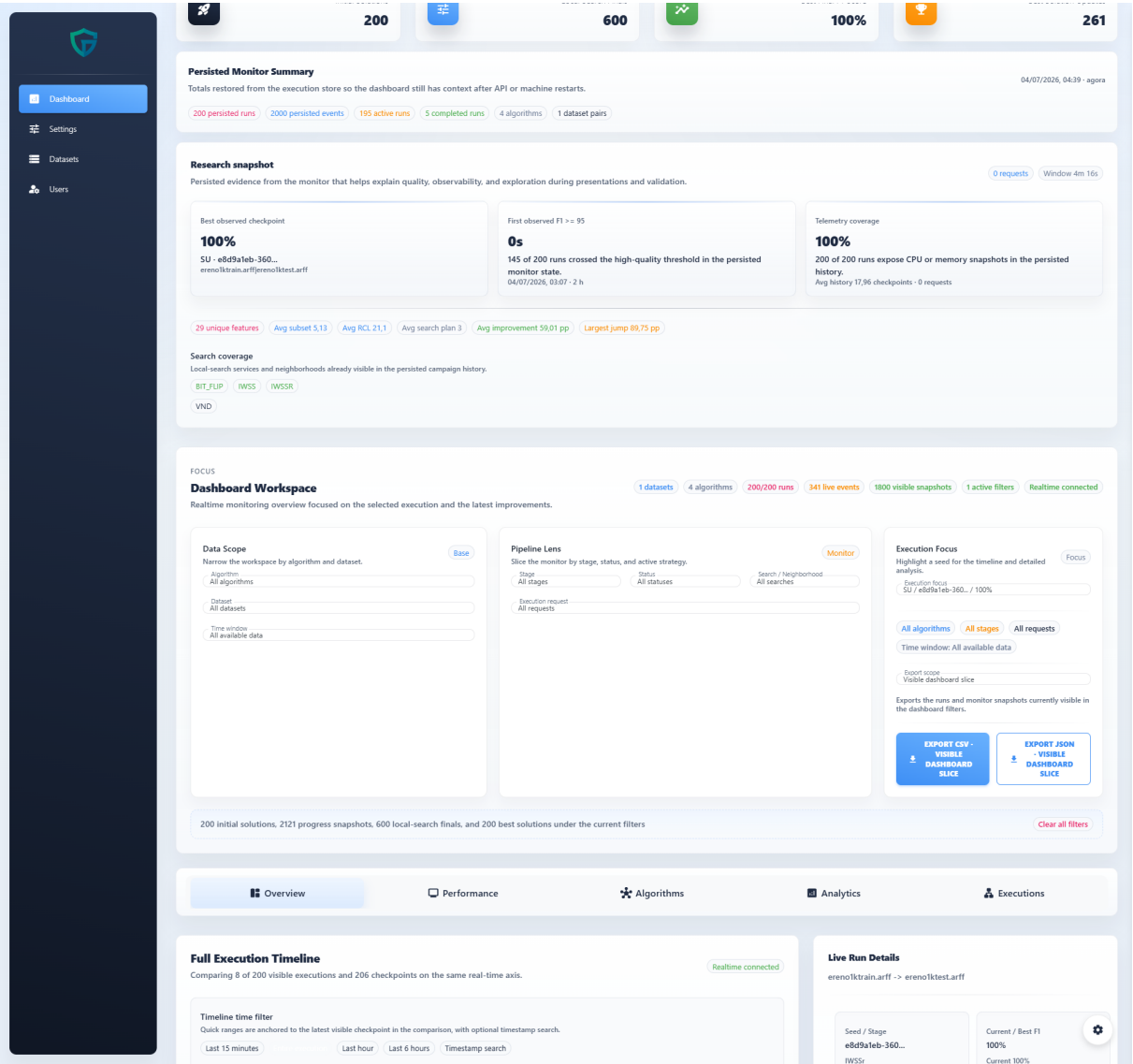


Figure 17 – Overview dashboard.

A.2 Performance and Algorithms

This section shows how the user moves between the performance and algorithm-comparison panels, and Figures 18 and 19 detail that reading.

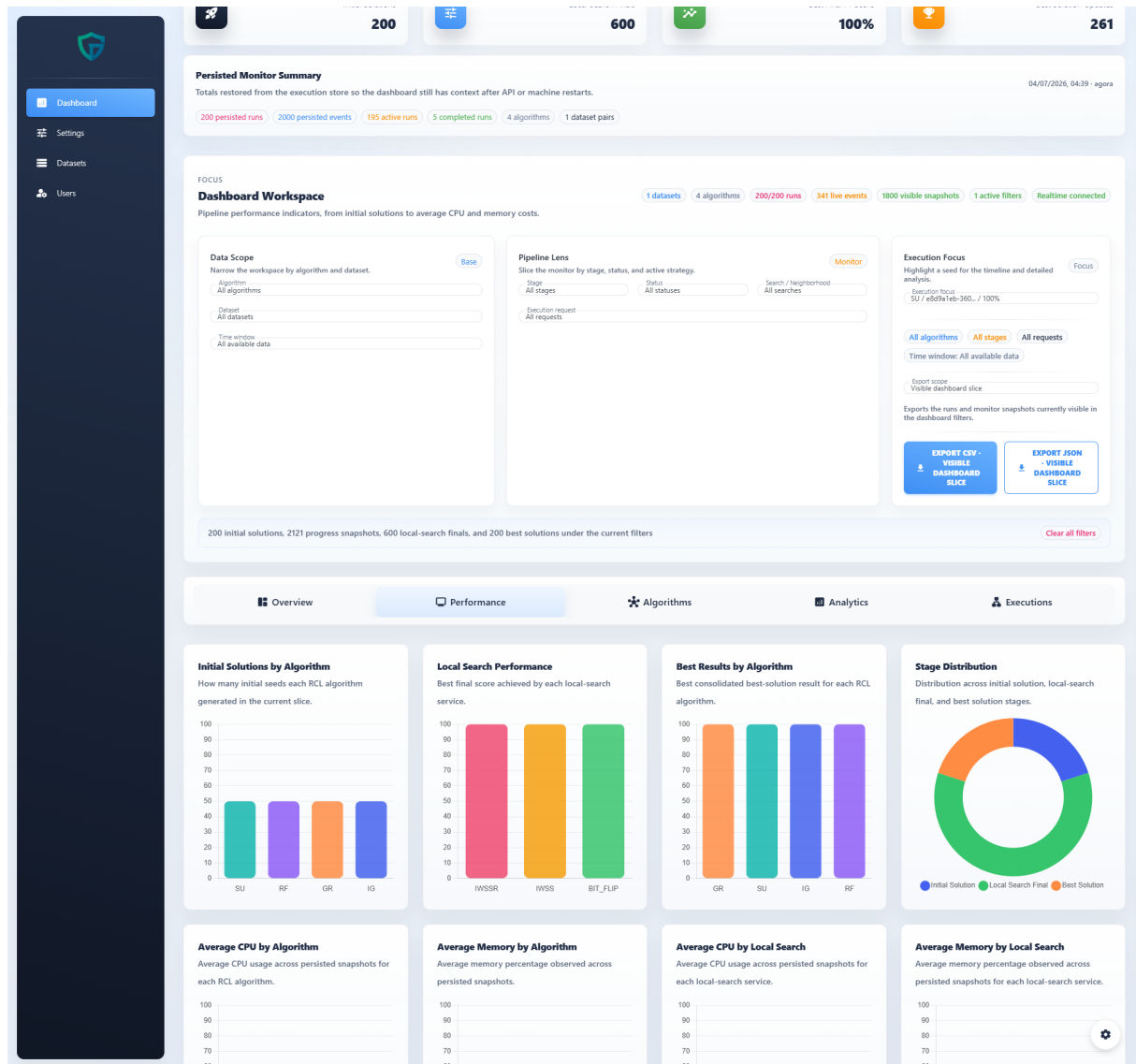


Figure 18 – Performance screen.

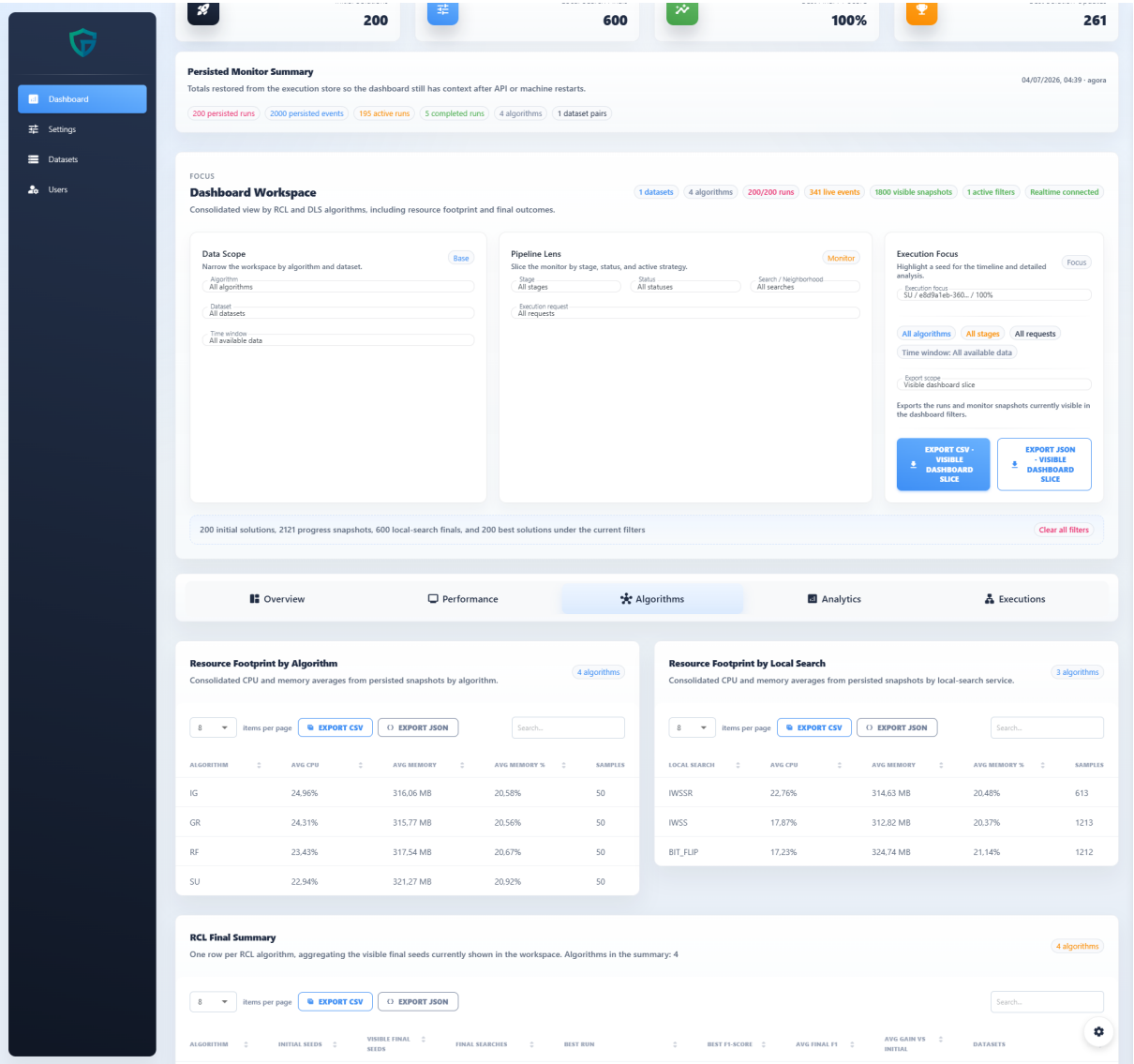


Figure 19 – Algorithm comparison screen.

A.3 Analytics

The analytics panels summarize execution activity and help expose aggregate trends, event-level details, and persisted topics. Figures 20, 21, 22, and 23 organize that view in layers.

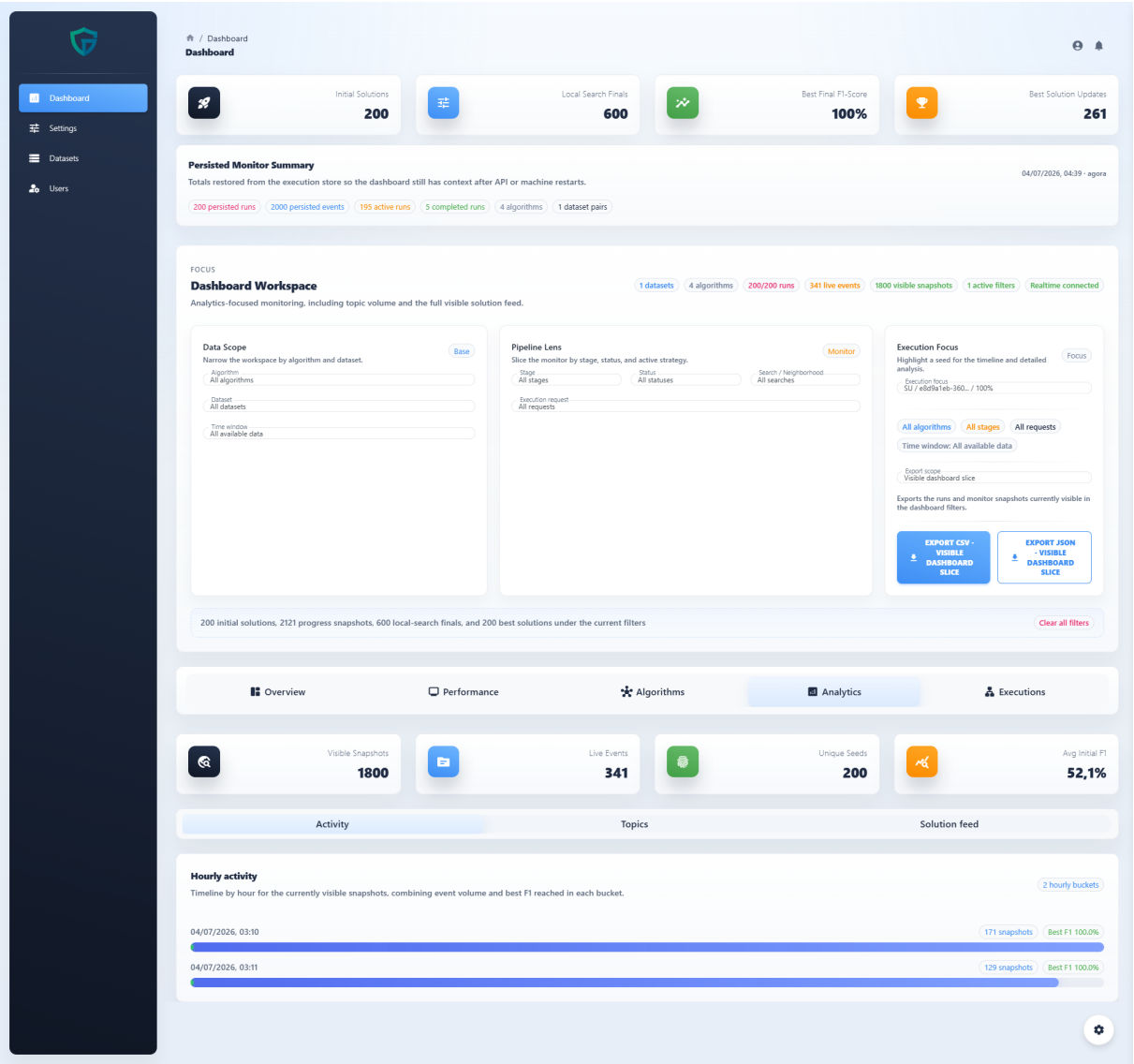


Figure 20 – Aggregated analytics activity.

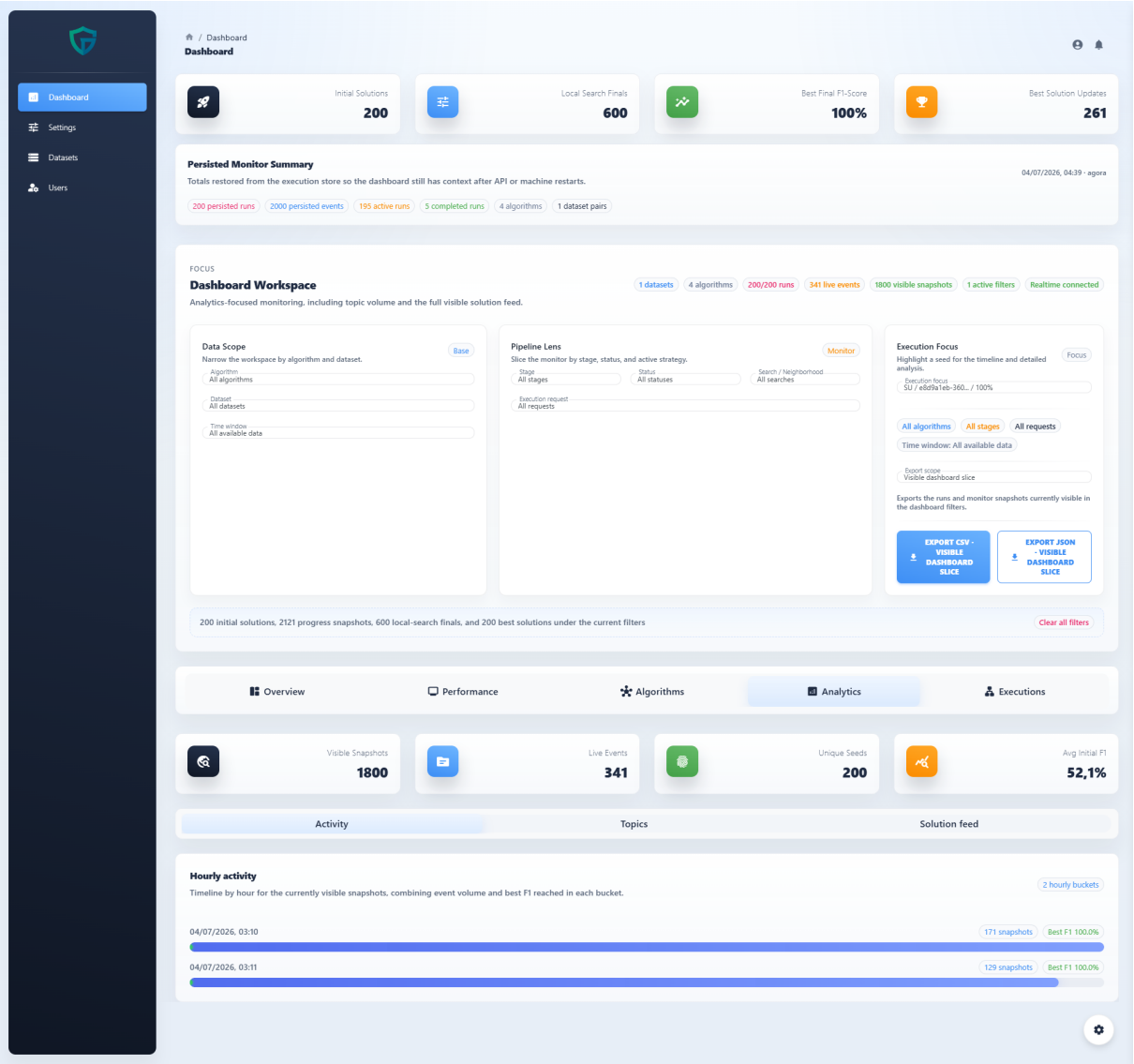


Figure 21 – Analytics activity detail.

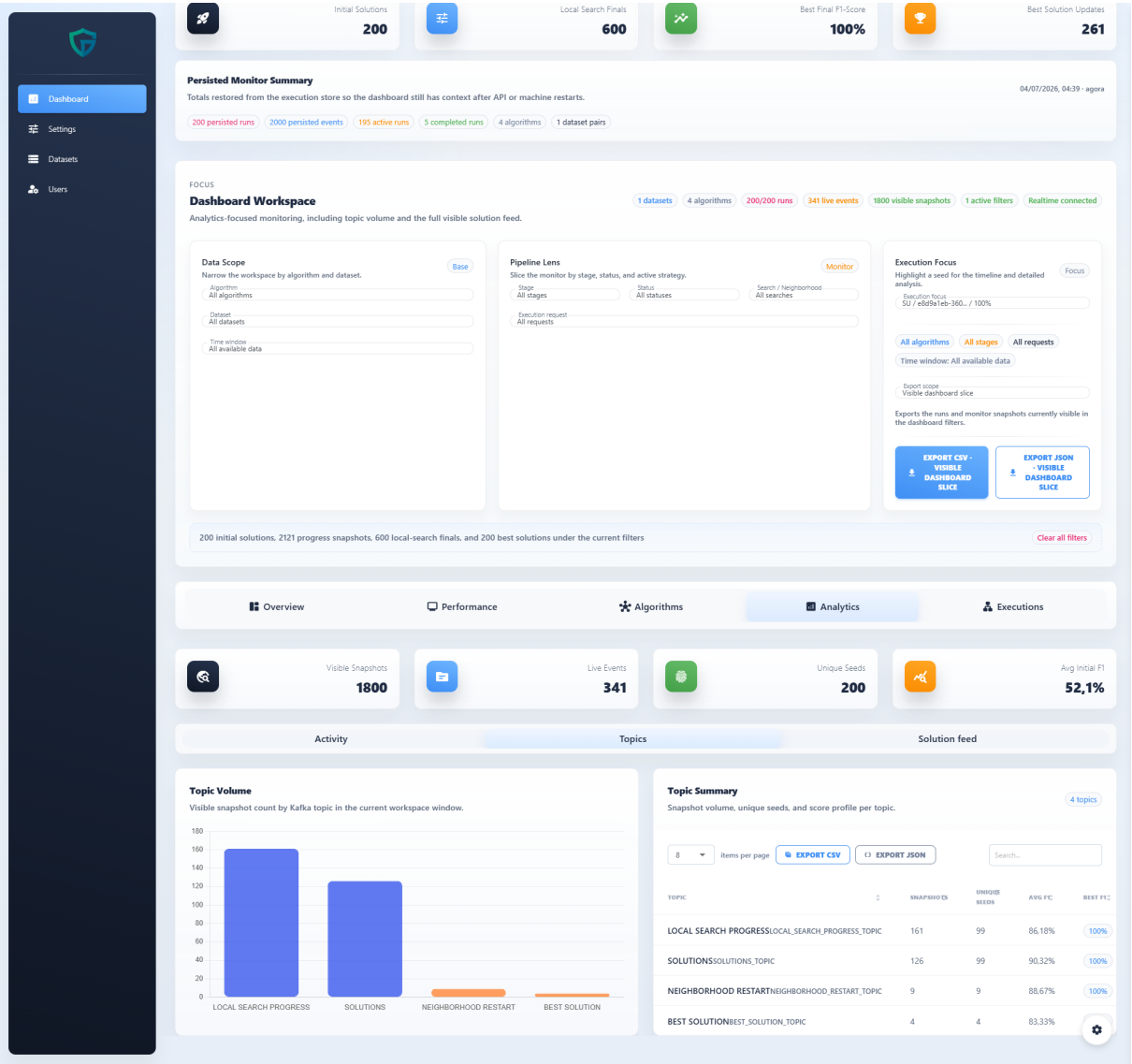


Figure 22 – Analytics topics.

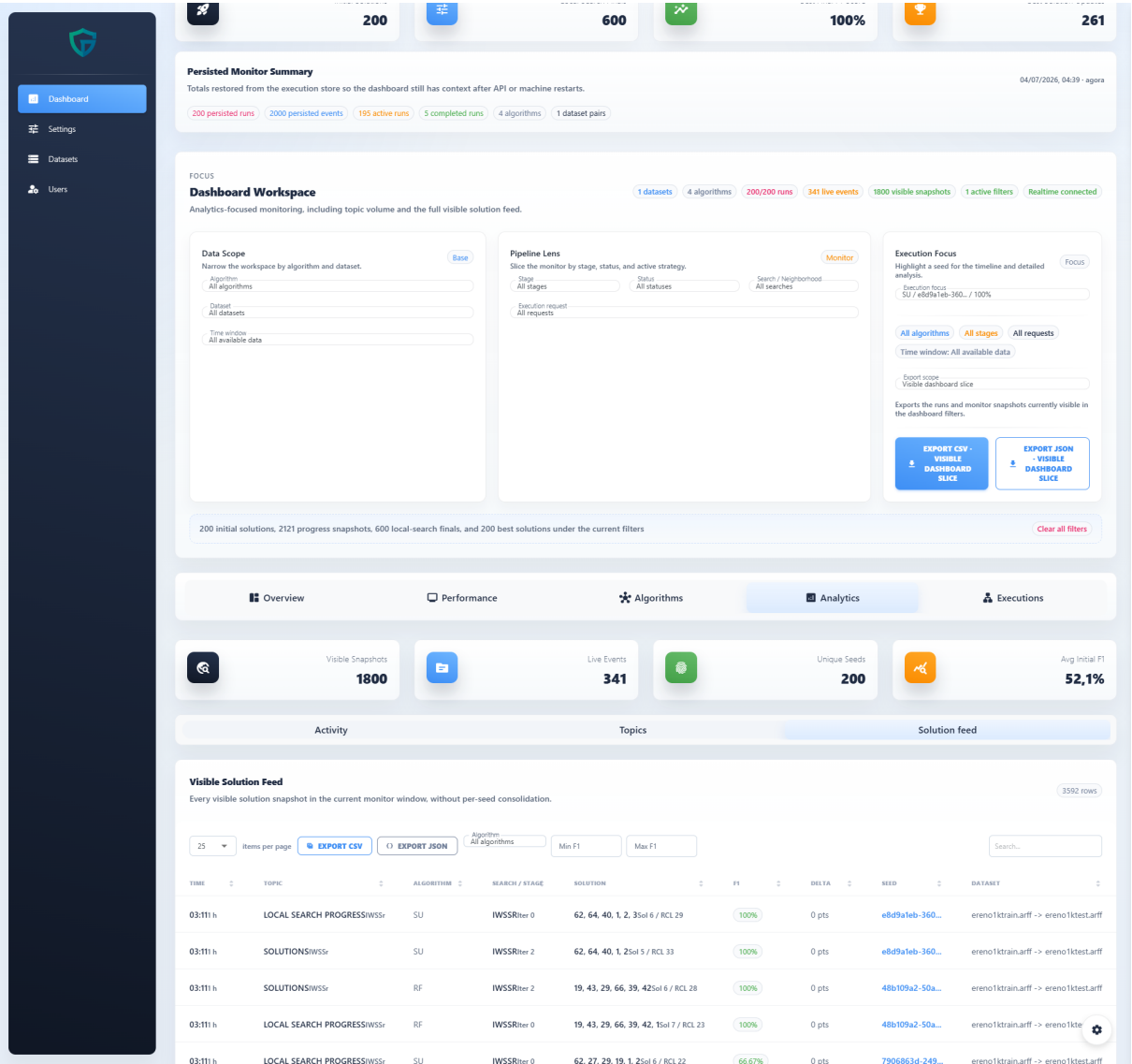


Figure 23 – Persisted analytics feed.

A.4 Run Comparison

This part highlights run comparison and inspection of selected seeds, reinforcing the platform’s exploratory and auditable nature. Figures 24, 25, and 26 show how that comparison works in practice.

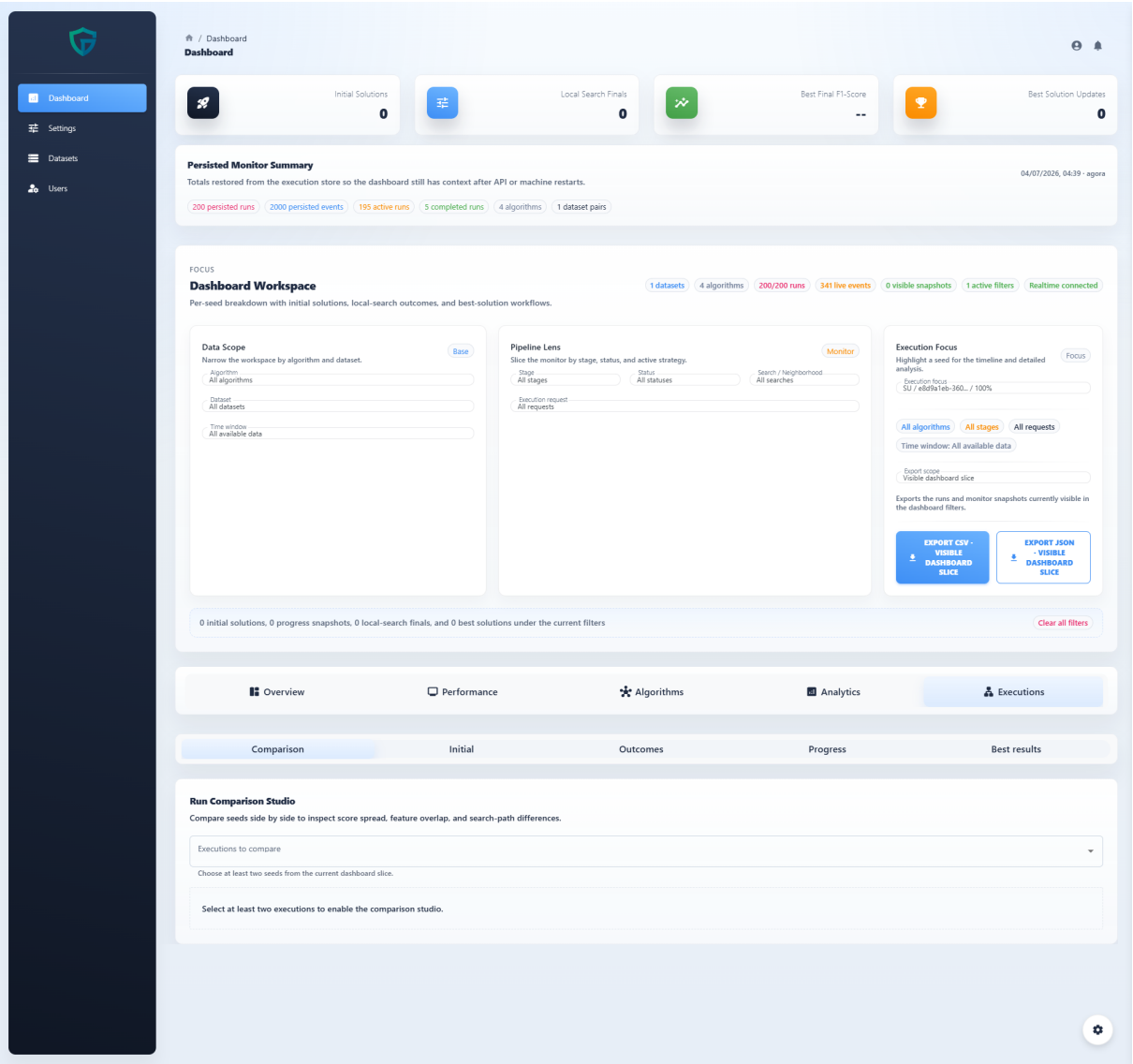


Figure 24 – Run comparison screen.

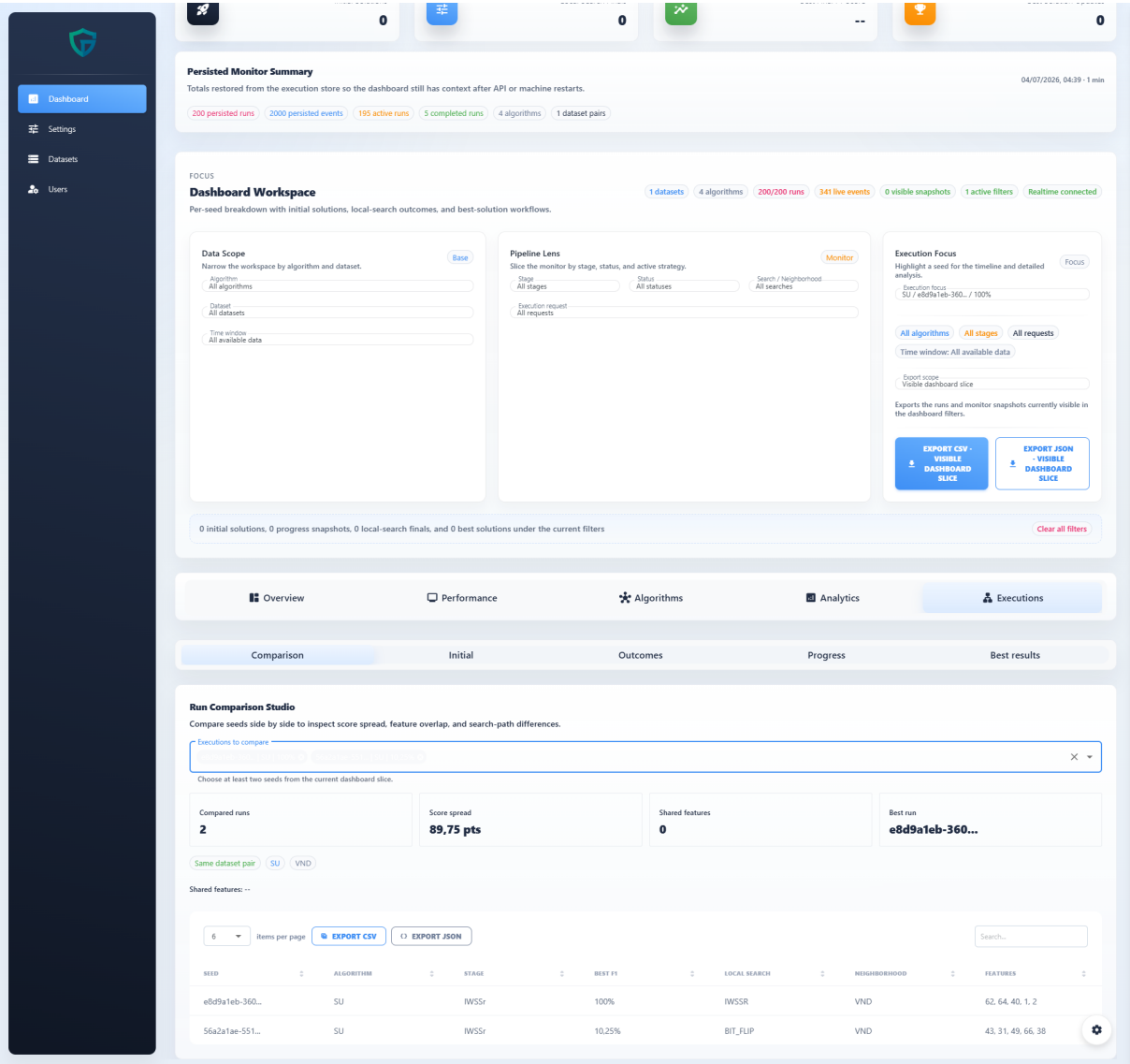


Figure 25 – Comparison with selected seeds.

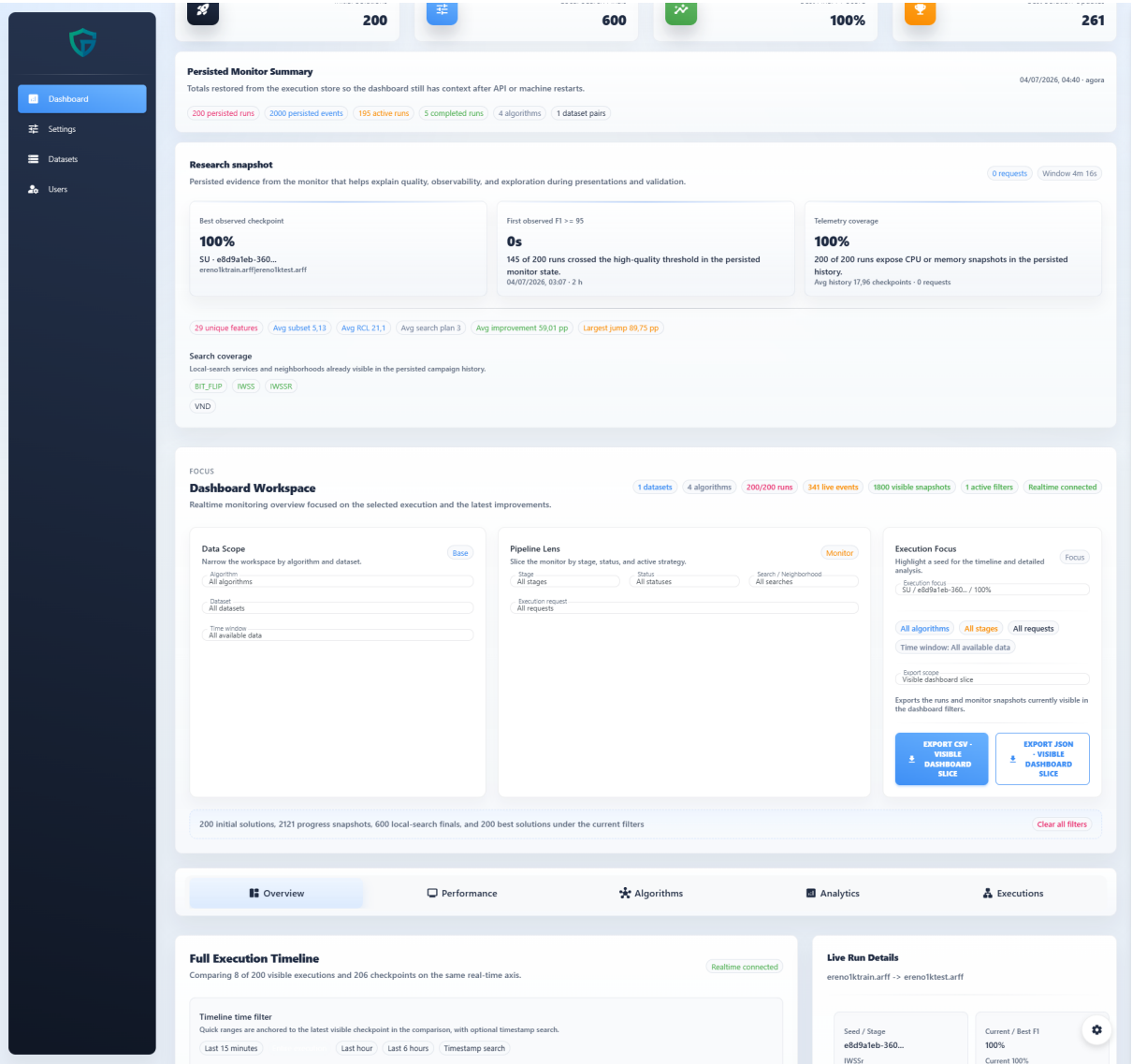


Figure 26 – Details of a specific run.

A.5 Timeline and Results

The captures in this section show how the frontend presents the evolution of solutions over time, supporting interpretation of execution progress and quality. Figures 27, 28, 29, and 30 complete that visual cycle.

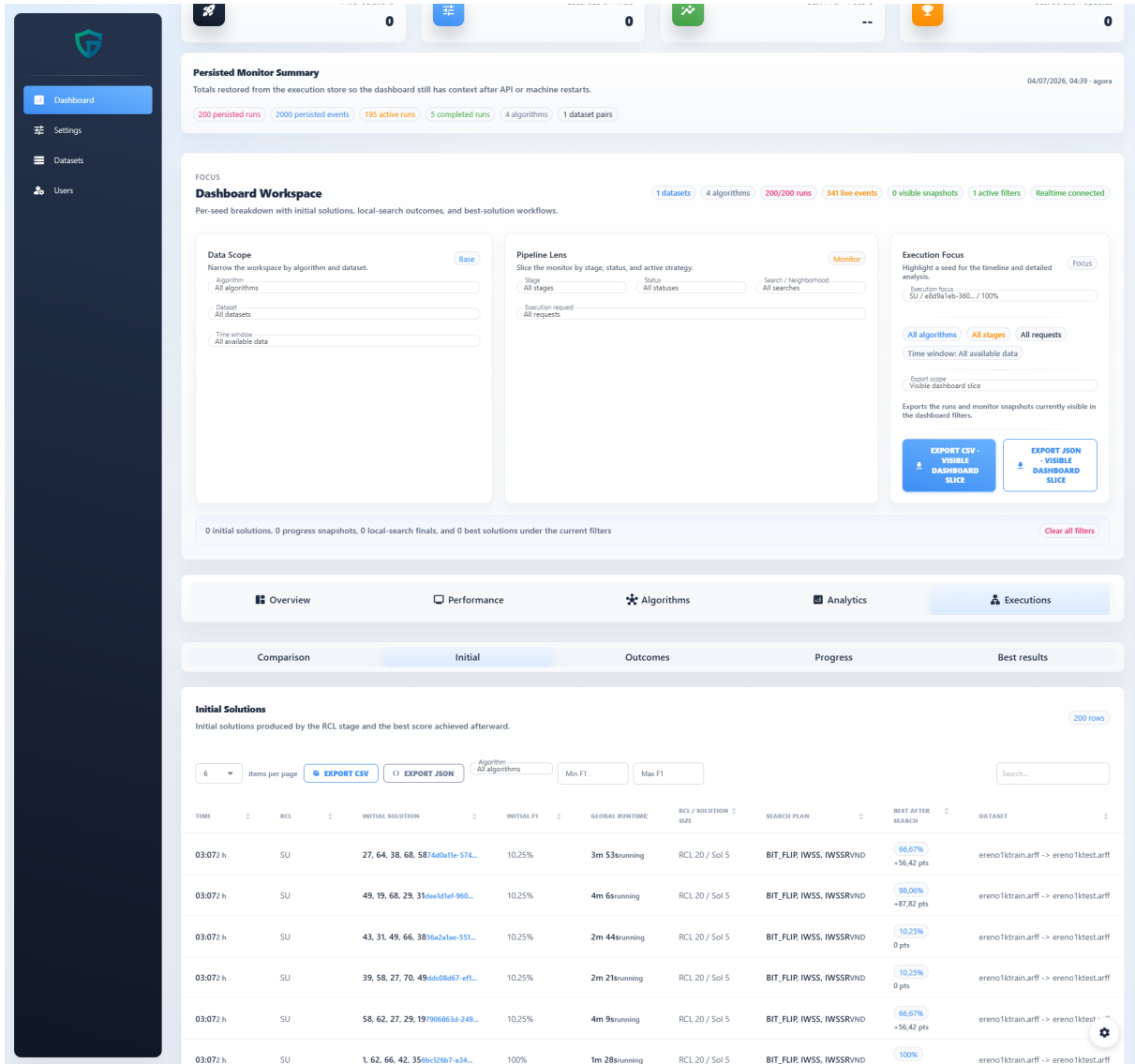


Figure 27 – Initial solutions.

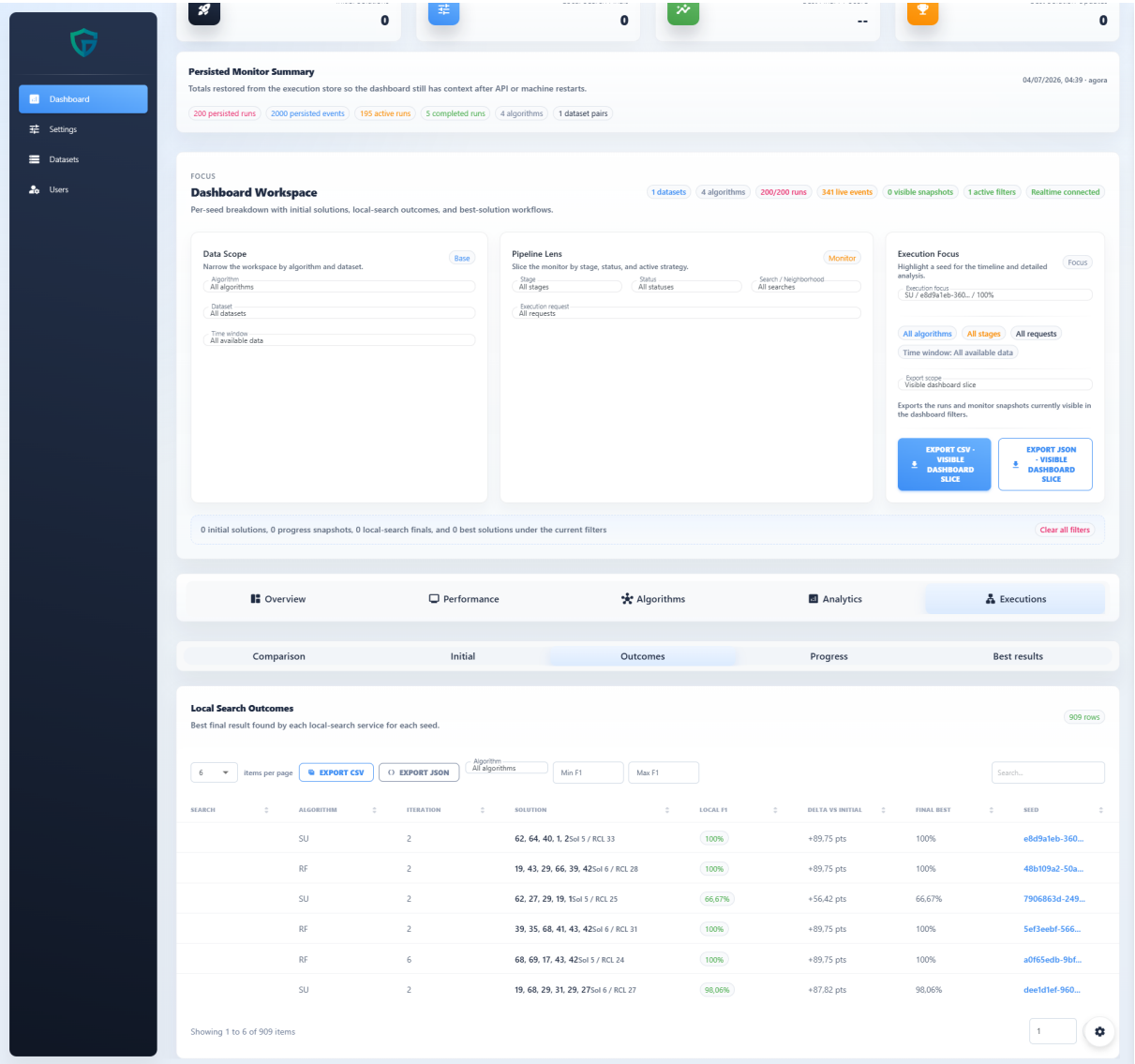


Figure 28 – Consolidated outcomes.

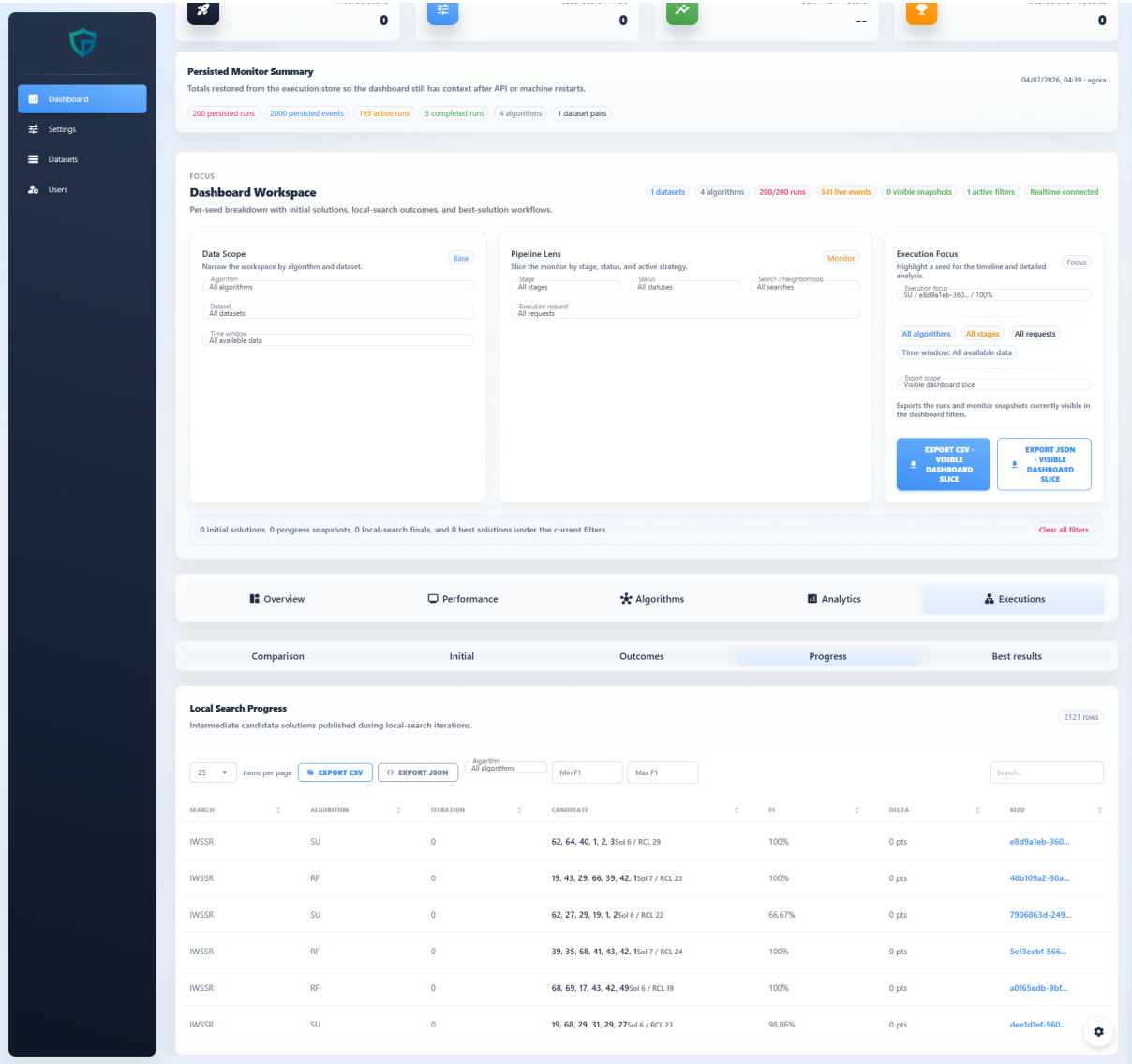


Figure 29 – Execution progress.

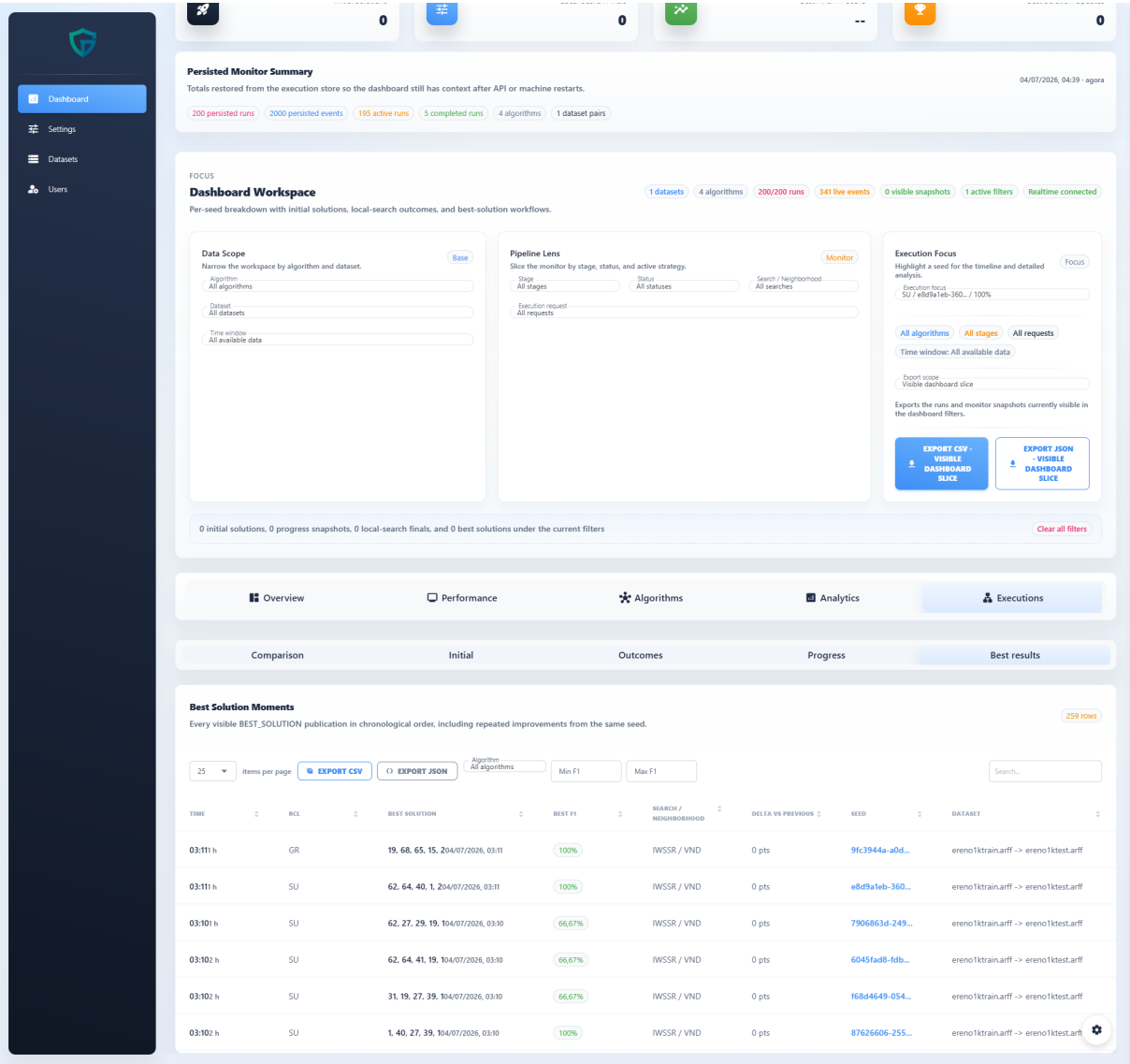


Figure 30 – Best solutions.

A.6 Settings and Administration

Finally, the administrative screens show that the platform also covers operation and maintenance aspects, and Figures 31, 32, 33, 34, 35, 36, and 37 show the administrative set.

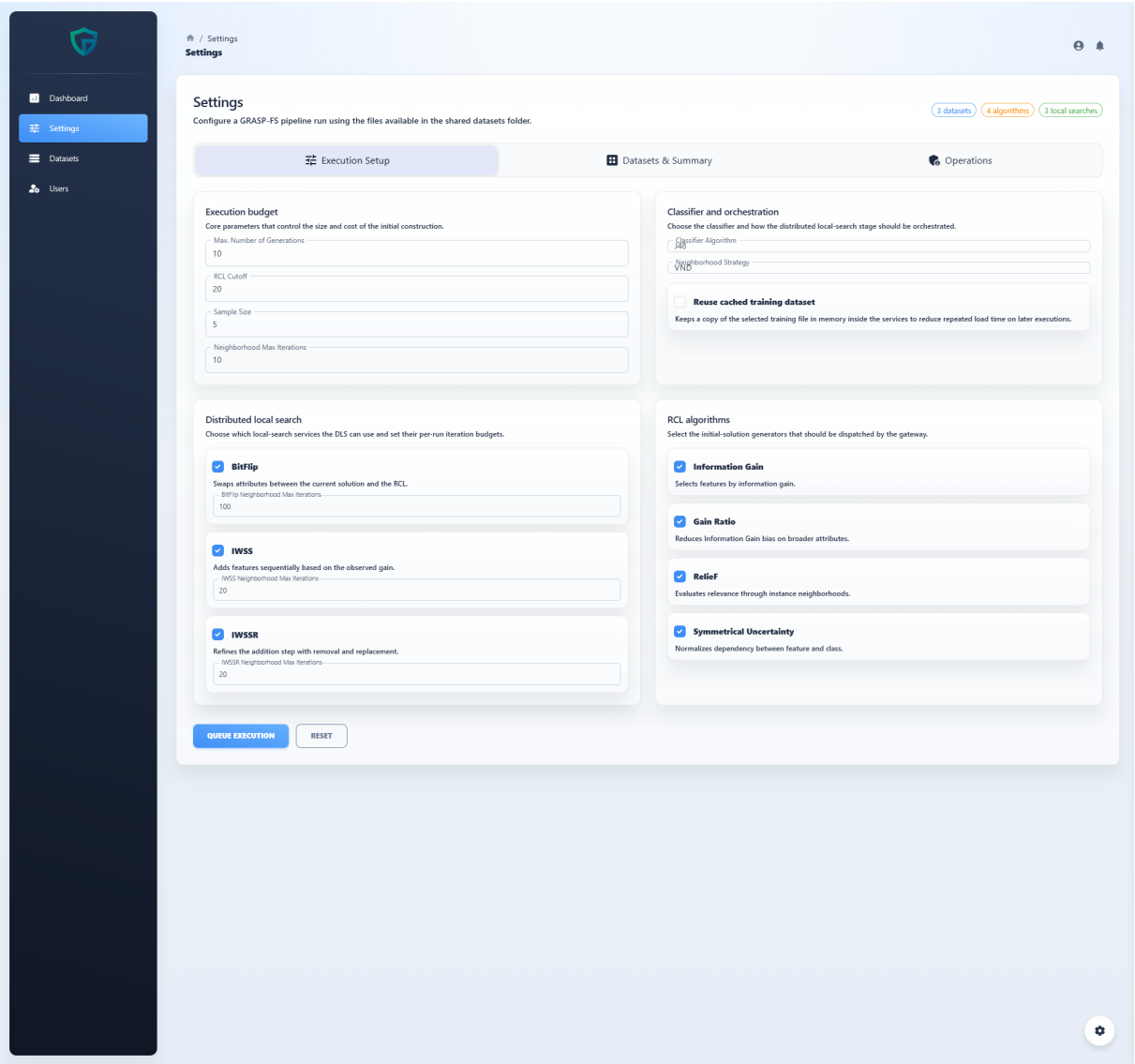


Figure 31 – Execution settings.

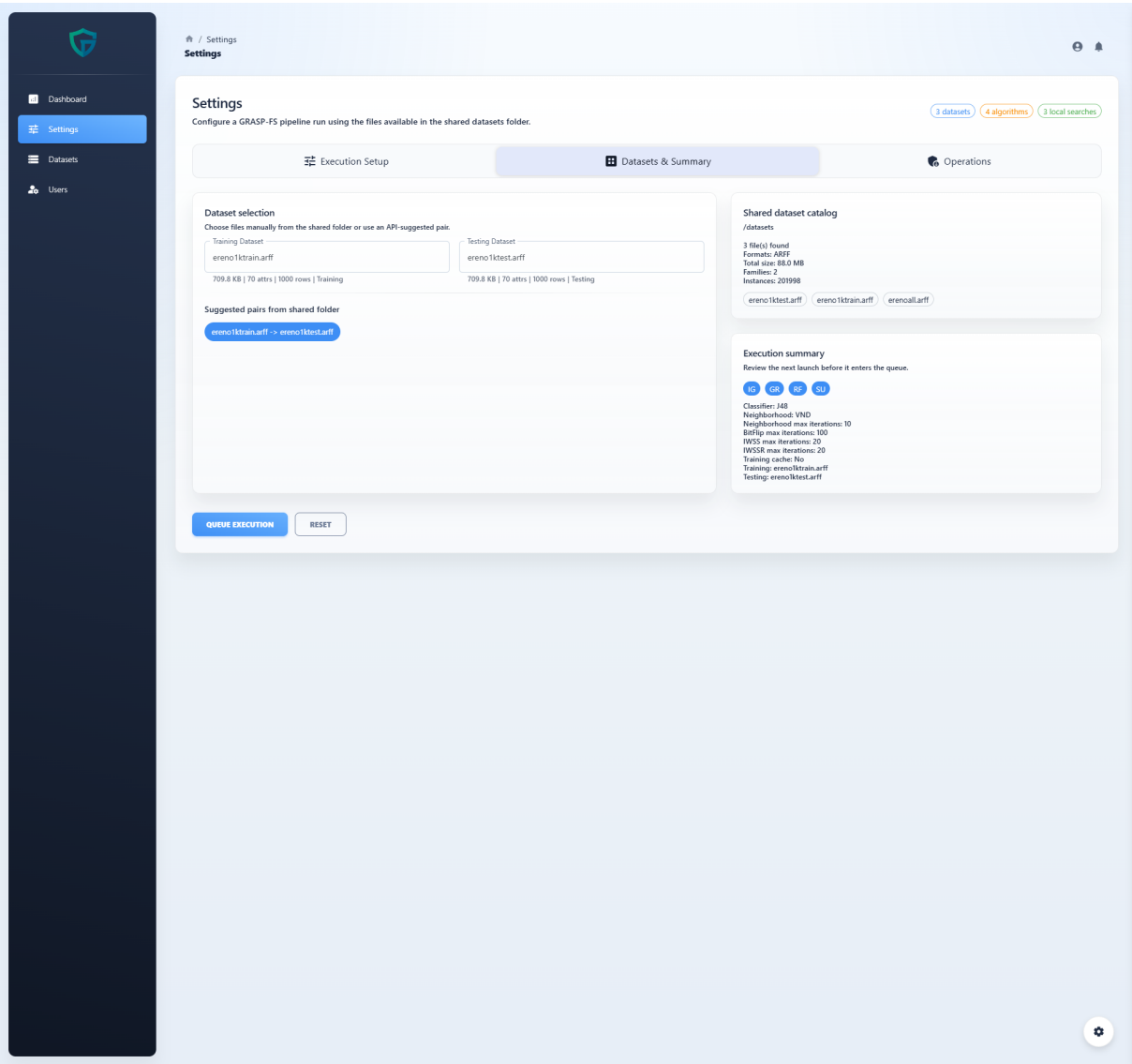


Figure 32 – Dataset settings.

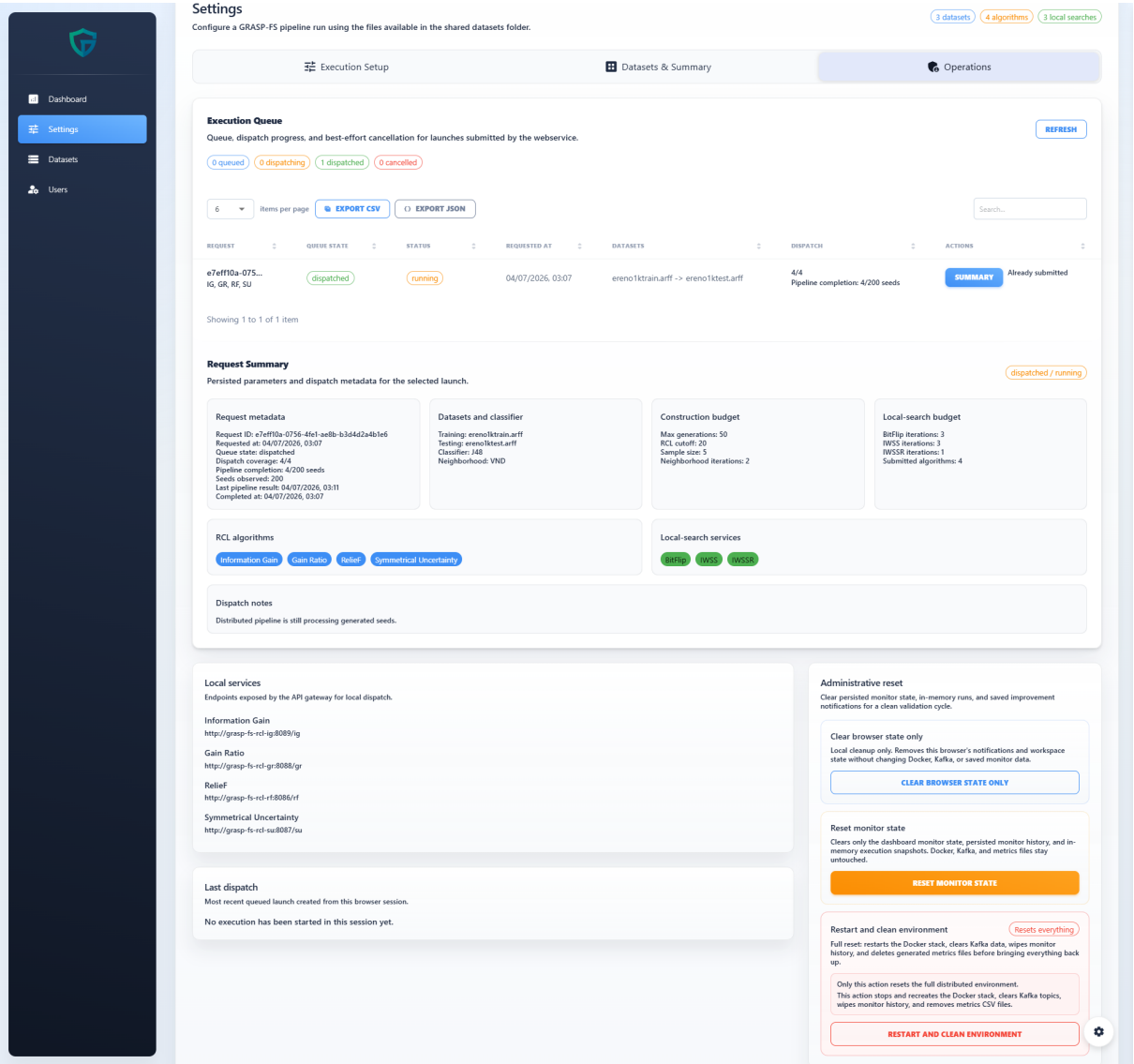


Figure 33 – Operations settings.

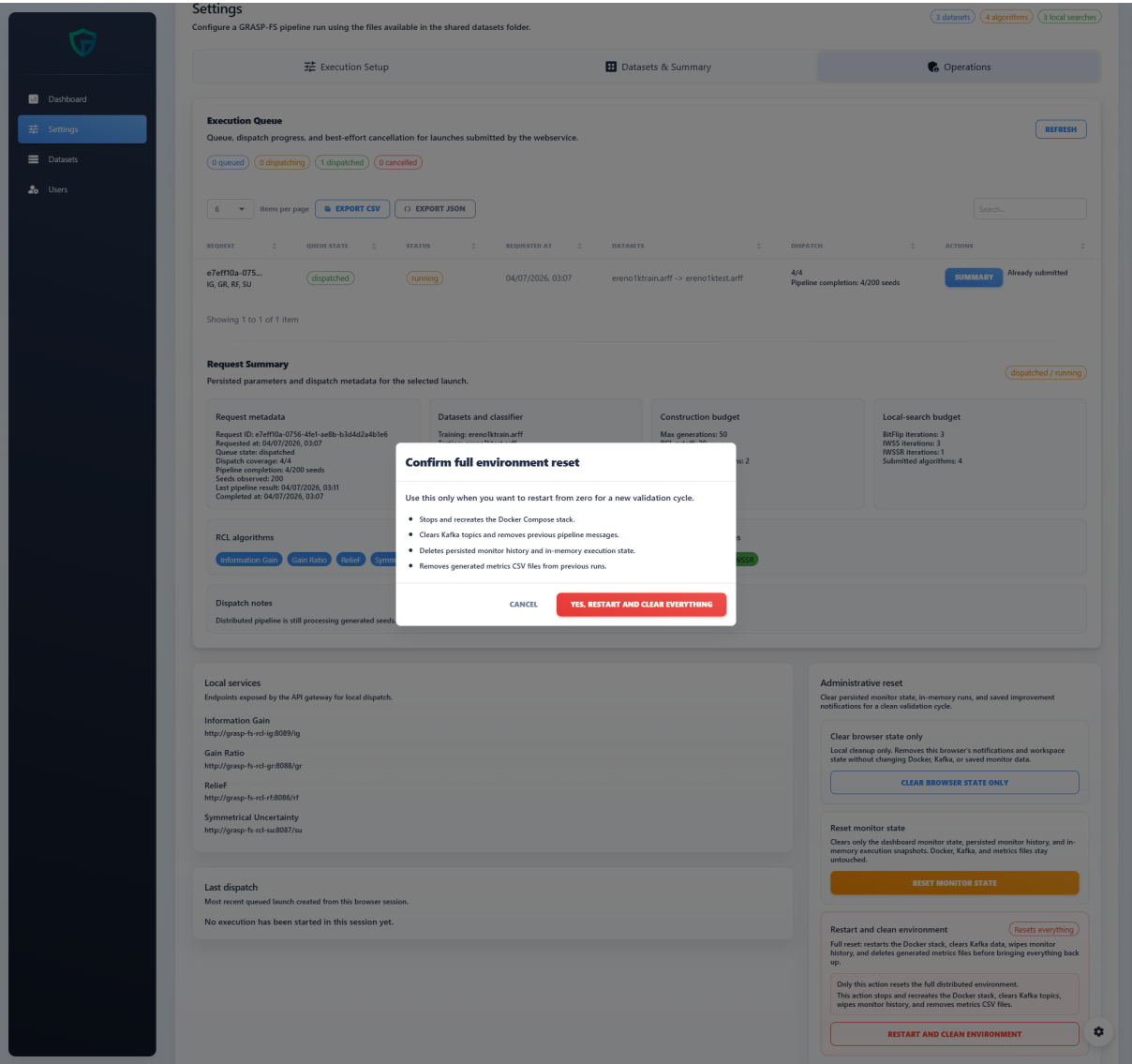


Figure 34 – Full reset dialog.

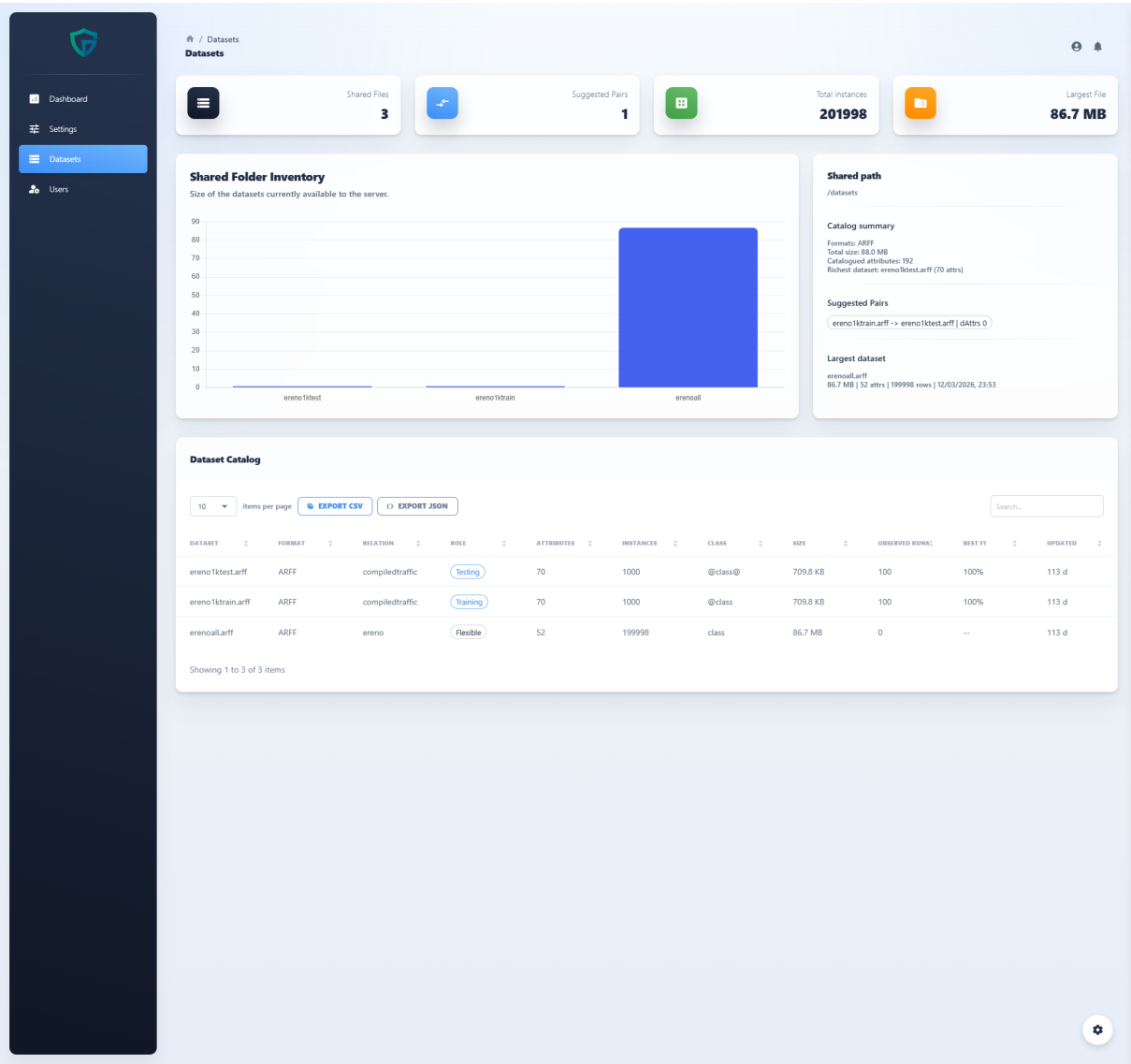


Figure 35 – Dataset list.

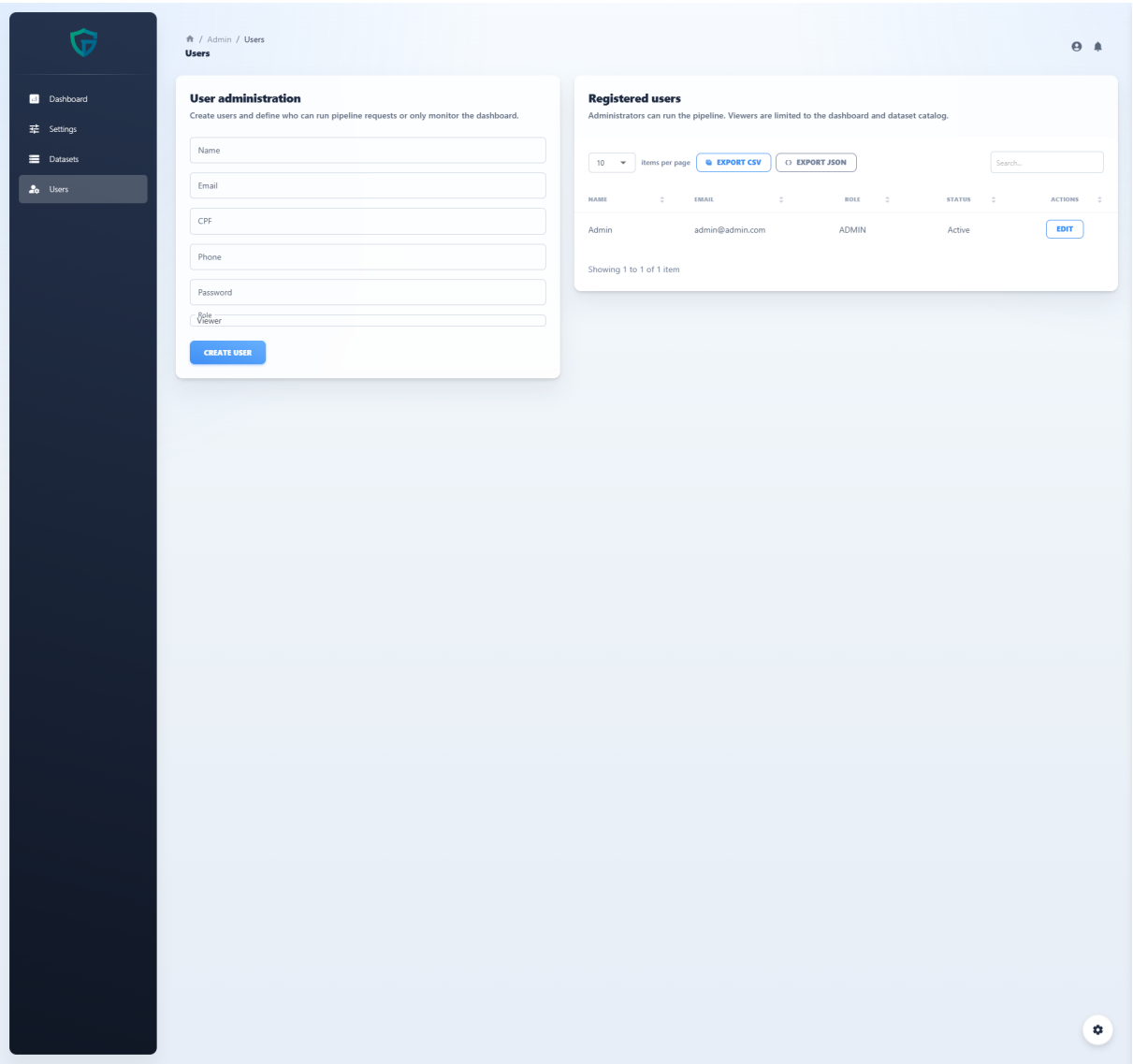


Figure 36 – User list.

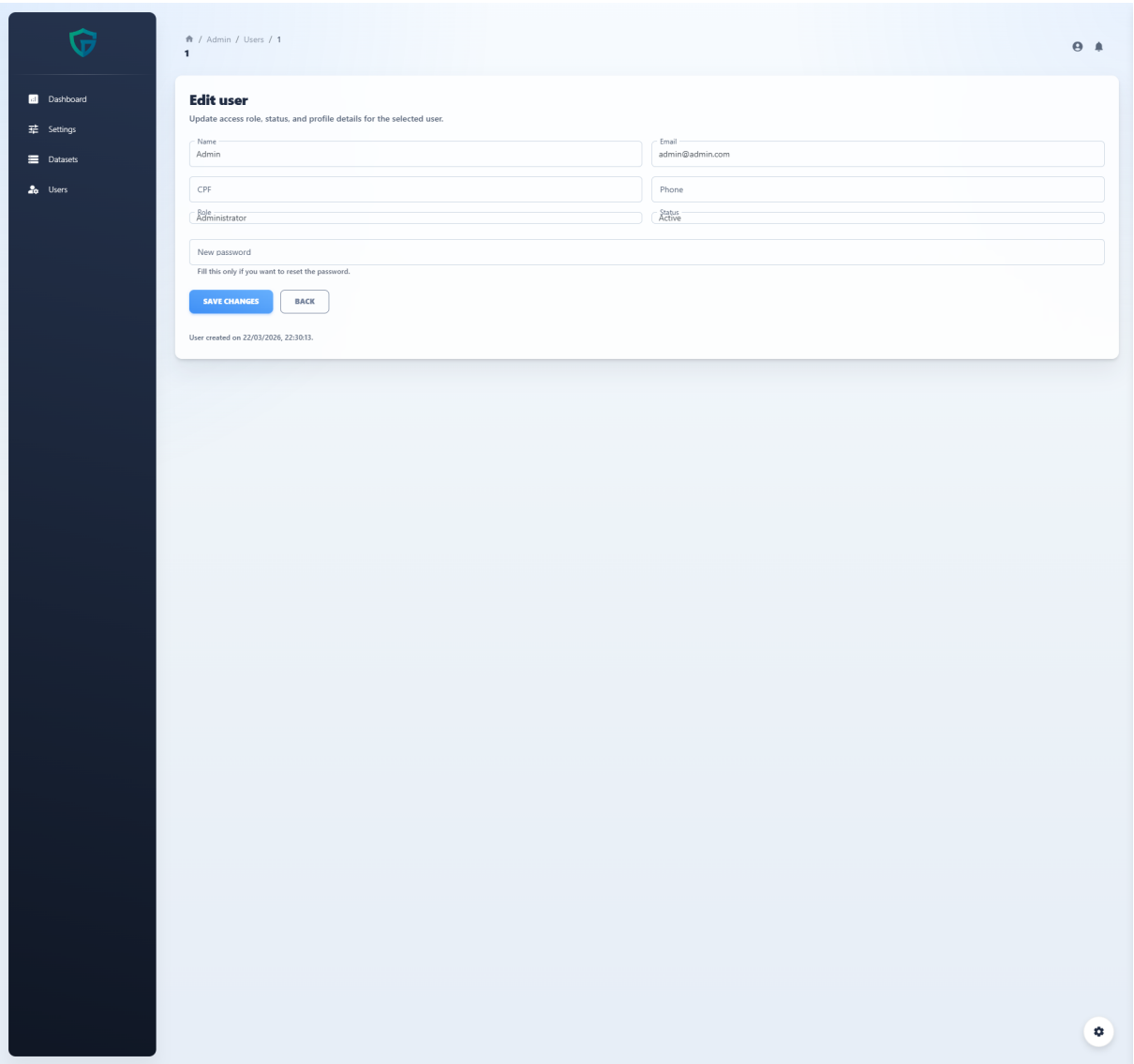


Figure 37 – User editing.

APPENDIX B – VIDEO AND DOCUMENTATION

The platform demo video is included as support material for the defense. Because MP4 files are not always rendered directly by PDF readers, the material is provided by link and is also kept in the project's figure directory. The poster helps identify the video content when the PDF is opened in a viewer that does not embed media. Figures 38 and 39 complete the support material with the demo and API documentation.

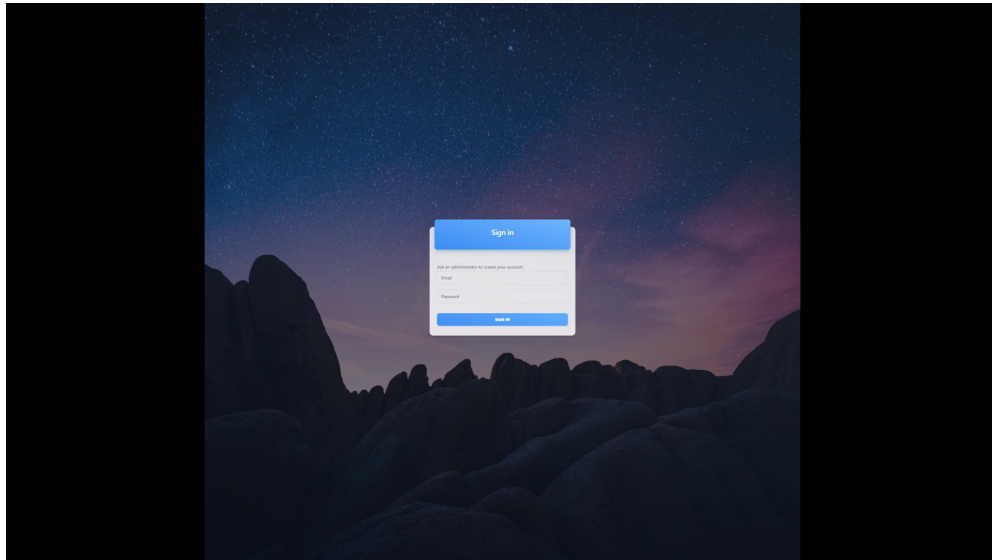


Figure 38 – Poster for the platform demo video.

Demo video: <<https://youtu.be/-uqZPaTOjsU>>

The demonstration video should be deposited in a persistent repository or artefact and identified by the version corresponding to this dissertation.

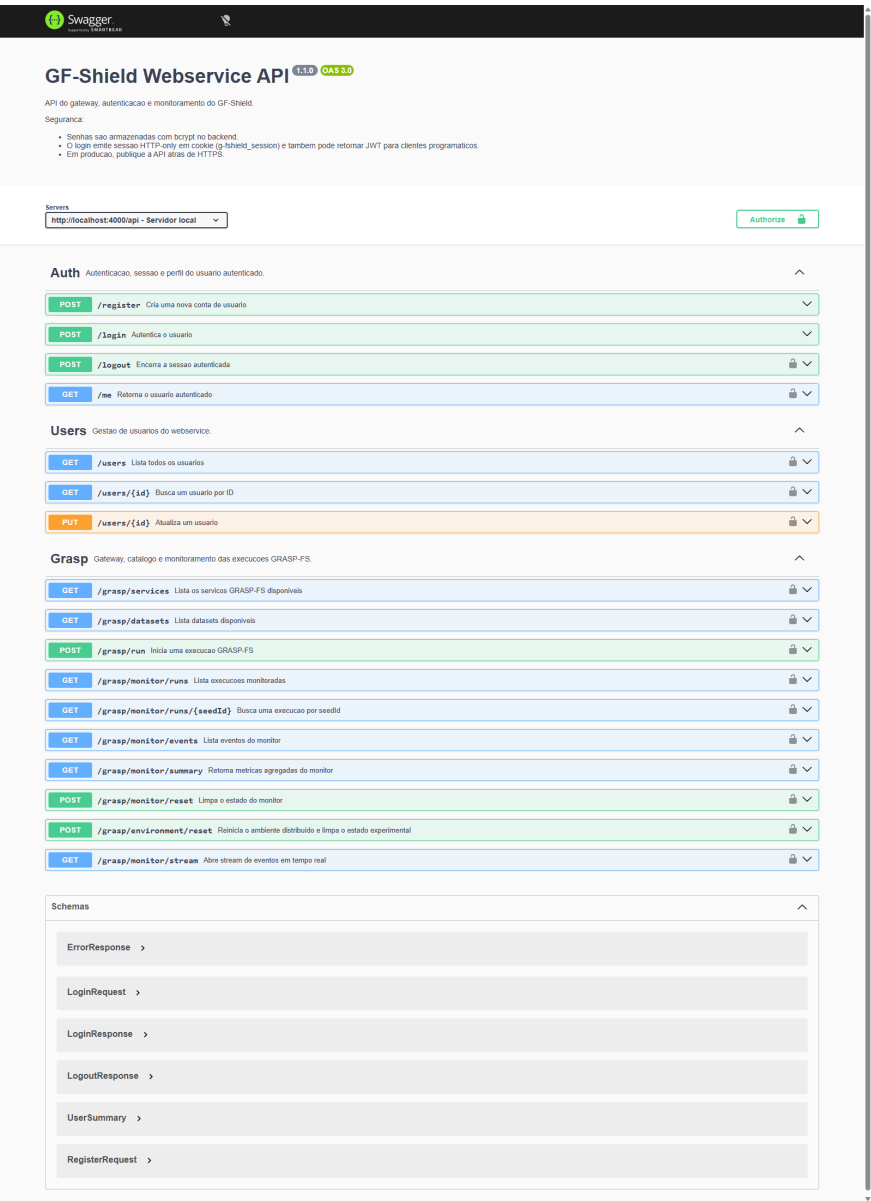


Figure 39 – Swagger documentation of the backend API.

APPENDIX C – ARCHITECTURE DIAGRAMS

This appendix gathers the specific diagrams that complement the main architecture discussion. They connect the development visuals to the screens and operational flows shown in the other sections. Figures 40, 41, 42, and 43 show the complete architectural chain.

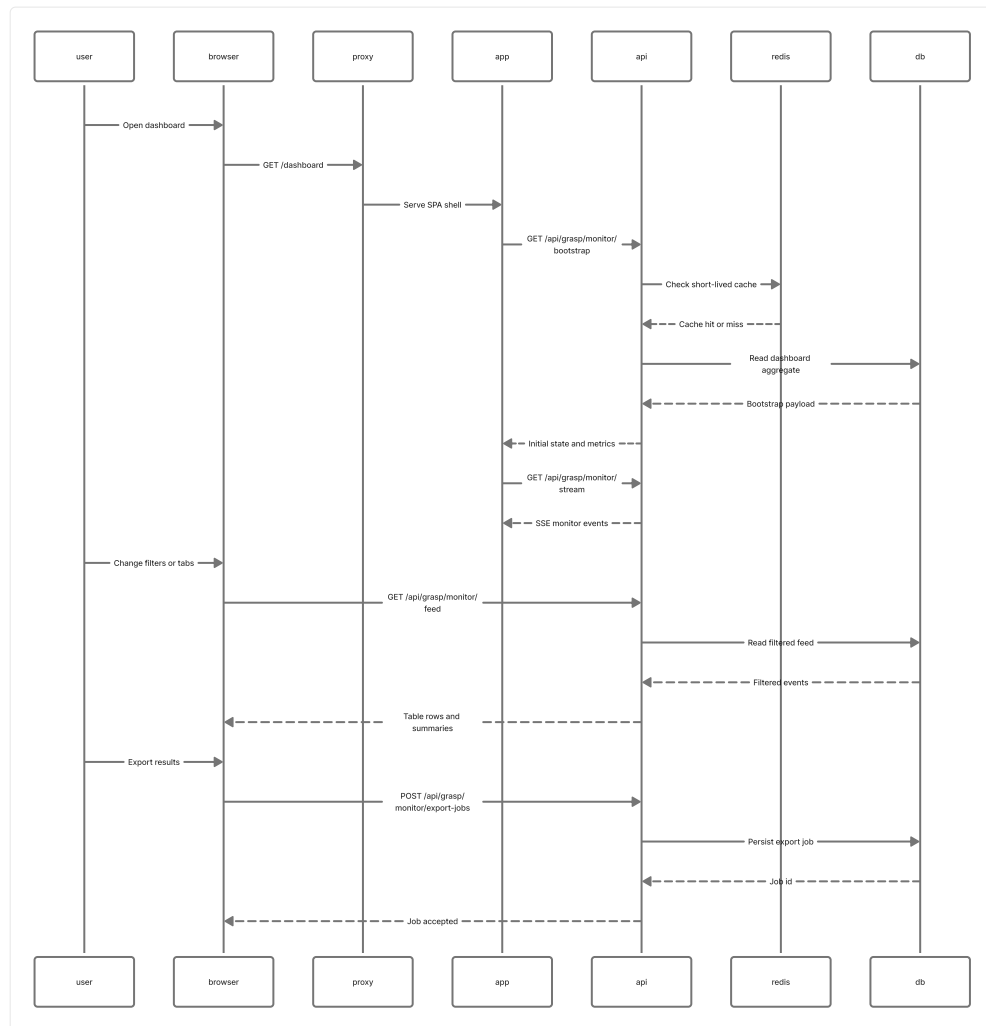


Figure 40 – Frontend flow of the system.

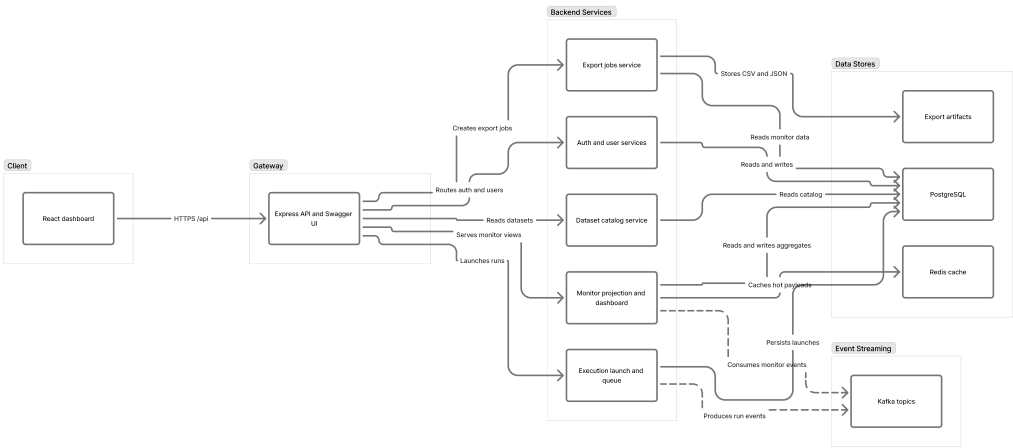


Figure 41 – Backend architecture of the system.

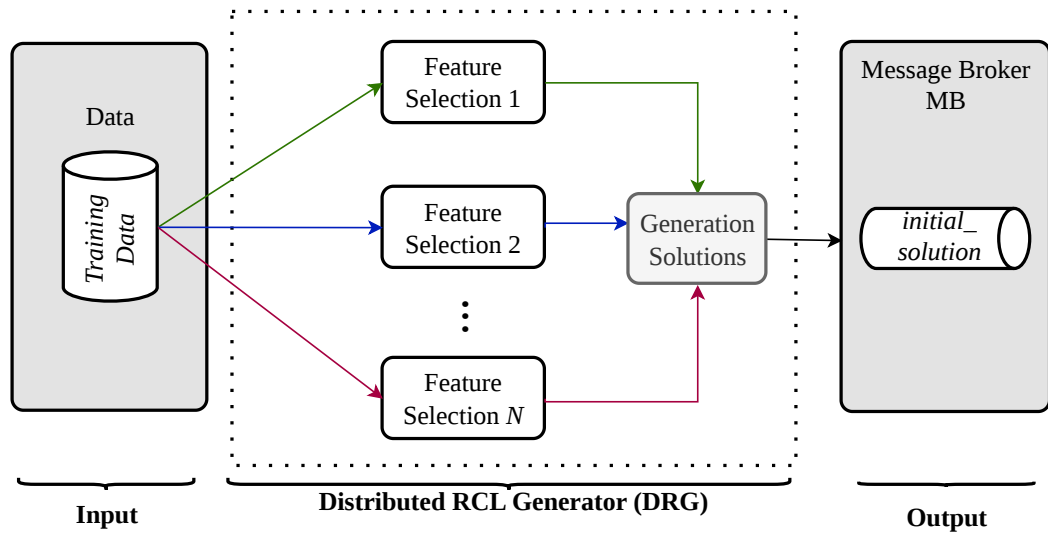


Figure 42 – Generic DRG view with abstract data input.

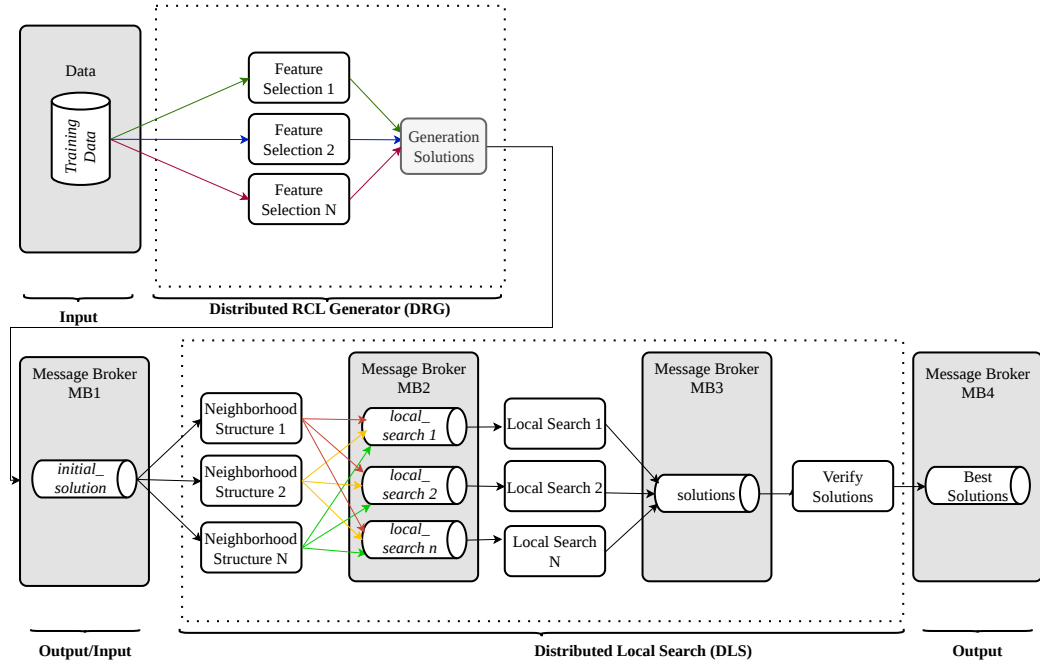


Figure 43 – Complete architecture summary.